Trust at scale.

Engineering that turns complexity into trust.

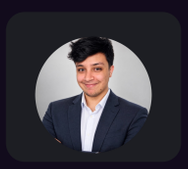
A contribution proposal for Nubank. From the app experience to architecture, delivery and leadership decisions.

320
PAGES

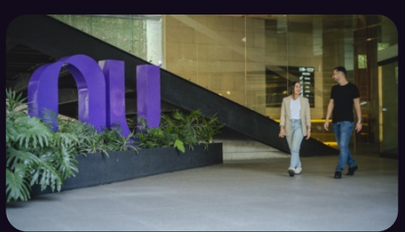
20
REAL CASES

98
CODE BLOCKS

252
PASSING TESTS



Wesley Vieira
Engineering Manager
Staff Software Engineer



CONTENTS

20 reports. 20 proposals.

Select a chapter or use the footer menu. Bookmarks reveal every subtopic.

Part 1 / 2 | Part 2 / 2

Wesley Vieira · a proposal for Nubank 4-17

Experience, evidence, and the contribution I want to make

Foundations and technologies 18-34

Code, contracts, and the limits of each choice

Architectures connecting the cases 35-38

Boundaries, authority, and recovery

Case 01 / BRL 300.00 Pix: clarify the refund 39-51

A trustworthy receipt for a failed attempt.

Case 02 / Incoming Pix, stale balance 52-64

The July 8 incident as an observability test.

Case 03 / Account restriction: review and exit 65-77

Evidence, ownership and reconciliation before release.

Case 04 / Broken camera, secure recovery 78-90

An assisted path that preserves identity controls.

Case 05 / New phone, missing verification code 91-103

Recover contact without confusing email proof with authorization.

Case 06 / New evidence during a dispute 104-116

Receive, scan and assess admissibility as separate stages.

Case 07 / MED: track progress without promising a refund 117-129

Three Pix total BRL 478.55; information needs an explicit state too.

Case 08 / Disputed loan: a decision must remain reviewable 130-142

From access evidence to a dossier, appeal and accountable ownership

Case 09 / Paid statement, uncertain credit: follow every cent 143-155

Receipt, allocation and projection need a shared contract

Case 10 / NuPay after an error message: recover intent before retrying 156-168

An unknown outcome is a financial state, not an invitation to charge again

CONTENTS

From decisions to delivery.

Select a chapter or use the footer menu. Bookmarks reveal every subtopic.

Part 1 / 2 | Part 2 / 2

Case 11 / A refund in three entries: the statement must explain its own sum 169-181

Provenance, revision and accessibility before assuming an accounting defect

Case 12 / Delivered in tracking, missing in hand: separate delivery from possession 182-194

Card generations, source evidence and an exception with an owner

Case 13 / A virtual card cap is not available credit 195-207

Explain the constraint without weakening financial authorization

Case 14 / Early repayment received, execution delayed: make the plan verifiable 208-220

Quote, expiry, allocation and discount with a single receipt

Case 15 / Money Boxes: visible money, reliable withdrawal 221-233

Resilience · Financial authority

Case 16 / Portability: eligible does not mean overdue 234-246

Campaigns · Explainability · Custody

Case 17 / Ultravioleta: one choice, one balance 247-259

Benefits · Consent · Units

Case 18 / NuCel: confirmation across two carriers 260-272

Number portability · Events · Accountability

Case 19 / Global Account: purpose before FX 273-285

Capabilities · Quotes · Expectations

Case 20 / NuPay: confirmed payment, missing order 286-298

Reconciliation · Partners · Continuity

Leadership, value, and execution 299-313

How to turn hypotheses into decisions and outcomes

Sources and credits 314-320

Public references and visual provenance

OPENING

The work I want to build.

Photograph published by Nubank: a workplace reflecting the company's identity.

I want to contribute to a Nubank where simplicity of experience is supported by technical rigor. My proposal starts with concrete journeys: paying without ambiguity, recovering access securely, understanding a decision, and getting help when an operation leaves the expected path. Each requires product, architecture, and organization to work as one system.

This book makes my candidacy open to examination. It contains twenty studies of public signals, accompanied by code, contracts, architecture diagrams, tests, and execution decisions. Its depth lies in the boundaries: where a guarantee ends, who can decide, how to recover from failure, and what evidence justifies expanding a change.

Wesley Vieira

Application for Engineering Manager or Staff Software Engineer. Independent study; not an official Nubank publication.

Nubank: About Nu

COVER LETTER

Dear Nubank team,

Editorial photograph published by Nubank: professionals in a meeting room.

I want to join Nubank because I am interested in the work between a simple experience and the engineering required to sustain it. For someone using the app, a Pix payment, a dispute, or access recovery is more than a flow. It involves money, time, and trust. I want to help make those journeys understandable when everything works and keep offering a safe next step when something goes wrong.

Today at Itaú, I am responsible for the technical direction of two squads, with more than 15 engineers and more than 20 fraud-prevention systems and services. My background also includes engineering leadership in credit at Bradesco and technical product management at Itaú. These experiences have taught me to bring architecture, business context, and people development together while keeping sight of those who depend on the system.

My practical foundation includes Java/JVM, AWS, messaging, systems integration, and legacy modernization. I also work with data, security, and applied AI. Beyond bringing a list of technologies, I want to contribute the ability to frame problems, discuss alternatives, and build solutions that fit the team's context and operational capacity.

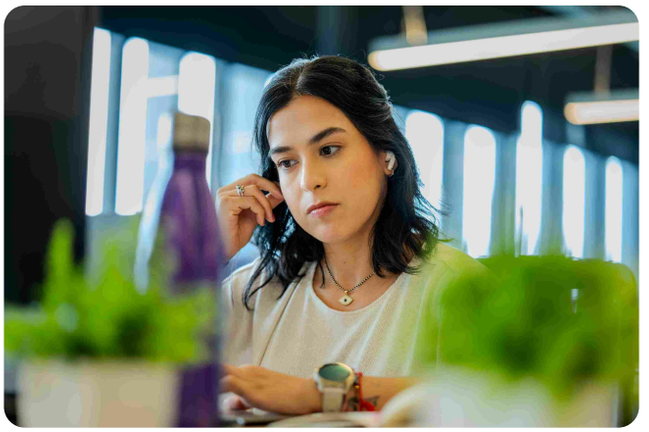
My proposal

Combine depth in financial systems, people development, and accountability for journey outcomes.

Wesley Vieira's portfolio
Nubank: editorial photograph

COVER LETTER

A technical conversation before the interview.



Editorial photograph published by Nubank: a workplace. The image does not depict the candidate.

I prepared this material to make that intention open to examination. I used public information about Nubank and customer reports to organize twenty studies. Each distinguishes what was observed, what remains a hypothesis, and what I would propose validating. I do not know the internal backlog and expect to find existing controls, different priorities, and good reasons to revise my initial ideas.

As an Engineering Manager, I want to help people grow, take ownership, and deliver with clarity. As a Staff Software Engineer, I want to improve technical decisions that cross team boundaries by making contracts, risks, and paths for evolution more explicit. In both roles, my commitment is to collaboration, quality, and results.

This book is a sample of how I would begin a conversation: listen to the pain, make the decision concrete, test the hypothesis, and learn from the result. I would like to build the next version together with the Nubank team.

I do not start from the assumption that Nubank needs to rebuild what it already does well. I start with a willingness to learn from its published capabilities, formulate a useful hypothesis, and test it with the team. The relevant judgment is not how many technologies fit into a diagram; it is how much uncertainty we can remove from the next decision.

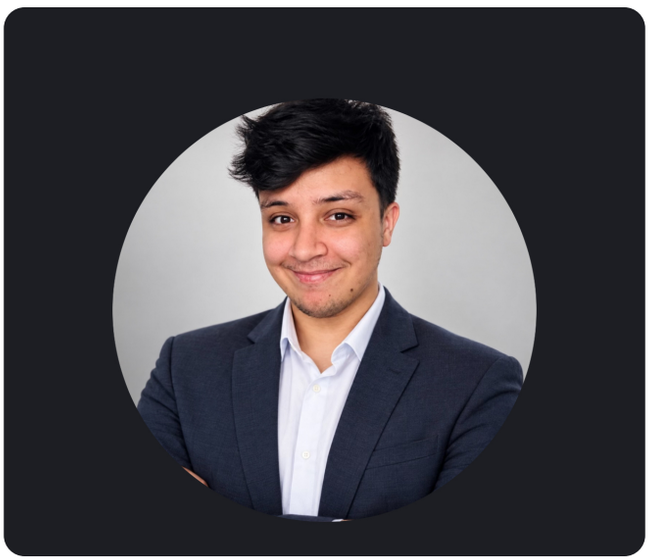
How to read my application

Career evidence supports my experience. The labs demonstrate executable reasoning. The cases show how that experience could serve Nubank's context.

Wesley Vieira's portfolio
Nubank: editorial photograph

INTRODUCTION

I am Wesley Vieira.



Professional photograph available in my portfolio.

I am a Lead Software Engineer working across financial services, fraud prevention, credit, and technical product. Today at Itaú, I lead the technical direction of two squads, with more than fifteen engineers and more than twenty systems and services. My background includes engineering leadership at Bradesco and technical product management at Itaú.

My practical foundation combines Java/JVM, AWS, messaging, systems integration, and legacy modernization. I use that experience to connect software behavior to business needs: what must remain correct, which risks we accept, and how the people operating the solution will recognize a failure.

What I bring to the conversation

Experience with critical financial journeys, the ability to work across team boundaries, and an interest in developing people who make better decisions autonomously.

Background and portfolio

EXPERIENCE

A career with clearly defined scope.

Period / context	Responsibility	Relevant contribution
Itaú · since Jun 2026	Lead Software Engineer	Technical direction of 2 squads and 20+ fraud-prevention systems.
Bradesco · Feb 2025-May 2026	Lead Software Engineer	Architecture and delivery across 3 credit and channel squads.
Itaú · Nov 2023-Oct 2024	Senior Technical Product Manager	Multidisciplinary leadership and protection priorities.

These scopes complement one another. In fraud prevention, technical decisions affect losses and friction. In credit, policies, channels, and integrations must remain coherent through change. In technical product, the challenge is organizing work so that different teams understand the same outcome and can negotiate trade-offs.

Technical direction and people management are distinct responsibilities. The fifteen or more engineers in my current scope are not presented as direct reports. In my previous technical product role, my profile describes leadership of multidisciplinary squads, discovery, priorities, and stakeholder alignment, without a confirmed direct-report count. That precision is part of how I want to lead: clear expectations, explicit responsibility, and evidence that supports the claim.

Modernization

My portfolio describes the evolution of a legacy environment with approximately 1,900 files. The number represents scope; it does not imply a completed migration or audited savings.

Candidate-reported background

CAREER EVIDENCE

Recognition is a collective achievement.

Record of the 21st Walther Moreira Salles Award, available in the candidate's portfolio.

My portfolio records recognition at the 21st Walther Moreira Salles Award in December 2023, associated with the WhatsApp fraud-alert project. The photograph shows a team achievement. For a leadership application, the most relevant aspect is the ability to bring technology, communication, and customer protection together around a real problem.

What I bring from that experience is not a recipe for copying a channel. It is the discipline of asking whether a message arrives in time, whether the person can recognize a legitimate intention, whether the response informs the risk decision, and whether recovery is available when automation does not resolve the issue. Communication also has contracts, observability, and operational ownership.

Evidence and scope

The image documents the recognition presented in the portfolio. On its own, it does not substantiate the financial figures reported by the candidate.

Photographs, recognition, and projects

IMPACT HISTORY

Impact with honest attribution.

Reported outcome	Nature	How to discuss it in the interview
R$ 350 million	Fraud mitigation associated with WhatsApp alerts	Clarify the period, baseline, team contribution, and calculation.
R$ 300 million	Loss mitigation in an ML initiative	Distinguish the counterfactual, prevention, and recovery.
R$ 150 million+	Revenue associated with a credit initiative	Distinguish revenue, margin, and causality.

These figures come from my self-reported background. They are not Nubank results, were not audited in this work, and must not be added together as if they represented a single saving. Initiatives may share periods, populations, or mechanisms of contribution. A good hiring conversation needs to explore the role played, the decision made, and the measurement method.

I would present each project with its prior context, my personal responsibility, contributions from other areas, rejected alternatives, and available evidence. If a measure came from the business, I would explain who produced it and what it does not cover. If a result depended on rollout or a policy change, I would show the sequence rather than attribute everything to software.

Working principle

A strong number should come with an equally strong explanation. The rigor I apply to my application is the same rigor I propose for product decisions.

Self-reported portfolio outcomes

ENGINEERING MANAGER

Why hire me as an EM.

Nubank editorial photograph showing collaboration between professionals.

As an Engineering Manager, my contribution is to create an environment where people understand the problem, have autonomy proportionate to risk, and can complete quality work. This requires more than tracking delivery: capacity must be visible, overload recognized, technical leadership developed, and dependencies resolved before they become permanent delays.

My experience in technical product and engineering lets me connect business objectives to implementation decisions without making myself the center of everything. I want to work with strong tech leads, establish expectations by level, offer specific feedback, and reserve space for learning. Management cadence should help the team decide; if it only produces reports, it needs to change.

Responsibility	Evidence in this book	Outcome to pursue
People	Delegation, mentoring, and capacity	Greater autonomy with accountability.
Execution	Pilots, stop criteria, and dependencies	Less work started without completion.
Product	Journey and value metrics	Verifiable improvement for customers.

Engineering Manager: official job description

STAFF SOFTWARE ENGINEER

Why hire me as a Staff engineer.

As a Staff Software Engineer, I want to improve decisions that cross team boundaries: event semantics, data authority, transaction boundaries, migration strategy, and how a journey is observed. My role is to make a problem clear enough for others to act safely without needing me in every review.

My foundation in Java/JVM and financial systems transfers to the functional reasoning demonstrated in this book. At the same time, I do not present my study of Clojure or Dart as professional experience already acquired at Nubank. I show original code, executed tests, and integration questions so that the technical discussion has something concrete to examine.

Scope	Typical decision	Artifact I leave behind
Domain	Which state is authoritative?	Invariants and a state machine.
Across teams	Who changes the contract, and how?	ADR, version, and compatibility plan.
Operations	How do we detect and recover?	SLO, failure test, and procedure.

The official Staff role consulted mentions Clojure, Kafka, AWS, Datomic/DynamoDB, Prometheus, and Kubernetes. The chapters use this context as a starting point. Complementary tools appear where they help test a hypothesis; they are not presented as universal company requirements.

Staff Software Engineer: official job description

RESEARCH METHOD

How to recognize each type of evidence.

Label	Meaning	Boundary
PUBLIC RECORD	Identified complaint or news report, with a real screenshot.	Not an independently reproduced bank incident.
COMPANY RESPONSE	Nubank's explanation and the available outcome.	A reply does not necessarily mean consumer-confirmed resolution.
IMPLEMENTED	A local executable model with tests.	No connection to Nubank production.
PROPOSED / MODELED	Original design, roadmap and financial scenarios.	No claimed realized return or internal approval.

All twenty studies now begin with specific public problems: eighteen individual Reclame AQUI complaints and two news reports. Each includes its source, date, company response and available outcome. Resolved cases remain identified as resolved; disagreement between parties remains visible. Selection is purposive, not a representative survey or a ranking of failure frequency.

Screenshots contain actual captured pixels, cropped for readability. The news video and official NuCel tutorial have separate credits and purposes. A product tutorial illustrates the intended journey; it does not prove the reported incident. No imagined app screen is used as evidence.

Research date

23 October 2026. The public record can change. Case incidence, causal effect and marginal costs require internal measurement.

PRODUCT AND EVIDENCE

The app: public promise and experience.

Official promotional material for the Nubank app on Google Play.

The public promise of simplicity helps define the unit of analysis: a person trying to complete a financial task. In this book, the app is more than a presentation layer. It participates in states, recovery decisions, accessibility, risk communication, and the perception of time.

Official images show the product's public presentation. Case evidence consists of actual captures of public complaints and reports, with clear provenance. These are not screenshots of an account accessed by the author. The source, statement and proposed engineering response remain separate.

Nubank on Google Play

READING MAP

Twenty questions, one discipline.

The book follows money and the decisions around it. It starts with a failed Pix refund, a balance that did not update and an account restriction reported in the press. It then examines identity recovery, evidence in disputes and the visibility of Pix recovery. The next group follows billing, delivery and card authorization. The final group covers investments, benefits, telecom, currency exchange and merchant order reconciliation.

Reading path	Cases	Engineering focus
Money and access	01-07	Authoritative state, recovery and evidence.
Credit and cards	08-14	Consent, reconciliation, delivery and quotes.
Product boundaries	15-20	Liquidity, eligibility, units and partners.

Every case contains an evidence dossier, a diagnosis that can be challenged, a distinct architecture, four code blocks, an executable lab, operational indicators, three financial scenarios and a staged roadmap. The roadmap names owners and exit gates. Financial outputs are conditional calculations, not the result of multiplying a complaint by the entire customer base.

How to examine the proposal

Open the source, inspect the outcome, run the lab, challenge the financial inputs and decide what evidence a pilot must produce.

TECHNOLOGIES

Documented stack, justified choices.

clojure	kafka kafka	aws aws	Datomic datomic
prometheus	kubernetes	flutter	dart

The official Staff job description consulted highlights Clojure, Kafka, AWS, Datomic/DynamoDB, Prometheus, and Kubernetes. Nubank's technical publications also document Flutter and the importance of standardized choices. This context guides the examples' language and the structure of the proposals.

Java/JVM provides a bridge to my professional experience. Python supports reproducible labs with minimal setup. PostgreSQL, Redis, OpenTelemetry, Grafana, Terraform, and TypeScript appear as options or teaching tools, with their reasons and limits made explicit. A logo identifies the technology being discussed, not a claim of universal use at Nubank.

This is not a statistical ranking of the "most requested" technologies. It is alignment with identified official sources. In a real team, I would confirm versions, canonical libraries, internal platforms, and constraints before introducing components. Technical variety creates value only when it addresses a concrete need.

Stack in the official job description
Nubank: The value of canonicity

ADAPTATION PLAN

From my experience to the team's stack.

Starting experience	Application in context	Evidence I would seek
Java/JVM and AWS	Clojure services and cloud operations	Paired reviews of code and incidents.
Messaging and integration	Kafka contracts and consumers	Controlled replays and compatibility.
Fraud and credit	Authority, risk, and money	Boundary cases and policy review.
Product and management	Experiments and execution	Baseline, owner, and decision criterion.

Joining a new organization means learning how it makes decisions. I would start with one complete flow: from the gesture in the app to the service, event, persistence, and support. From there, I would identify what is standard, where exceptions exist, and which commitments were made to sustain current operations.

The functional lab included in this edition demonstrates that approach. It runs in Clojure and separates state transformation from external effects. This does not replace experience with the internal platform; it narrows the distance between saying I learn a language and showing an artifact that can be reviewed. The same standard applies to management: explain how I would learn from the team before promising reorganization.

The first delivery I would aim for

A small, observable, reversible improvement built with people who know the domain. The initial objective is to earn context and trust through the work.

FOUNDATIONS · CLOJURE

Clojure: decisions as data.

The functional core receives one state and returns another. An invalid transition produces an explicit error; it does not call a network, queue, or database. This separation allows the contract to be discussed before infrastructure is chosen. Within the model, versioning prevents an old intent from changing a newer state.

CLOJURE / ORIGINAL PROPOSAL

```clojure
(def transitions
  {:received #{:pending :rejected}
   :pending #{:settled :rejected}
   :settled #{}
   :rejected #{}})

(defn advance [db key expected-version next-status]
  (let [{:keys [status version] :as current} (get db key)]
    (when-not current
      (throw (ex-info "unknown operation" {:reason :not-found})))
    (when-not (= expected-version version)
      (throw (ex-info "stale writer" {:reason :version-conflict})))
    (when-not (contains? (get transitions status #{}) next-status)
      (throw (ex-info "illegal transition"
                      {:reason :illegal-transition})))
    (assoc db key (assoc current :status next-status
                         :version (inc version)))))
```

Excerpt from decision_core.clj; local execution verified. In-memory model without networking.

The excerpt is part of decision_core.clj, executed in Clojure 1.12.0 on Java 17. The complete file has 11 tests and 22 assertions. Value immutability provides neither durable persistence nor distributed exclusion: when integrating a database, version comparison and writing must share the same atomic boundary.

Official technology documentation

FOUNDATIONS · CLOJURE

Clojure: repetition does not create another intent.

The idempotency key identifies intent; the fingerprint identifies its relevant content. Repeating the same key with the same value preserves state. Reusing it with a different value is a conflict, not a new purchase. In the synthetic example, key scope is the in-memory database; production would need customer, operation, and retention policy.

CLOJURE / ORIGINAL PROPOSAL

```clojure
(defn register [db {:keys [key amount currency] :as cmd}]
  (when-not (valid-command? cmd)
    (throw (ex-info "invalid command"
                    {:reason :invalid-command})))
  (let [fingerprint [amount currency]
        existing (get db key)]
    (cond
      (nil? existing)
      (assoc db key {:fingerprint fingerprint
                     :status :received :version 0})
      (= fingerprint (:fingerprint existing)) db
      :else
      (throw (ex-info "key reused with different payload"
                      {:reason :key-conflict})))))

;; Em um único processo: atualização atômica do modelo.
;; Nunca adicionar efeitos externos à função de swap!.
(def state (atom {}))
(swap! state register
       {:key "synthetic-001" :amount 1250 :currency :BRL})
```

Excerpt from the local lab; valid-command? is defined in the complete file. Concurrency test executed.

The concurrency test submits eighty identical registrations against an atom and verifies one entry. This exercises local atomicity; it does not demonstrate exactly-once processing between services. Production still needs authenticated identity, persistence, expiration control, reconciliation, and a consistent response when the connection drops after commit.

Official technology documentation

FOUNDATIONS · DATOMIC

Datomic: identity and decision history.

The model separates operation identity from state evolution. The identity constraint prevents the same operation being represented as two entities. Version compare-and-swap accompanies the status change in the transaction. This is an educational proposal, not a description of Nubank's schema.

CLOJURE / ORIGINAL PROPOSAL

```clojure
;; Schema proposto para um exercício de domínio.
[{:db/ident :operation/id
  :db/valueType :db.type/uuid
  :db/cardinality :db.cardinality/one
  :db/unique :db.unique/identity}
 {:db/ident :operation/status
  :db/valueType :db.type/keyword
  :db/cardinality :db.cardinality/one}
 {:db/ident :operation/version
  :db/valueType :db.type/long
  :db/cardinality :db.cardinality/one}]

;; Uma transação propõe a mudança de versão e status.
;; entity-id e expected-version são obtidos pelo adapter.
[[:db/cas entity-id :operation/version
  expected-version (inc expected-version)]
 [:db/add entity-id :operation/status :operation/pending]]
```

Independent reference example; not executed against real infrastructure.

A correct version alone does not guarantee a valid business transition. The adapter must validate the state machine and ensure the observed condition belongs to the authorized transaction. History helps explain decisions but requires retention policy and care not to persist unnecessary sensitive data. Exact semantics depend on the edition and API the team adopts.

Official technology documentation

FOUNDATIONS · KAFKA

Kafka: contract before topic.

The proposed envelope carries event identity, aggregate identity, and domain version. Each answers a different question: have I processed this delivery, which operation does it belong to, and where does it fit in the sequence? The consumer must not confuse a transport offset with a business version.

JSON / ORIGINAL PROPOSAL

```json
{
  "event_id": "synthetic-event-001",
  "event_type": "operation.status_changed",
  "schema_version": 1,
  "aggregate_id": "synthetic-operation-001",
  "aggregate_version": 2,
  "occurred_at": "2026-10-01T12:00:00Z",
  "producer": "proposed-operation-service",
  "correlation_id": "synthetic-correlation-001",
  "payload": {
    "previous_status": "pending",
    "status": "settled",
    "currency": "BRL",
    "amount_minor": 1250
  }
}
```

Proposed JSON contract with synthetic identifiers; not an internal Nubank event.

The partition key must reflect the unit for which order matters. Reprocessing a message requires idempotency in the effect, not just the producer. A Kafka transaction does not automatically turn an external call or a write to another database into a single global transaction. The relevant test interrupts the consumer between persistence and acknowledgment and observes recovery.

Official technology documentation

FOUNDATIONS · AWS

AWS / DynamoDB: arbitrate the first write.

A conditional reservation makes key creation a competition resolved by storage. A service must not perform query-then-insert as two independent operations and assume exclusivity. Scope prevents a customer's key from being confused with another intent.

PYTHON / ORIGINAL PROPOSAL

```python
# Exemplo de adapter com boto3; cliente e tabela são externos.
# Nomes e valores são sintéticos.
def reserve(table, scope, request_key, fingerprint):
    pk = f"IDEMPOTENCY#{scope}#{request_key}"
    return table.put_item(
        Item={
            "pk": pk,
            "fingerprint": fingerprint,
            "status": "reserved",
            "version": 0,
        },
        ConditionExpression="attribute_not_exists(pk)",
    )

# Uma ConditionalCheckFailedException exige consultar
# o registro: replay compatível ou conflito de payload.
# Não converter todo erro em 'já processado'.
# A expiração da chave precisa respeitar a janela de replay.
```

Independent reference example; not executed against real infrastructure.

Reservation is not settlement. If the process crashes after reserving and before submitting the operation, pending work must be distinguished from completed work. Reserved state needs recovery with ownership and a time bound. Expiration policy must not allow a late retry to repeat a financial effect already performed.

Official technology documentation

FOUNDATIONS · KUBERNETES

Kubernetes: availability during change.

The example reserves capacity for one additional replica while gradually removing the previous version. Readiness should indicate whether an instance can receive work; it must not fluctuate with every shared-dependency failure and remove the entire service simultaneously. The numbers are educational assumptions.

YAML / ORIGINAL PROPOSAL

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: proposed-journey-reader
spec:
  replicas: 3
  selector:
    matchLabels: {app: journey-reader}
  strategy:
    rollingUpdate: {maxUnavailable: 0, maxSurge: 1}
  template:
    metadata:
      labels: {app: journey-reader}
    spec:
      containers:
        - name: reader
          image: example.invalid/reader:replace-with-reviewed-digest
          ports: [{containerPort: 8080}]
          readinessProbe:
            httpGet: {path: /ready, port: 8080}
            periodSeconds: 5
          resources:
            requests: {cpu: "250m", memory: "256Mi"}
            limits: {memory: "512Mi"}
```

Reference manifest, deliberately without an executable image. Not applied to a cluster.

Before applying it, a reviewed image, service identity, network policies, graceful shutdown, concurrency limits, and distribution across zones would be required. Three replicas in the same failure domain do not fail independently. Rollout must observe journey completion and saturation as well as count ready pods.

Official technology documentation

FOUNDATIONS · FLUTTER

Flutter: state drives the interface.

The component receives domain state; it does not infer completion from an absent spinner. A pending message must remain true when the network fluctuates. The design avoids offering retry while the financial outcome remains unknown; the next action depends on the reconciliation contract.

DART / ORIGINAL PROPOSAL

```dart
// Fragmento didático: imports e modelos do app são omitidos.
Widget statusPanel(JourneyState state) {
  final String label;
  switch (state) {
    case JourneyState.pending:
      label = 'Estamos confirmando a operação';
      break;
    case JourneyState.settled:
      label = 'Operação concluída';
      break;
    case JourneyState.rejected:
      label = 'Operação não concluída';
      break;
  }
  return Semantics(
    liveRegion: true,
    child: Padding(
      padding: const EdgeInsets.all(16),
      child: Text(label),
    ),
  );
}
```

Reference Flutter/Dart excerpt; not compiled in this edition.

The example needs widget, screen-reader, text-scaling, and real-navigation tests. A live region can announce a change without moving focus, but excessive announcements also harm the experience. Final wording should be developed with Design and Content, including expected time, a safe action, and a help channel when the delay is no longer normal.

Official technology documentation

FOUNDATIONS · DART

Dart: states that do not contradict one another.

A collection of booleans can represent impossible combinations: loading and completed, no receipt and available. A closed set of states makes these possibilities more visible. Here, a confirmed receipt requires its own identifier; query unavailability is not presented as an operation failure.

DART / ORIGINAL PROPOSAL

```dart
sealed class ReceiptState {
  const ReceiptState();
}
final class AwaitingConfirmation extends ReceiptState {
  const AwaitingConfirmation(this.operationId);
  final String operationId;
}
final class Confirmed extends ReceiptState {
  const Confirmed(this.operationId, this.receiptId);
  final String operationId;
  final String receiptId;
}
final class Unavailable extends ReceiptState {
  const Unavailable(this.operationId);
  final String operationId;
}

String actionLabel(ReceiptState state) => switch (state) {
  AwaitingConfirmation() => 'Consultar andamento',
  Confirmed() => 'Abrir comprovante',
  Unavailable() => 'Tentar consultar novamente',
};
```

Educational Dart 3 example; not compiled in this edition.

Typing helps implementation, but the server remains authoritative for financial state. The client must handle old contract versions and unknown responses. Fallback should preserve confirmed data without inventing a new success state. The proposal must be validated against the team's Dart version and modeling conventions.

Official technology documentation

FOUNDATIONS · PROMETHEUS

Prometheus: measure a promise.

Numerator and denominator must observe the same population. If abandoned journeys disappear from instrumentation, latency can look better while the experience gets worse. The measurement contract must define start, completion, timeout, legitimate cancellation, and missing events.

PROMQL / ORIGINAL PROPOSAL

```promql
# Proporção de jornadas com resultado confirmado em até 2s.
# Métricas e rótulos propostos; janela ilustrativa.
sum(rate(journey_confirmation_seconds_bucket{
  journey="payment", le="2"
}[30m]))
/
sum(rate(journey_confirmation_seconds_count{
  journey="payment"
}[30m]))

# Percentil de cauda: analisar por versão do app/serviço.
histogram_quantile(
  0.99,
  sum by (le, release) (
    rate(journey_confirmation_seconds_bucket[5m])
  )
)
```

Proposed PromQL queries; not executed on a Prometheus server.

Aggregated percentiles require care with buckets and distributions. A percentile alone reveals neither volume nor success rate; it must be read alongside counts, errors, and segmentation. Labels need bounded cardinality. Account, operation, and customer identifiers belong in controlled diagnostic trails, not time-series labels.

Official technology documentation

FOUNDATIONS · OPENTELEMETRY

OpenTelemetry: follow causality.

A trace should explain relationships between operations, not merely enumerate calls. Payment confirmation can cross synchronous and asynchronous stages, retries, and reconciliation. Sampling must retain enough failure and high-latency examples for investigation.

PYTHON / ORIGINAL PROPOSAL

```python
# Fragmento de instrumentação; tracer fornecido pelo runtime.
def confirm(command, repository, tracer):
    with tracer.start_as_current_span("journey.confirm") as span:
        span.set_attribute("journey.kind", "payment")
        span.set_attribute("contract.version", 1)
        try:
            result = repository.confirm(command)
            span.set_attribute("journey.outcome", result.kind)
            return result
        except TimeoutError:
            span.set_attribute("journey.outcome", "unknown")
            raise

# Não registrar CPF, token, saldo ou payload integral.
# Propagar contexto entre produtores e consumidores.
# Para processamento assíncrono, modelar links/parentesco
# de acordo com a semântica real do trabalho.
```

Independent reference example; not executed against real infrastructure.

A trace does not replace a durable financial record. It may be sampled, delayed, or missing precisely during an incident. Diagnostics should associate an operation with a trace through controlled access while keeping personal data out of common attributes. The goal is shorter investigations without creating another indiscriminate repository of sensitive information.

Official technology documentation

FOUNDATIONS · JAVA

Java / JVM: explicit budgets.

A timeout for each dependency can add up to more time than the journey allows. A shared budget forces every stage to account for time already consumed. A monotonic clock suits duration within a process; distributed timestamps require another contract and tolerance for clock differences.

JAVA / ORIGINAL PROPOSAL

```java
// Fragmento autoral: todas as durações são ilustrativas.
record Budget(long deadlineNanos) {
    static Budget start(java.time.Duration timeout) {
        return new Budget(System.nanoTime() + timeout.toNanos());
    }
    long remainingNanos() {
        return Math.max(0L, deadlineNanos - System.nanoTime());
    }
    boolean exhausted() {
        return remainingNanos() == 0L;
    }
}

// O adapter repassa apenas o orçamento restante.
// O relógio monotônico mede intervalo dentro do processo.
// Não serializar deadlineNanos para outro host.
var budget = Budget.start(java.time.Duration.ofMillis(800));
if (budget.exhausted()) {
    throw new java.util.concurrent.TimeoutException();
}
```

Reference Java excerpt; not compiled as a program in this edition.

Canceling a wait does not undo a remote effect. After a timeout, the outcome may remain unknown and require reconciliation. This is where my JVM experience connects to the financial domain: controlling threads and connections is necessary, but retry decisions depend on idempotency, authority, and state. The example is not a complete HTTP client.

Official technology documentation

FOUNDATIONS · PYTHON

Python: an experiment must be allowed to fail.

Python can quickly make a behavioral hypothesis concrete. A useful test describes an observable consequence, such as blocking expansion when the sample is insufficient. The code example is a deterministic gate, not a statistical significance test. The threshold must come from an approved journey policy.

PYTHON / ORIGINAL PROPOSAL

```python
import unittest

def can_release(samples, failures, min_samples=1000):
    if type(samples) is not int or type(failures) is not int:
        raise ValueError("counts must be integers")
    if type(min_samples) is not int or min_samples <= 0:
        raise ValueError("invalid sample policy")
    if not 0 <= failures <= samples:
        raise ValueError("invalid counts")
    if samples < min_samples:
        return False
    return failures / samples <= 0.001

class ReleaseGateTests(unittest.TestCase):
    def test_small_sample_does_not_pass(self):
        self.assertFalse(can_release(50, 0))

    def test_error_budget_is_respected(self):
        self.assertTrue(can_release(2000, 2))
        self.assertFalse(can_release(2000, 3))

    def test_invalid_evidence_is_rejected(self):
        with self.assertRaises(ValueError):
            can_release(2000, 2001)
```

Educational example separate from the twenty executed labs; synthetic threshold.

The twenty case labs use synthetic data and local tests. They help detect contract regressions, illegal states, and representation errors. They do not measure Nubank's infrastructure, estimate real load, or certify production controls. Each chapter records exactly what the lab covers and what requires additional validation.

Official technology documentation

FOUNDATIONS · POSTGRESQL

PostgreSQL: integrity in storage.

The schema makes certain invariants impossible to ignore accidentally: positive amount, supported currency, and a unique key per scope. The update conditions the transition on state and version. If no row returns, the service must distinguish conflict, replay, and a nonexistent identifier.

SQL / ORIGINAL PROPOSAL

```sql
CREATE TABLE proposed_operation (
  operation_id UUID PRIMARY KEY,
  customer_scope TEXT NOT NULL,
  request_key TEXT NOT NULL,
  amount_minor BIGINT NOT NULL CHECK (amount_minor > 0),
  currency CHAR(3) NOT NULL CHECK (currency = 'BRL'),
  version BIGINT NOT NULL DEFAULT 0 CHECK (version >= 0),
  status TEXT NOT NULL CHECK (
    status IN ('received','pending','settled','rejected')
  ),
  UNIQUE (customer_scope, request_key)
);

UPDATE proposed_operation
SET status = 'pending', version = version + 1
WHERE operation_id = $1
  AND version = $2
  AND status = 'received'
RETURNING operation_id, version, status;
```

Independent reference example; not executed against real infrastructure.

PostgreSQL is an educational option in this book, not an inference about every Nubank domain's database. A status constraint limits vocabulary but does not express the complete state machine. Transaction, indexes, isolation, and outbox strategy must be evaluated together so persistence and publication recover coherently from failures.

Official technology documentation

FOUNDATIONS · REDIS

Redis: cache with domain validity.

Cache policy starts with a product question: how much delay can this data tolerate? An informational projection may accept brief validity; financial authorization may require the authoritative source. The same tool can be appropriate for one path and unsuitable for another.

PYTHON / ORIGINAL PROPOSAL

```python
# Interface proposta; não é um cliente Redis completo.
def read_projection(key, cache, source, clock):
    cached = cache.get(key)
    if cached and cached["valid_until"] > clock():
        return {
            "data": cached["data"],
            "origin": "cache",
            "version": cached["version"],
        }
    current = source.read(key)
    cache.set(key, {
        "data": current.data,
        "version": current.version,
        "valid_until": clock() + 15,
    }, ttl_seconds=15)
    return {"data": current.data, "origin": "source",
            "version": current.version}
```

Independent reference example; not executed against real infrastructure.

TTL bounds duration but does not prevent a late write from reinstalling an older version. Distributed implementation needs version comparison, invalidation, or another workload-appropriate strategy. The example deliberately omits that mechanism and identifies it as remaining work. Redis is a proposed alternative, not a claim about Nubank's internal design.

Official technology documentation

FOUNDATIONS · TERRAFORM

Terraform: reviewable changes.

Infrastructure as code makes decisions available for review: ownership, recovery, resource identity, and protection against accidental removal. Someone capable of assessing impact must interpret the plan; it should not be approved merely because the file passed formatting.

HCL / ORIGINAL PROPOSAL

```hcl
# Fragmento de referência. Provider e backend são externos.
resource "aws_dynamodb_table" "operation_keys" {
  name         = "proposed-operation-keys"
  billing_mode = "PAY_PER_REQUEST"
  hash_key     = "pk"

  attribute {
    name = "pk"
    type = "S"
  }
  point_in_time_recovery {
    enabled = true
  }
  server_side_encryption {
    enabled = true
  }
  lifecycle {
    prevent_destroy = true
  }
  tags = {
    owner       = "proposed-journey-team"
    environment = "example"
  }
}
```

Independent reference example; not executed against real infrastructure.

prevent_destroy replaces neither access control nor backup policy. Removing the configuration block or changing the management strategy requires additional care. The example was not applied and includes no remote backend, provider version, managed key, or IAM policy. These omissions are explicit so an editorial excerpt does not look production-ready.

Official technology documentation

FOUNDATIONS · TYPESCRIPT

TypeScript: validate the boundary.

A static type does not validate JSON received at runtime. The proposed parser treats the boundary as unknown and rejects states outside the contract. In a product, rejection must become a safe experience with an alternative query or incompatibility message, without inventing financial confirmation.

TYPESCRIPT / ORIGINAL PROPOSAL

```ts
type Status = "pending" | "settled" | "rejected";
type Projection = { version: number; status: Status };

function parseProjection(input: unknown): Projection {
  if (typeof input !== "object" || input === null) {
    throw new Error("invalid object");
  }
  const x = input as Record<string, unknown>;
  if (typeof x.version !== "number" ||
      !Number.isSafeInteger(x.version) || x.version < 0) {
    throw new Error("invalid version");
  }
  if (typeof x.status !== "string" ||
      !["pending", "settled", "rejected"].includes(x.status)) {
    throw new Error("unsupported status");
  }
  return { version: x.version as number,
           status: x.status as Status };
}
```

Independent reference example; not executed against real infrastructure.

TypeScript is used here for contracts and visualization, not as a general Nubank requirement. For a real SDK, I would generate types from a versioned schema, maintain fixtures for old consumers, and define handling of additional fields. Safe evolution expands the contract without forcing every client to update simultaneously.

Official technology documentation

FOUNDATIONS · GRAFANA

Grafana: a dashboard that guides action.

The dashboard starts with the decision someone needs to make. Confirmed completion, unknown state, and reconciliation age describe different aspects of trust. CPU and memory aid diagnosis but do not independently demonstrate that the customer completed the task.

JSON / ORIGINAL PROPOSAL

```json
{
  "title": "Proposta: confiança na jornada",
  "panels": [
    {"title": "Conclusão confirmada", "unit": "percent"},
    {"title": "Estado desconhecido", "unit": "short"},
    {"title": "Tempo de confirmação P99", "unit": "ms"},
    {"title": "Reconciliação mais antiga", "unit": "s"},
    {"title": "Contatos por 1.000 jornadas", "unit": "short"}
  ],
  "dimensions": ["journey", "release", "cohort"],
  "decision": {
    "owner": "proposed-journey-team",
    "action": "hold rollout when guardrails fail",
    "runbook": "referência local do serviço"
  }
}
```

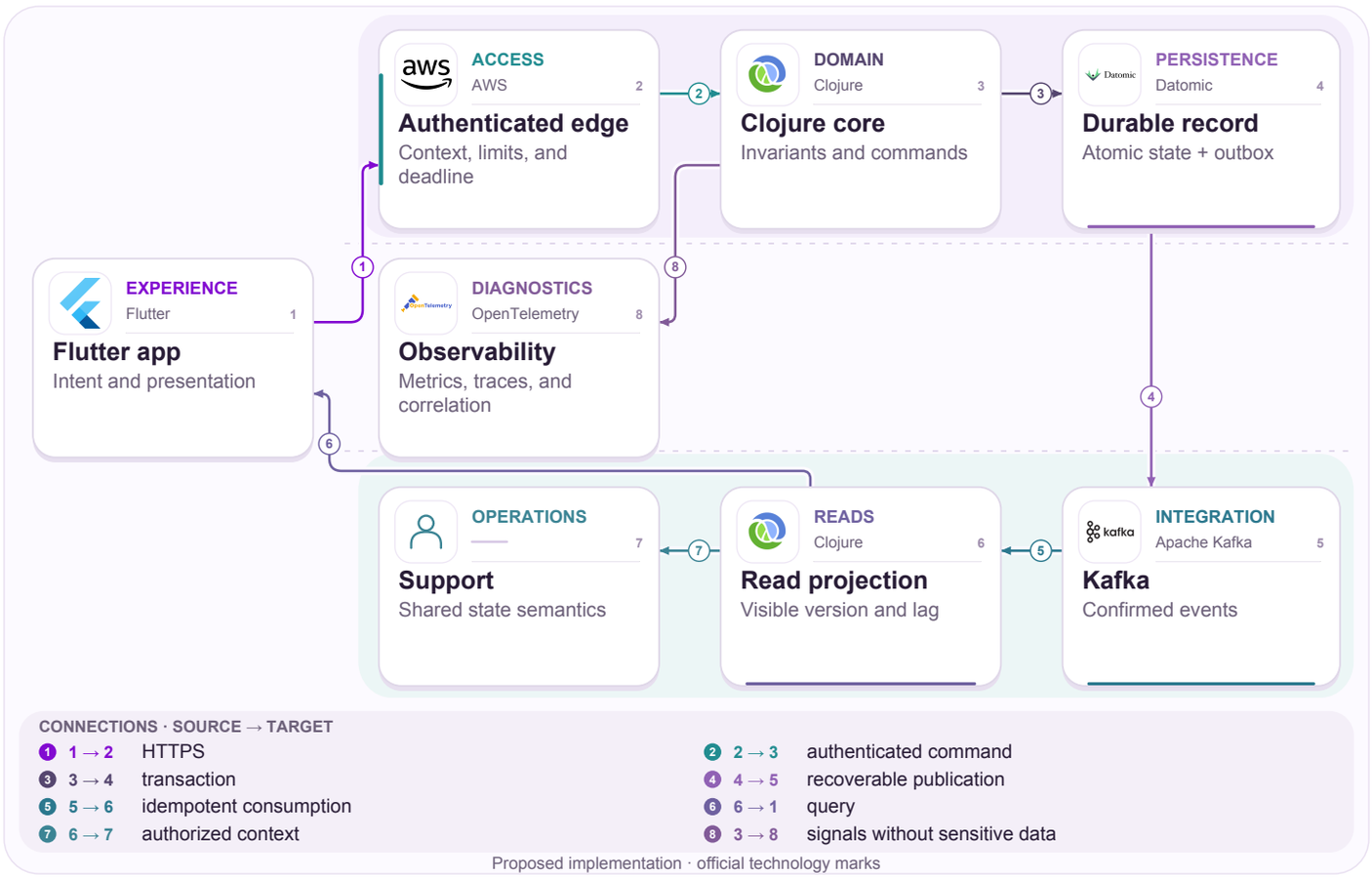
Conceptual dashboard specification; not Grafana's import schema.

The JSON is an editorial specification, not an importable Grafana dashboard. Each indicator needs a definition, denominator, window, owner, and connection to the response procedure. Red without an agreed action creates noise. A good dashboard also displays missing data and small samples so telemetry silence is not interpreted as success.

Official technology documentation

CROSS-CUTTING ARCHITECTURE · EXTERNAL PROPOSAL

One journey, two authorities.



Independent reference design. Does not represent Nubank's internal architecture.

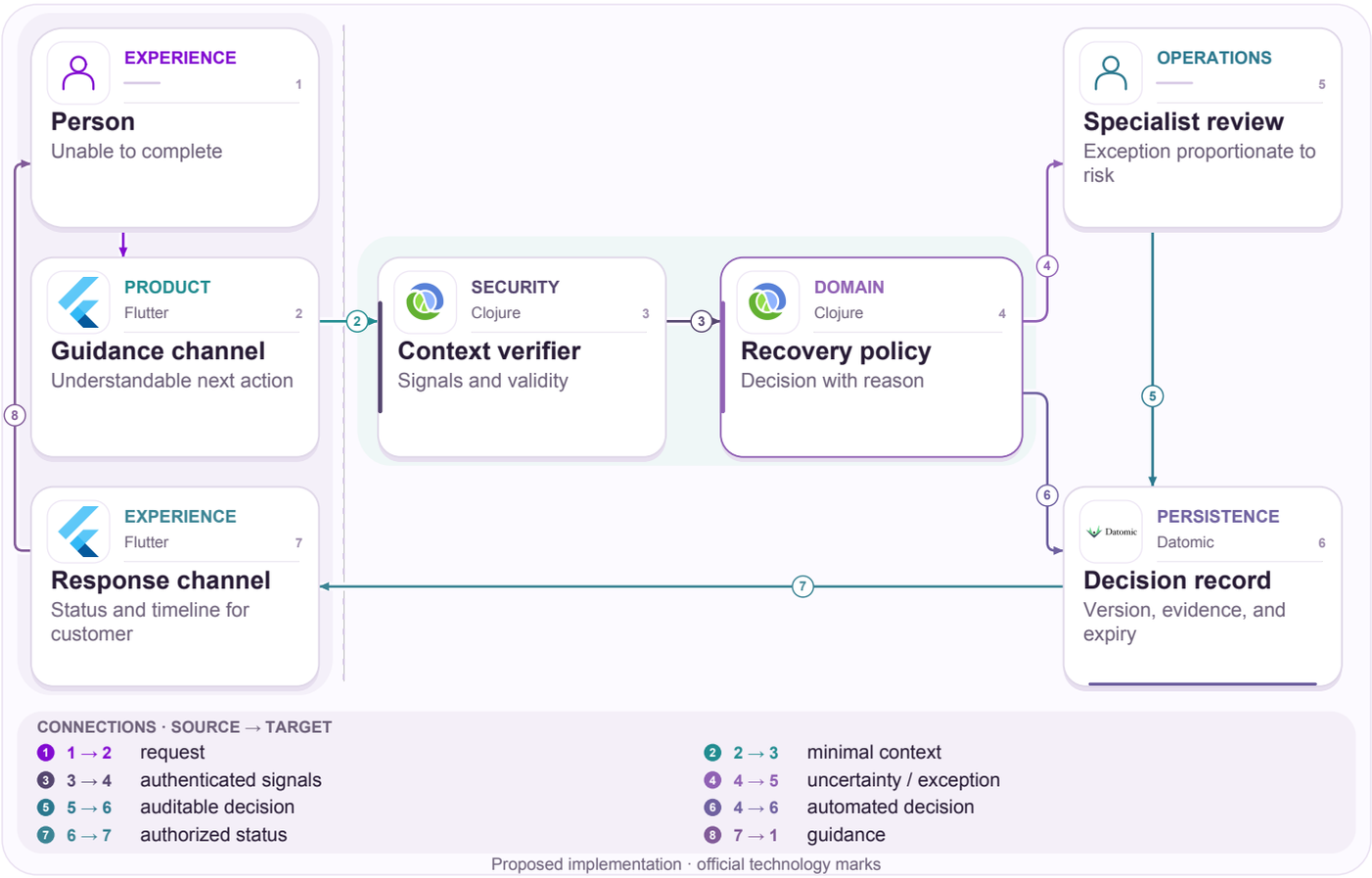
The architecture separates the authority that changes state from the authority that presents it. The domain core and its storage decide whether an operation was accepted. The projection makes reading easier, but carries version and lag. A screen cannot turn a missing update into rejection, and support cannot promise reversal without consulting the state responsible for the effect.

The outbox represents publication intent within the same boundary as the domain write. A publisher may repeat messages, so the consumer must deduplicate effects. The design allows failures between stages and makes recovery part of the contract. The choice between Datomic, DynamoDB, or another persistence option depends on the domain and existing platform; all three are not added simultaneously for aesthetic reasons.

The Clojure lab demonstrates only the in-memory decision. The case labs explore specific properties. Distributed integration, authorization, retention, and capacity still require validation in the team's context.

CROSS-CUTTING ARCHITECTURE · EXTERNAL PROPOSAL

Recovery as part of the product.



Independent reference design. Does not represent Nubank's internal architecture.

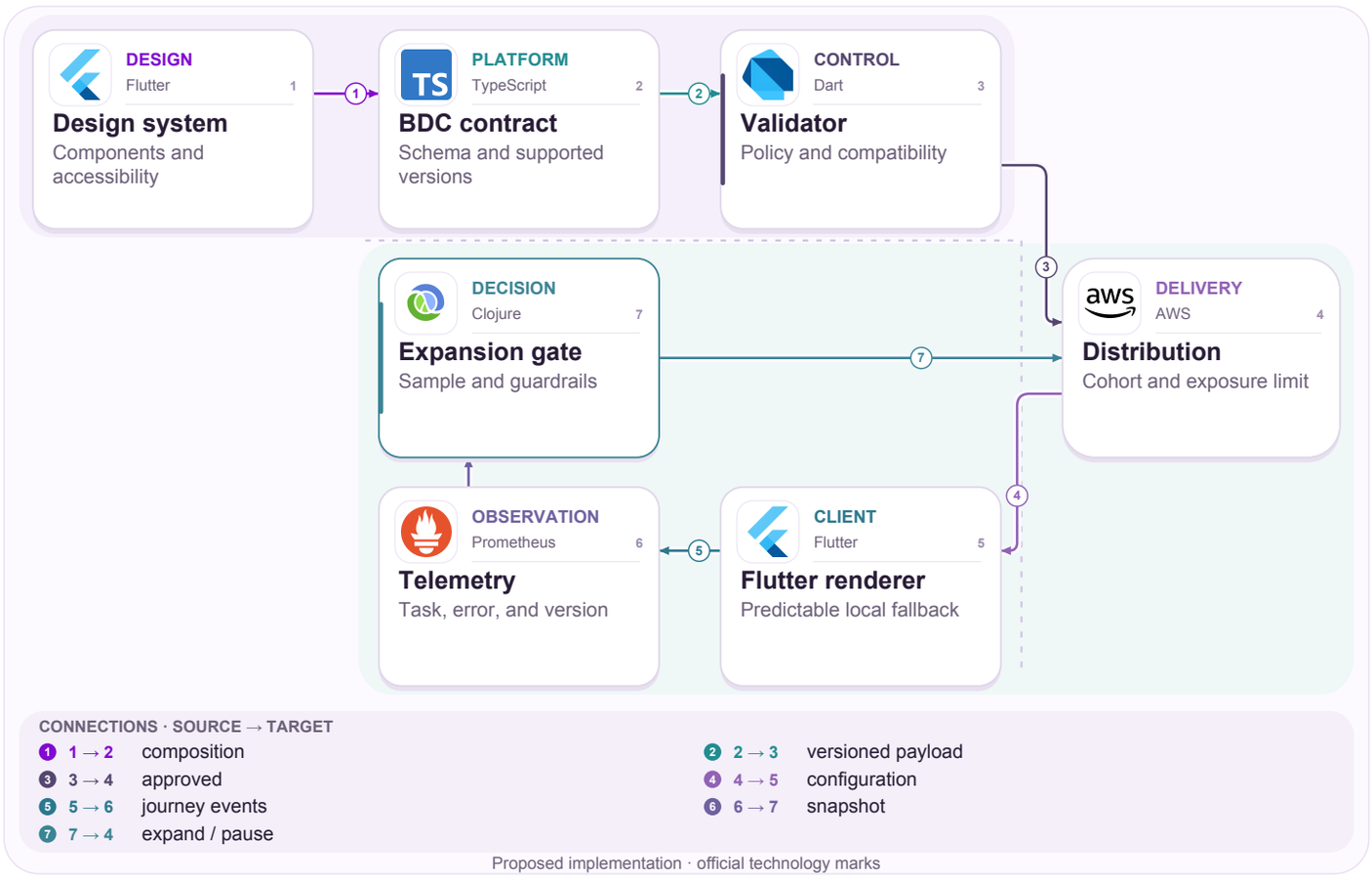
Recovery should not be a collection of ownerless exceptions. The flow must distinguish technical inability, suspected fraud, and insufficient evidence. Each may require a different path. Human review belongs where it adds judgment, not as an unlimited destination for every integration error.

The record associates decision, policy, and validity. An old signal must not authorize a sensitive action indefinitely. When an alternative is denied, the product must explain the next available step without disclosing details that help bypass controls. This language requires collaboration between Security, Product, Content, and Support.

Success combines legitimate resolution, recovery time, and protection. Reducing a queue by rejecting more people can improve an operational metric while worsening the outcome. As EM, I would ensure capacity and responsibilities; as Staff, I would define authority, expiration, and review mechanisms.

CROSS-CUTTING ARCHITECTURE · EXTERNAL PROPOSAL

An experience platform with contracts.



Independent reference design. Does not represent Nubank's internal architecture.

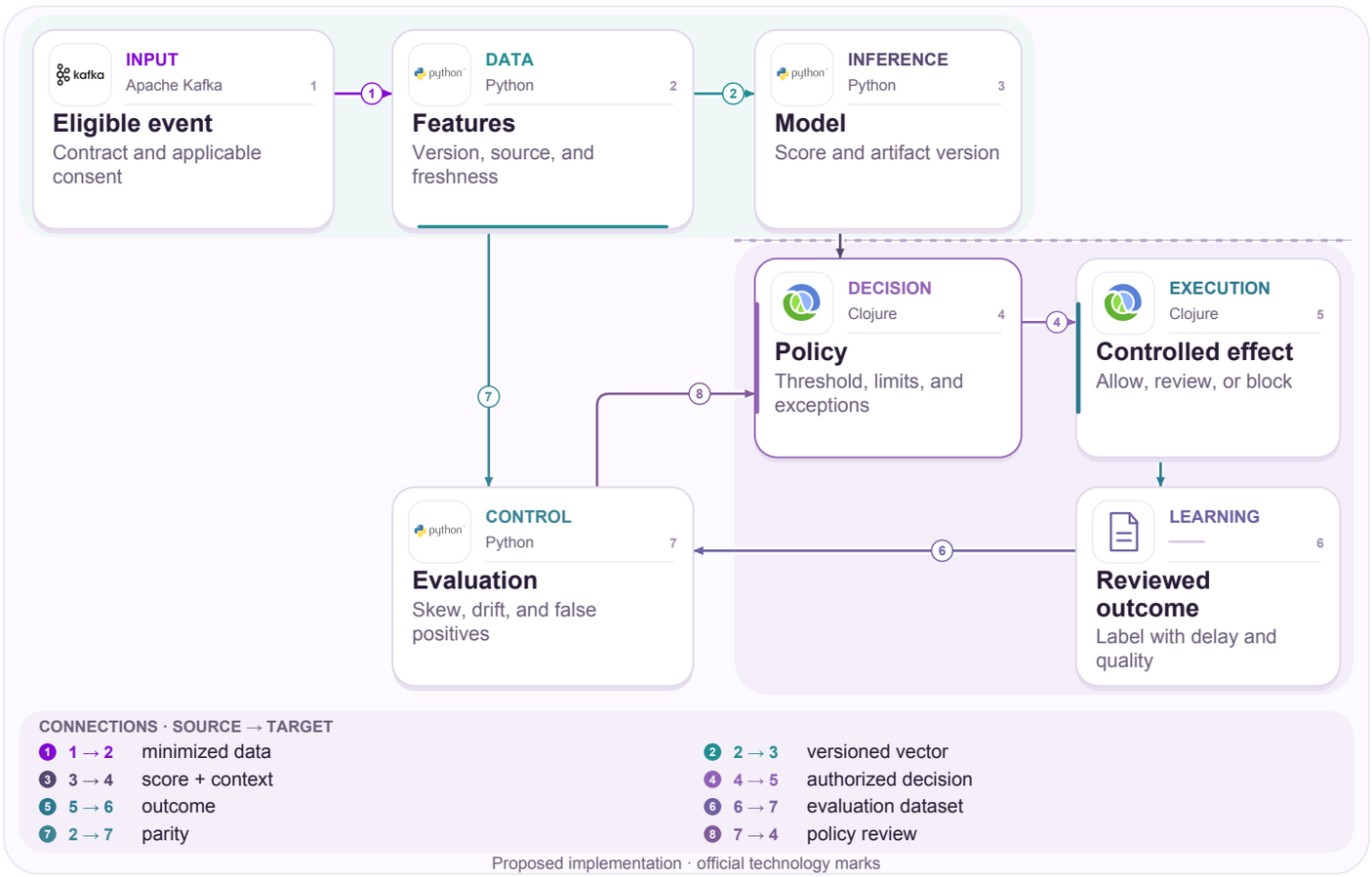
A server-driven interface platform increases composition speed, but also increases the importance of its contract. The installed client may lag behind the server version. The validator must understand the renderer's minimum capability and prevent a configuration requiring nonexistent behavior.

Fallback is a product component: it must preserve the task and accessibility, not merely prevent crashes. Telemetry connects configuration, app version, and journey outcome. The gate requires sufficient evidence before expanding exposure. The decision retains an owner; it is not delegated to one context-free metric.

Nubank publishes work on BDC; this design is an educational proposal for discussing governance and evolution. I would investigate existing controls and seek the smallest necessary extension. The platform creates value when it reduces consumer-team effort while maintaining quality and recovery capability.

CROSS-CUTTING ARCHITECTURE · EXTERNAL PROPOSAL

ML: separate prediction from decision.



Independent reference design. Does not represent Nubank's internal architecture.

A score is not a business decision. Policy combines prediction with limits, context, and review paths. Separating these responsibilities allows a threshold to change without pretending the model was retrained and supports investigation into whether the problem lies in data, inference, or use of the result.

Features need identity and time: what was available when a decision was made may differ from what we know today. Evaluating with future information creates an impression of quality that would not exist in production. The training-serving contract must prevent this leakage and measure relevant differences.

Ground truth may arrive late and remain incomplete. Experiments must preserve version trails and treat false positives as legitimate people harmed by a decision. The proposed value is to improve protection and recovery together, with specialists responsible for policy and exposure criteria.

CASE 01 · EXTERNAL PROPOSAL

The record and its outcome

01

Actual crop of the public source, captured 23 October 2026. The text below includes the company reply and reported outcome.

Resolved historical case · RA 250462257

On June 3, 2026, a customer reported receiving BRL 700.00, transferring BRL 200.00 and attempting another BRL 300.00 Pix that failed. More than 24 hours later, she said the money had not returned. On June 9, Nubank acknowledged a localized communication issue, reported an automatic refund and a privately supplied statement. On June 10, she marked the complaint resolved and rated support 9. The exact refund time is not public.

The product question is whether a failed transfer lets a person distinguish the attempt, temporarily unavailable money and a confirmed return. The private statement was not inspected, so this chapter does not reconstruct a balance or a financial timeline beyond the report. The capture documents a public complaint and subsequent resolution; it is not a customer app screen or evidence of a current defect. The proposal preserves that outcome while examining a possible ambiguity in the journey.

Reclame AQUI - public complaint and response

CASE 01 · EXTERNAL PROPOSAL

Diagnosis before design

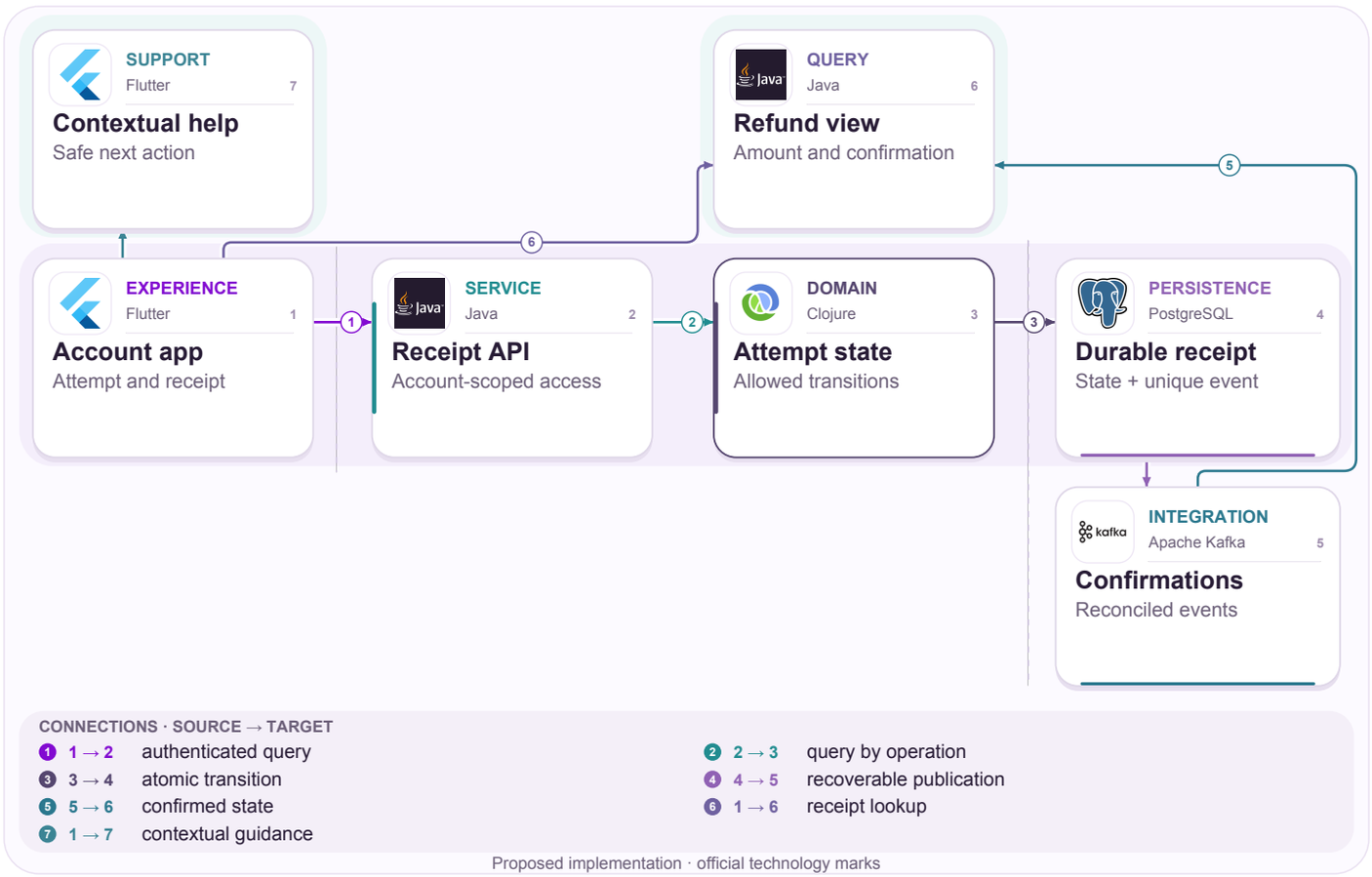
A falsifiable hypothesis is that return information reaches the query channel after financial confirmation, extending uncertainty without necessarily extending the hold. I would correlate the attempt, confirmation, posting and first visible query, accounting for clocks and versions. If the information was already available and understood, the hypothesis weakens; investigation moves to presentation or support context.

The team would distinguish transport failure, rejection before settlement and an actual refund. These are different events. The public response identifies no service, database or queue, so attributing this episode to Kafka, caching or distributed architecture would be unsupported. The first deliverable is a map of observed states and owners, not an infrastructure replacement.

Investigation must also separate client time from server time. An attempt may be created offline, sent later and queried from another device. Without this distinction, a dashboard attributes connectivity delay to financial confirmation. I would use timestamps with declared provenance and a per-operation timeline under restricted investigative access. Clock discrepancies should appear as measurement uncertainty rather than negative durations silently discarded. This also makes comparisons between app versions more credible and prevents an apparently faster path from merely using a different clock.

CASE 01 · EXTERNAL PROPOSAL

Architecture boundaries



Independent proposed architecture; suggested technologies, not Nubank internal infrastructure.

The authenticated entry queries the domain, which applies only authorized transitions. The durable record keeps event identity and confirmed state in one transaction. The help path provides guidance without creating another financial instruction.

Asynchronous publication can lag, so the read model declares its latest confirmation and has a reconciliation path. The design does not move authority over money into the interface. The lab verifies a local boundary without simulating settlement in the Pix scheme.

CASE 01 · EXTERNAL PROPOSAL

Executable core

PYTHON / ORIGINAL PROPOSAL

```python
    def confirm(self, op, event_id, target, trusted=False):
        if trusted is not True or not event_id:
            raise PermissionError("verified adapter required")
        allowed = {"pending": {"settled", "refund_pending"},
                   "refund_pending": {"refunded"}}
        with self.connect() as db:
            db.execute("BEGIN IMMEDIATE")
            prior = db.execute("SELECT op,target FROM events WHERE id=?",
                               (event_id,)).fetchone()
            if prior:
                if prior != (op, target):
                    raise ValueError("event identity conflict")
                return False
            row = db.execute("SELECT state,amount FROM ops WHERE id=?",
                             (op,)).fetchone()
            if not row:
                raise KeyError(op)
            if target not in allowed.get(row[0], set()):
                raise ValueError("illegal transition")
            returned = row[1] if target == "refunded" else 0
            db.execute("UPDATE ops SET state=?,returned=? WHERE id=?",
                       (target, returned, op))
            db.execute("INSERT INTO events VALUES (?,?,?)",
                       (event_id, op, target))
            return True
```

Literal excerpt from the executed independent lab; remaining validation is in the complete file.

An operation starts pending. A failure confirmation allows it to await a refund; only a subsequent authorized event marks the return. BEGIN IMMEDIATE serializes writers in the educational SQLite file. Event identity is compared with its payload, preventing the same key from representing another transition.

The returned amount is recorded in cents and never increases on replay. Ten queries do not change the state. This defines the guarantee: the model records confirmation rather than deciding to return money after a timeout. Signature verification, adapter authenticity and ledger reconciliation remain required integrations.

Python - unittest

Executable case lab on GitHub

CASE 01 · EXTERNAL PROPOSAL

Stack implementation

CLOJURE / ORIGINAL PROPOSAL

```clojure
(defn receipt-transition [state event]
  ;; Proposed pure rule; persistence must serialize the decision.
  (let [allowed {:pending #{:settled :refund-pending}
                 :refund-pending #{:refunded}}
        target (:target event)]
    (cond
      (not (true? (:verified event)))
      {:error :untrusted-event}
      (not (contains? (get allowed state #{}) target))
      {:error :illegal-transition}
      :else {:next-state target
             :event-id (:event-id event)})))
```

Proposed integration example, not executed; this is not Nubank internal code.

The Clojure example makes the transition rule inspectable and separates rejection from side effects. In production it must operate on a still-valid version inside an appropriate transaction or with version comparison. A correct pure decision does not eliminate races between reading and writing.

I would start with the existing stack. Choosing PostgreSQL or Datomic depends on the owning domain, transaction mechanisms already operated and migration cost. The Flutter UI would consume explicit states, with copy reviewed so that “awaiting confirmation” does not become “money lost.” This Clojure excerpt was not executed.

Clojure - Data Structures

CASE 01 · EXTERNAL PROPOSAL

Contracts and ownership

Field / rule	Owner	Invariant
operation_id	Pix	Stable intent
event_id + payload	Adapter	Conflict rejected
owner_scope	Identity	Authorized query
returned_cents	Financial domain	Confirmed only

JSON / ORIGINAL PROPOSAL

```json
{
  "schema_version": 1,
  "operation_id": "synthetic-pix-a",
  "event_id": "synthetic-return-a",
  "state": "refunded",
  "amount_cents": 30000,
  "returned_cents": 30000,
  "owner_scope": "authenticated-subject"
}
```

Illustrative proposed contract; synthetic values, not customer data.

The contract distinguishes attempt ID, event identity and authorized account holder. operation_id is not a credential: knowing it does not authorize a query. Schema versioning allows messages to evolve without silently accepting unknown states.

Retention must cover investigation and legitimate replay under a policy approved by the domain and privacy owners. Operational logs would keep minimal correlation; names, Pix keys and statements would not become metric labels. A receipt is historical and must not be presented as the current balance. Migration would preserve existing identities before enabling new consumers.

CASE 01 · EXTERNAL PROPOSAL

Adversarial verification

PYTHON / ORIGINAL PROPOSAL

```python
    def test_concurrent_refund_replays_once(self):
        self.apply("failed", "refund_pending")
        barrier = threading.Barrier(4)
        def retry(_):
            barrier.wait(timeout=5)
            return self.apply("refund", "refunded")
        with ThreadPoolExecutor(max_workers=4) as pool:
            results = list(pool.map(retry, range(4)))
        self.assertEqual(results.count(True), 1)
        self.assertEqual(self.bank.read("pix-a", "owner-a")[
            "returned_cents"], 30000)

    def test_timeout_is_not_refund_authority(self):
        for _ in range(10):
            self.bank.read("pix-a", "owner-a")
        receipt = self.bank.read("pix-a", "owner-a")
        self.assertEqual(receipt["state"], "pending")
        self.assertEqual(receipt["returned_cents"], 0)
```

Literal lab tests; count and result come from an actual run.

The 12 executed tests cover four concurrent consumers, replay, payload conflict, authorization, invalid money and reopening the file. Repeated queries ensure the application does not hide a financial decision inside a read. The rejected-transition test verifies that an event identity is not consumed before a valid effect exists.

These tests do not validate signed messages, power failure, participant availability or production capacity. Before a pilot, I would add replay of anonymized real messages and publisher recovery tests. The required evidence is preservation of identity and amount, not merely an HTTP 200 response.

Python - unittest

CASE 01 · EXTERNAL PROPOSAL

Measure the right journey

Metric	Numerator / denominator	Window and segment
Trusted-state p 95	Attempt→confirmation time	7 days; failure type
Contacts /1,000	Contacts / eligible attempts	7 days; app version
Unintended repeat	Duplicate orders / attempts	Daily; cohort
Divergence	Incorrect / audited receipts	Daily; state

The experiment begins with shadow queries compared against the financial source, without displaying the new receipt. An eligible cohort then receives the explanation while the control retains the previous experience and the same support access. The analysis unit is an unresolved attempt, so retries are not counted as additional customers.

Collection combines confirmed events and support contacts through restricted references, never through a customer’s public text. I would observe each attempt for seven days and segment by app version and failure path. Any false confirmation, cross-account access or duplicate Pix attributable to the receipt stops expansion. The baseline is unknown and must be measured.

CASE 01 · EXTERNAL PROPOSAL

Financial value with assumptions

Assumed scenario	Main input	Net/month	Payback (months)
Conservative	20,000 × 0.05 × BRL 8.00	BRL 4,000.00	30.00
Base	50,000 × 0.10 × BRL 8.00	BRL 32,000.00	3.75
Ambitious	100,000 × 0.15 × BRL 8.00	BRL 105,000.00	1.14

BRL 300.00 is the principal discussed in the complaint; it is not bank profit and does not enter this calculation. The scenarios assume 20,000 / 50,000 / 100,000 eligible attempts per month, incremental contact reductions of 0.05 / 0.10 / 0.15 and an avoidable marginal cost of BRL 8.00 per contact. Assumed additional monthly operating costs are BRL 4,000.00 / BRL 8,000.00 / BRL 15,000.00.

The calculation is volume × contacts avoided per attempt × marginal cost, less operations. The assumed BRL 120,000.00 investment pays back only when monthly net contribution is positive. The conservative scenario shows that a useful idea may not fund its implementation within the first year at that scale. Released capacity without lower expenditure must be reported as capacity; average salary is not a substitute for marginal cost.

Assumed implementation budget: BRL 120,000.00. Annual net results before implementation: BRL 48,000.00 / BRL 384,000.00 / BRL 1,260,000.00. Every input is an assumption, not a Nubank baseline.

CASE 01 · EXTERNAL PROPOSAL

Accountable roadmap

Proposed phase	Accountable role	Delivery / exit gate
Days 0-15	Pix lead	Map and reconciled sample
Days 16-30	Staff + identity	Contract; scope validated
Days 31-60	Engineering + support	Shadow without divergence
Days 61-90	EM + product	Pilot; economic decision

The 90-day horizon is a proposal conditional on discovery. The Pix technical owner validates states and evidence; product approves interpretation; support checks that it can explain the same receipt. The EM coordinates dependencies and specialist availability without assuming authority over financial rules.

I would not move directly from design to a broad cohort. The intermediate gate requires reconciling a sample of attempts and proving that a lost response remains queryable later. If the main observed cause is an actual refund delay, the work must include the corresponding financial domain; changing copy alone would be insufficient. The budget can be revised after that diagnosis.

CASE 01 · EXTERNAL PROPOSAL

Rollout and operations

Rollout starts with the observer, then authenticated querying and finally presentation. A flag can remove the new screen but must not delete recognized receipts or events. During rollback, the previous service must still query a compatible record; incompatible versions must be stopped before receiving traffic.

The payments on-call owner receives divergence alerts while support receives approved guidance for unknown states. A drill injects duplicate events, publication lag and unavailable reads. The safe response preserves evidence and initiates reconciliation rather than issuing another transfer. Alerts without an owner or a resolution procedure do not satisfy operational acceptance.

CASE 01 · EXTERNAL PROPOSAL

My contribution

As Staff, I would propose the authority boundary, review invariants and document the choice between querying a projection and reaching the financial source. I would also facilitate review with the owning teams to reduce unnecessary dependencies. The rejected alternative is a timer that turns silence into a refund: it confuses unavailability with a financial decision and may contradict settlement that already occurred.

As EM, I would organize small deliveries, investigation capacity and a shared definition of done involving support and product. These are proposed contributions, not claims that I already led this initiative at Nubank. An interview can explore how I would negotiate reduced scope that preserves correctness when a commercial deadline favors an apparently faster solution.

In an interview, I would ask for an anonymized unresolved attempt and explain which decision each additional piece of evidence would enable. This demonstrates reducing ambiguity before choosing technology. As a leader, I would identify who can authorize a financial-contract change and who owns the customer message. Collaboration prevents a local improvement from merely transferring uncertainty to another team. The result should be a decision record that another engineer can challenge and operate, rather than a design whose correctness depends on its author remaining available.

CASE 01 · EXTERNAL PROPOSAL

Handoff and decision

Local execution

python3 labs/case_01.py -v · 12 tests passed.

A reviewer starts with Receipts.create, creates the synthetic attempt and follows confirm through to read. Notice that settled is terminal in this flow; a settled transfer would require a different refund process. The 30,000-cent fixture illustrates the reported amount without reproducing a real account or statement.

The handoff includes code, tests, the contract and integration questions: who authenticates the event, which system confirms the return, and how does publication recover after failure? Further investment depends on these answers and a contact baseline. If ambiguity is not measurable or reconciliation cost exceeds benefit, I would end the pilot while retaining useful instrumentation.

- Replay neither duplicates money nor changes content.
- No holder can query another account's receipt.
- Rollback preserves identities and history.

Executable case lab on GitHub

CASE 02 · EXTERNAL PROPOSAL

The record and its outcome

02

Actual crop of the public source, captured 23 October 2026. The text below includes the company reply and reported outcome.

Historical incident resolved · July 8, 2026

On July 8, 2026, Canaltech reproduced a Pix sent to Nubank that did not appear in the app. Its test between Nubank accounts did not reproduce the problem. The report described an account-loading notice. Nubank reported temporary instability and recovery around 11:15. The transferred amount, affected population and exact duration were not published.

The evidence combines a newsroom's own observation with the bank's response. This is more specific than a mention count but does not replace financial telemetry. The article photograph shows the application's store listing, so its caption is editorial. It is not presented as an error screenshot. The article quotes posts on X that were not individually verified at their origin. This chapter uses the reported observation and recovery without estimating losses.

Canaltech - July 8, 2026 incident

CASE 02 · EXTERNAL PROPOSAL

Diagnosis before design

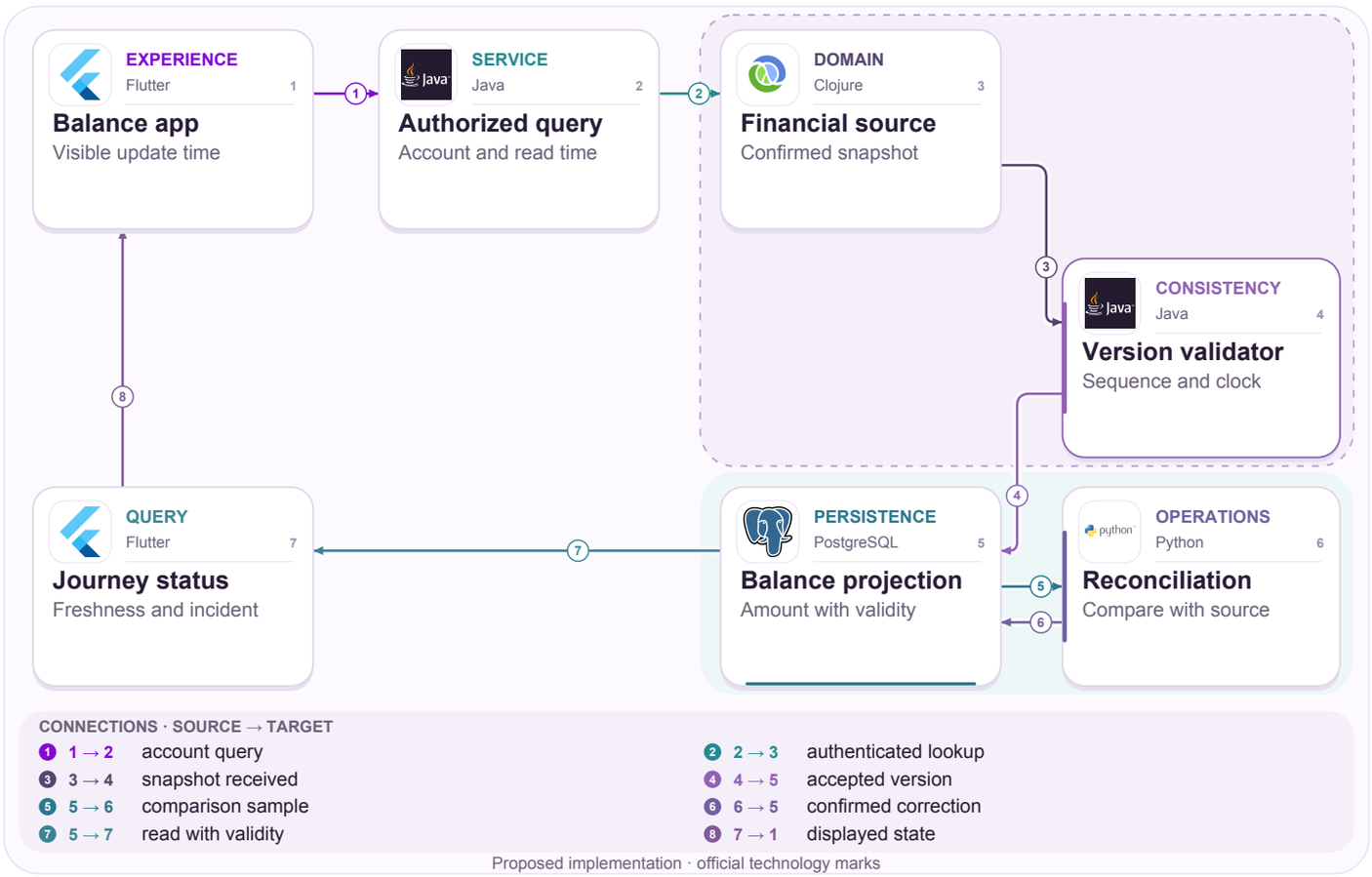
A missing displayed balance permits different hypotheses: unavailable reads, a delayed projection, interrupted integration or an unconfirmed credit. I would begin with the latest known confirmation at each layer without assuming ledger failure. Cross-institution and internal Pix transfers are different probes; the published sample does not establish that only external transfers were affected.

The testable hypothesis is that an old snapshot remained visible without enough validity context. Testing requires source version, source time, query time and displayed message. If the source itself has not confirmed the credit, the product should express uncertainty rather than turn a refreshed screen into financial certainty. Support needs to distinguish these situations.

A snapshot version must be comparable within one account and a defined source. Adding versions from different components creates an artificial order. If a summary combines multiple sources, the response needs coverage and age for each relevant component. I would start with the smallest set that explains the displayed balance rather than turning this case into a rewrite of the entire home screen. This boundary makes correctness demonstrable before broadening composition and gives product a concrete choice about which information remains useful during partial failure.

CASE 02 · EXTERNAL PROPOSAL

Architecture boundaries



Independent proposed architecture; suggested technologies, not Nubank internal infrastructure.

The financial source remains authoritative. The validator accepts increasing versions, and the projection stores validity alongside the amount. Reconciliation compares samples with the origin and returns only confirmed corrections.

Journey status combines two independent facts: freshness of that balance and presence of an incident. Closing a banner does not renew the snapshot. This separation avoids a green interface that conveys confidence without new financial evidence.

CASE 02 · EXTERNAL PROPOSAL

Executable core

PYTHON / ORIGINAL PROPOSAL

```python
    def read(self, owner, now):
        if owner != self.owner:
            raise PermissionError("account scope")
        integer(now)
        with self.lock:
            snapshot = self.snapshot
            active_incident = now < self.incident_until
            if snapshot is None:
                return {"state": "unknown", "balance_cents": None,
                        "incident": active_incident, "may_resend": False}
            version, cents, observed, valid_until = snapshot
            if now < observed:
                return {"state": "unknown", "balance_cents": None,
                        "incident": active_incident, "may_resend": False}
            fresh = now < valid_until
            return {"state": "fresh" if fresh else "stale",
                    "balance_cents": cents, "version": version,
                    "as_of": observed, "incident": active_incident,
                    "may_resend": False}
```

Literal excerpt from the executed independent lab; remaining validation is in the complete file.

BalanceView.read checks the account holder and clock before building the response. Without a snapshot it returns unknown with an absent amount; zero would be a false financial assertion. A time preceding the observation also yields unknown. At the exact validity boundary, the state becomes stale.

The model permits displaying the last amount with its timestamp but never recommends resending Pix. Its lock protects one process’s memory only. Persistence, authentication and clock synchronization must be implemented outside this core. Integer seconds simplify testing without prescribing the real API’s time format.

Python - unittest

Executable case lab on GitHub

CASE 02 · EXTERNAL PROPOSAL

Stack implementation

DART / ORIGINAL PROPOSAL

```dart
String balanceLabel(Map<String, Object?> view) {
  // Proposed presentation only; never authorizes a transfer.
  switch (view['state']) {
    case 'fresh':
      return 'Confirmed balance';
    case 'stale':
      return 'Last known balance - awaiting update';
    default:
      return 'Balance currently unavailable';
  }
}
bool mayResendPix(Map<String, Object?> view) => false;
```

Proposed integration example, not executed; this is not Nubank internal code.

In the proposed Dart example, a presentation function treats unknown, stale and fresh as explicit states. Incident status arrives separately. The visual layer neither computes balance nor changes validity; it translates contracts into understandable, accessible information.

I would check assistive-technology reading, localized time and reconnection behavior. An endless skeleton can hide an earlier balance; showing that amount without context can mislead. The decision requires comprehension research using real tasks. The example was not compiled, and its messages require product review before use.

Dart - Switch statements

CASE 02 · EXTERNAL PROPOSAL

Contracts and ownership

Field / rule	Owner	Invariant
source_version	Source	Increasing per account
observed_at	Financial domain	Validated clock
valid_until	Contract	Expires without renewal
incident_until	On-call	Does not change balance

JSON / ORIGINAL PROPOSAL

```json
{
  "schema_version": 1,
  "source_version": 8,
  "balance_cents": 12000,
  "observed_at": 100,
  "valid_until": 120,
  "incident_until": 125,
  "state": "stale"
}
```

Illustrative proposed contract; synthetic values, not customer data.

The version belongs to the source, not to the number of client refreshes. Two snapshots with the same version and different amounts are a conflict worth investigating. An older version does not replace a recent one; this model also rejects a newer version whose clock moves backward.

Validity duration must reflect the data contract and be configured by the financial owner rather than chosen arbitrarily by the UI. Telemetry records aggregate age and state without account or amount labels. Reconciliation uses restricted access to investigate samples. Expiration of an incident notice never modifies these contracts.

CASE 02 · EXTERNAL PROPOSAL

Adversarial verification

PYTHON / ORIGINAL PROPOSAL

```python
    def test_incident_expiry_does_not_refresh_balance(self):
        self.view.incident(125, True)
        result = self.view.read("a", 130)
        self.assertFalse(result["incident"])
        self.assertEqual(result["state"], "stale")
        self.assertFalse(result["may_resend"])

    def test_concurrent_versions_keep_newest(self):
        def publish(version):
            return self.view.publish(version, version * 100, 110, 140, True)
        with ThreadPoolExecutor(max_workers=4) as pool:
            list(pool.map(publish, (5, 2, 4, 3)))
        self.assertEqual(self.view.read("a", 115)["version"], 5)
        self.assertEqual(self.view.read("a", 115)["balance_cents"], 500)
```

Literal lab tests; count and result come from an actual run.

The 14 tests challenge time boundaries, version conflict, unauthorized sources, another account and concurrent publication. The key test lets the banner expire while the balance remains old. Another publishes versions out of order and requires the highest to survive. Testing whether a message appears is insufficient: amount, version and context must remain coherent.

The program does not establish distributed-clock quality or external-system freshness. Integration would add clock-skew simulation, partial reads and delayed probes, alongside accessibility testing. The 12,000-cent fixture is synthetic and does not come from the report.

Python - unittest

CASE 02 · EXTERNAL PROPOSAL

Measure the right journey

Metric	Numerator / denominator	Window and segment
Freshness	Expired reads / reads	15 min; origin
Detection	Failure→alert time	Incident; path
Comprehension	Correct tasks / tasks	Research; state
Contacts	Contacts / exposed sessions	24 h; severity
False current label	Incorrect / audited labels	Continuous; version

Collection must separate availability, correctness and freshness. A fast endpoint can return old data; a healthy probe does not represent every originating bank. I would segment by receiving path and app version, using aggregate volumes and a rolling 15-minute operations window.

A communication experiment must compare equivalent incidents or cohorts because severity varies. The hypothesis is fewer contacts and repeated attempts without hiding uncertainty. Expansion stops for a balance labeled current without valid evidence, an amount discrepancy or unsafe retry guidance. I would not set a latency target before measuring the existing path.

CASE 02 · EXTERNAL PROPOSAL

Financial value with assumptions

Assumed scenario	Main input	Net/month	Payback (months)
Conservative	2 × 20,000 × 0.01	BRL -2,200.00	None
Base	3 × 40,000 × 0.02	BRL 8,800.00	17.05
Ambitious	4 × 60,000 × 0.03	BRL 38,400.00	3.91

The modeled cost concerns incident handling, not supposedly lost money. Monthly assumptions are 2 / 3 / 4 comparable incidents, 20,000 / 40,000 / 60,000 sessions per incident and incremental contact reductions of 0.01 / 0.02 / 0.03. Each avoidable contact hypothetically costs BRL 7.00; probes and operations add BRL 5,000.00 / BRL 8,000.00 / BRL 12,000.00 per month.

I multiply incidents × sessions × contact-rate difference × cost and subtract operations. Hypothetical implementation investment is BRL 150,000.00. These volumes are sensitivity inputs, not estimates of the July event. Results cannot be transferred automatically to an incident of different severity. The same session must not be counted in both this case and case 01 to add benefits.

Assumed implementation budget: BRL 150,000.00. Annual net results before implementation: BRL -26,400.00 / BRL 105,600.00 / BRL 460,800.00. Every input is an assumption, not a Nubank baseline.

CASE 02 · EXTERNAL PROPOSAL

Accountable roadmap

Proposed phase	Accountable role	Delivery / exit gate
Days 0-15	Finance + SRE	Semantics and probes
Days 16-30	Staff + mobile	Contract and shadow
Days 31-60	Product + support	Comprehension validated
Days 61-90	EM + on-call	Pilot and drill

During the first weeks, financial engineering and reliability define what each source confirms. Product and support agree on messages for absence, lag and recovery. The probe owner verifies that probes create no unauthorized transfers and do not congest the real path.

The next milestone is an invisible comparison between calculated labels and evidence. Only then does a small cohort receive the interface. The EM coordinates review and on-call coverage; Staff resolves source, projection and client contracts. If freshness cannot be measured reliably, launch remains limited to unavailability guidance rather than simulated precision.

CASE 02 · EXTERNAL PROPOSAL

Rollout and operations

The runbook separates query failure from absent financial confirmation. If the observer loses connectivity, it must not announce that every Pix failed. An incident has an owner, scope, version and expiration; a recovery update must be traceable to its owner.

In the failure drill I freeze the projection, expire the banner and restore only the API. The UI must still recognize old data. Rollback removes new messages while retaining snapshots and instrumentation. Support receives an answer consistent with the technical view and is not instructed to confirm a credit the system has not confirmed.

Lab limitation: incident notices are unversioned, so an old notice can replace a newer one. This does not refresh the balance. Notice ordering belongs to the proposed integration and is not implemented in this example.

CASE 02 · EXTERNAL PROPOSAL

My contribution

My Staff contribution would make validity semantics explicit and review how the client receives a composite response. I would reject a single “system normal” boolean because it mixes global health, local availability and correctness for one account. A richer representation requires product care but explains limits without inventing certainty.

As EM, I would lead a delivery involving reliability, payments, mobile and support with named accountable roles. Stopping can be valuable: a banner that creates confusion or false confirmations should not expand even if it reduces calls. I would bring evidence and alternatives to the team while respecting its decisions and existing stack expertise.

A sound Staff decision also makes the observer’s unknowns explicit. I would discuss with product the difference between reporting unavailability and claiming that an operation did not happen. As EM, I would protect post-incident analysis time and review alert usefulness with the people carrying the pager. This routine reduces pressure to publish a green status without sufficient evidence merely to close an administrative incident. It also gives support a reliable escalation path when the global message and one customer’s experience do not agree.

CASE 02 · EXTERNAL PROPOSAL

Handoff and decision

Local execution

python3 labs/case_02.py -v · 14 tests passed.

A reviewer runs the lab, calls publish with different versions and observes read at the validity boundary. Then the incident is closed without updating the balance. This walkthrough demonstrates the central separation without real infrastructure.

Integration must identify who issues the version, how time is obtained, the allowed lag and who confirms recovery. Before expansion, require an audited sample, assistive-technology testing and the ability to reconcile divergent snapshots. If those responsibilities lack owners, keep instrumentation in shadow and resolve ownership before customer exposure.

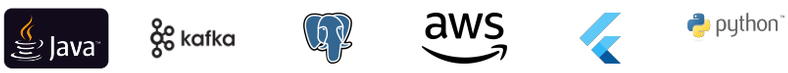
- An expired banner does not refresh data.
- Unknown balance is not displayed as zero.
- Conflicting versions trigger investigation.

Executable case lab on GitHub

CASE 03 · EXTERNAL PROPOSAL

The record and its outcome

03



Dinheiro e Negócios

“Indícios de ilícitos”: Nubank bloqueia contas e deixa clientes sem acesso ao dinheiro

Actual crop of the public source, captured 23 October 2026. The text below includes the company reply and reported outcome.

Outcome reported by the press · March 2026

Metrópoles reported that more than BRL 2,000.00lion belonging to a beauty business was blocked on January 20, 2026, followed by account closure. The article attributes a tax-refund origin to the defense and reports a court order. Nubank cited security and compliance and declined case details on confidentiality grounds. In the March 11 video, the reporter said the money was transferred to another bank. Court records and compliance were not independently verified.

The article and transcript make different contributions. The article describes the dispute; the 06:59-07:23 transcript adds the outcome according to the reporter. I do not claim to have watched the entire video or inspected court records. This proposal does not treat the restriction as current. It also does not transfer compensation from other customers in the article to this episode. The case informs review, explanation and exit design with controls appropriate to each decision’s authority.

Metrópoles - restriction and reported outcome

Metrópoles - reported outcome, 06:59-07:23

CASE 03 · EXTERNAL PROPOSAL

Diagnosis before design

Actual frame from Metrópoles, 11 March 2026. The transcript describes the company case at 02:26 and reports release at 06:59-07:23.

The technical question is whether a restriction remains connected to grounds, evidence, an owner and its next review as it crosses support, risk and account closure. This does not determine whether a particular suspicion was valid. A testable hypothesis is lost context between initial restriction and execution of an authorized exit.

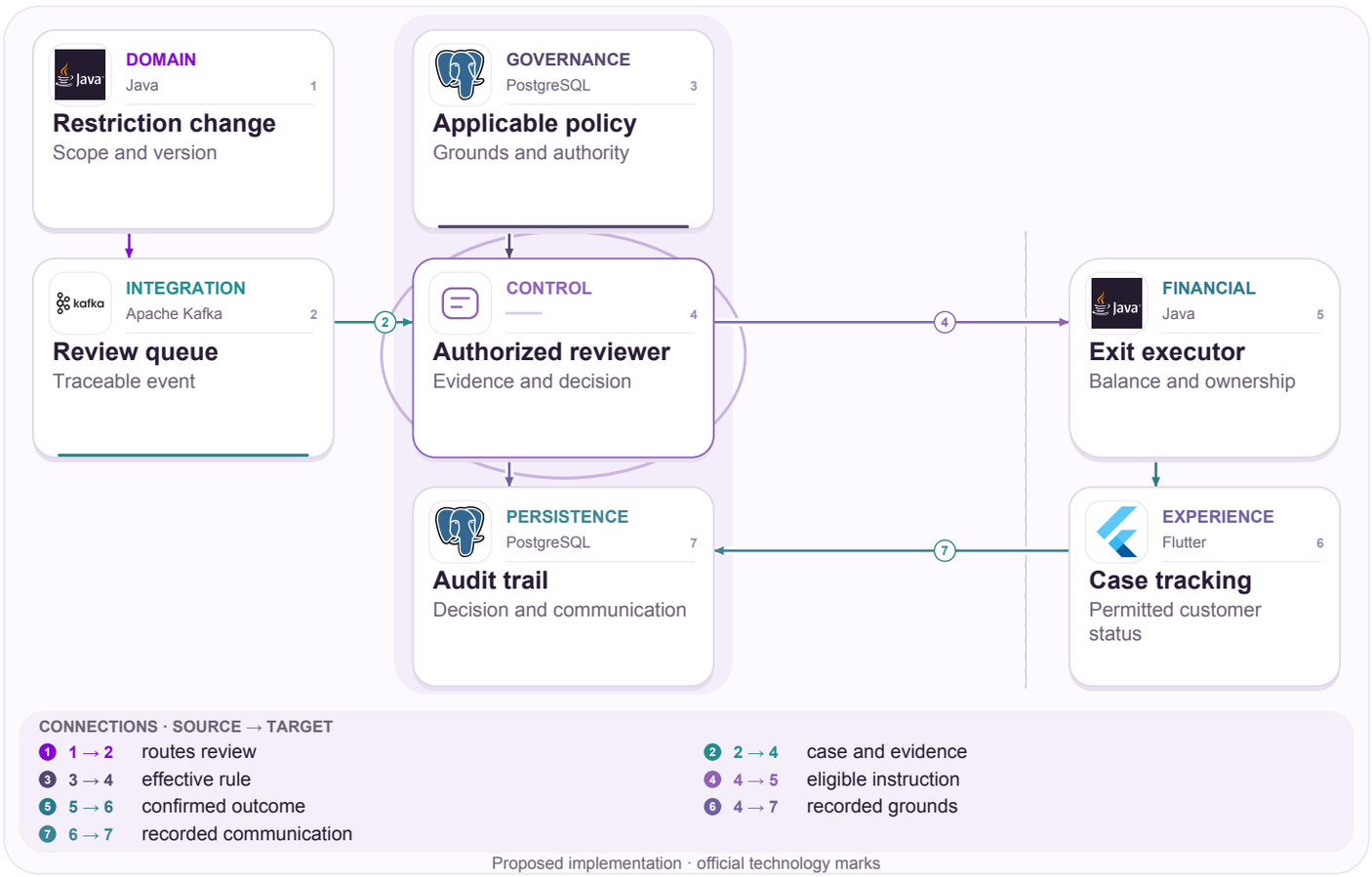
I would sample cases by measure type and reconstruct state changes with accountable owners. Decision time would be compared with actual transfer instruction, distinguishing document waits, active legal holds and operational queues. The report does not provide that breakdown. I would not apply a universal 72-hour deadline: the rule depends on the relevant grounds and requires legal validation.

A document submitted by a business may be relevant without being conclusive. The system must record receipt, reviewer and the authorized decision arising from it rather than automating a legal conclusion because a file exists. I would also investigate handoffs: a support task marked complete does not necessarily close risk review. Distinguishing work status from restriction status prevents productivity indicators from hiding operationally unresolved cases. Each transition should expose its prerequisite to the next owner so that progress does not depend on reconstructing a conversation from scattered messages.

Watch source video

CASE 03 · EXTERNAL PROPOSAL

Architecture boundaries



Independent proposed architecture; suggested technologies, not Nubank internal infrastructure.

Review receives events and consults the effective policy. An eligible decision proceeds to financial execution while the trail retains grounds and communication. The customer receives only approved status information, without antifraud thresholds or third-party data.

The design separates review from execution. Approving an exit does not prove that the balance was transferred. Confirmation depends on reconciliation, an authorized destination and no active impediments, checked again at the execution boundary.

CASE 03 · EXTERNAL PROPOSAL

Executable core

PYTHON / ORIGINAL PROPOSAL

```python
    def execute_exit(self, expected, role, destination_owner,
                     reconciled_cents, no_active_hold=False):
        with self.lock:
            if role != "executor":
                raise PermissionError("execution scope")
            if type(expected) is not int or expected != self.version:
                raise ValueError("stale execution")
            if self.state != "exit_approved":
                raise ValueError("review required")
            if (not isinstance(destination_owner, str)
                    or not destination_owner.strip()
                    or destination_owner != self.owner):
                raise PermissionError("destination ownership")
            if type(reconciled_cents) is not int:
                raise ValueError("integer cents required")
            if reconciled_cents != self.cents:
                raise ValueError("reconcile before transfer")
            if no_active_hold is not True:
                raise ValueError("legal and risk holds unresolved")
            self.state = "transferred"
            self.released = self.cents
            self.version += 1
            return self.released
```

Literal excerpt from the executed independent lab; remaining validation is in the complete file.

Restriction.execute_exit requires an executor role, expected version, destination ownership, reconciled balance and explicit clearance of active impediments. Each rejection preserves the prior approval without transferring money. The review clock does not authorize execution: an overdue date creates pending work, not technical permission to move funds.

The 200,000,100-cent amount is merely a synthetic fixture above BRL 2,000.00lion, not the case's exact amount. This model uses an in-memory lock and makes no payments. Real integration requires durable transactional control, operator identity and transfer confirmation by an authorized financial domain.

Python - unittest

Executable case lab on GitHub

CASE 03 · EXTERNAL PROPOSAL

Stack implementation

SQL / ORIGINAL PROPOSAL

```sql
-- Illustrative compare-and-write; not executed.
UPDATE restriction_case
SET state = 'exit_approved', version = version + 1,
    decision_ref = :approved_decision
WHERE case_id = :case_id
  AND version = :expected_version
  AND state = 'restricted'
  AND reconciliation_version = :confirmed_version;
-- Require exactly one row; otherwise reload and review.
-- Transfer execution is a separate authorized transaction.
```

Proposed integration example, not executed; this is not Nubank internal code.

The illustrative SQL conditions an update on version, state and reconciliation confirmation. It demonstrates compare-and-write but does not independently implement separation of duties or legal assessment. The service must check identity and policy before the transaction and treat a zero-row update as a conflict.

I would favor integration with the existing restriction record rather than a second decision center. A queue can distribute work; it cannot declare a case eligible because it is old. Permissions should distinguish reviewers, executors and auditors, with documented and reviewed exception paths.

PostgreSQL - Transaction Isolation

CASE 03 · EXTERNAL PROPOSAL

Contracts and ownership

Field / rule	Owner	Invariant
restriction_scope	Risk/legal	Specific grounds
decision_version	Reviewer	Conflict invalidates
evidence_ref	Custodian	Restricted access
exit_instruction	Financial domain	Recheck active holds

JSON / ORIGINAL PROPOSAL

```json
{
  "schema_version": 1,
  "case_id": "synthetic-restriction",
  "restriction_scope": "account_exit",
  "decision_version": 2,
  "evidence_ref": "restricted-object-reference",
  "state": "exit_approved",
  "execution_confirmed": false
}
```

Illustrative proposed contract; synthetic values, not customer data.

A restriction should not be a context-free boolean. Its contract must represent scope, grounds, version and an evidence reference while keeping sensitive documents out of broadly distributed events. A reference can remain auditable without exposing its contents to general support.

An exit instruction retains the authorizing decision but rechecks impediments before execution. A new restriction between review and transfer invalidates the earlier version. Approved communication explains the stage and permitted appeal channel without revealing detection signals. Retention and access to court or supporting documents follow specific policies rather than generic logging defaults.

CASE 03 · EXTERNAL PROPOSAL

Adversarial verification

PYTHON / ORIGINAL PROPOSAL

```python
    def test_overdue_review_does_not_release_money(self):
        self.assertTrue(self.case.due(999))
        self.assertEqual(self.case.state, "restricted")
        self.assertEqual(self.case.released, 0)

    def test_reconciliation_failure_preserves_approval(self):
        self.approve()
        with self.assertRaises(ValueError):
            self.execute(reconciled_cents=1)
        self.assertEqual(self.case.state, "exit_approved")
        self.assertEqual(self.case.released, 0)
        self.assertEqual(self.execute(), 200000100)
```

Literal lab tests; count and result come from an actual run.

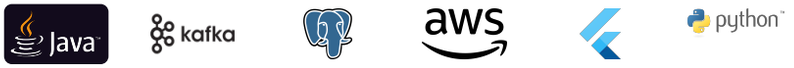
The 14 tests show that overdue review does not release money, failed reconciliation blocks execution and a reviewer does not automatically hold the executor role. They also cover another owner’s destination, missing evidence, stale version, replay and invalid values. A second execution cannot increase the transferred total.

The suite does not validate a policy’s legal correctness or document authenticity. The next level requires role-integration tests and simulation of a new restriction arriving between approval and sending. The goal is to prove that operational simplification creates no unauthorized release while retaining an auditable path for eligible cases.

Python - unittest

CASE 03 · EXTERNAL PROPOSAL

Measure the right journey



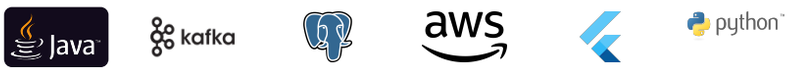
Metric	Numerator / denominator	Window and segment
Age p 95	Time per open case	Daily; grounds
Unowned	Unowned / active cases	Daily; category
Confirmed exit	Approval→execution time	Weekly; impediment
Control failure	Failures / audited executions	Continuous; version

Queue age must be segmented by grounds; mixing a document request with a court order can hide distinct priorities. The dashboard measures cases without an owner or scheduled review and tracks approval-to-confirmed-execution time. Individual case data remains restricted.

I would compare equivalent cohorts by category and complexity while preserving human review. Fewer contacts are beneficial only if they do not result from obstructed appeals. Unauthorized release, exposure of detection signals or unreconciled execution immediately stops the pilot. Baselines and operating limits must be set with risk, legal and operations rather than inferred from the reported amount.

CASE 03 · EXTERNAL PROPOSAL

Financial value with assumptions



Assumed scenario	Main input	Net/month	Payback (months)
Conservative	1,000 × 10 min × 25%	BRL -1,250.00	None
Base	2,500 × 15 min × 40%	BRL 12,500.00	14.40
Ambitious	4,000 × 20 min × 50%	BRL 42,000.00	4.29

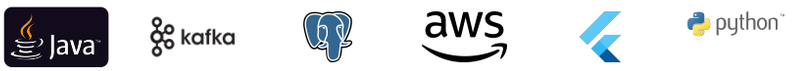
The calculation does not treat held funds as a benefit. Assumptions are 1,000 / 2,500 / 4,000 monthly reviews, 10 / 15 / 20 minutes saved per review and BRL 90.00 per hour. Only 25% / 40% / 50% of released capacity is assumed to become avoidable expenditure. Monthly operations of BRL 5,000.00 / BRL 10,000.00 / BRL 18,000.00 and a hypothetical BRL 180,000.00 implementation are deducted.

The formula is cases × minutes / 60 × hourly cost × realizable share, less operations. That share must be demonstrated through an actual spending change; without it, the corresponding financial benefit is zero even if the queue improves. Avoided compensation is excluded because no verifiable causal probability or cost supports it.

Assumed implementation budget: BRL 180,000.00. Annual net results before implementation: BRL -15,000.00 / BRL 150,000.00 / BRL 504,000.00. Every input is an assumption, not a Nubank baseline.

CASE 03 · EXTERNAL PROPOSAL

Accountable roadmap



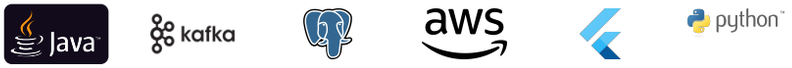
Proposed phase	Accountable role	Delivery / exit gate
Days 0-15	Legal + risk	Approved taxonomy
Days 16-30	Staff + operations	Roles and evidence
Days 31-60	Financial engineering	Reconciled shadow
Days 61-90	EM + governance	Limited pilot; review

Planning begins with a taxonomy of measures and named legal and operational owners. Architecture follows understanding where each team's authority ends. A closed account needs an exit path as explicit as the path that created its restriction.

Staff coordinates contracts and revalidation points; the EM secures legal, risk and support capacity at acceptance milestones. Shadow mode must reproduce previous decisions without executing them. Cases with unapproved grounds or no owner remain outside the pilot until resolved. The horizon is proposed rather than a commitment made by Nubank.

CASE 03 · EXTERNAL PROPOSAL

Rollout and operations

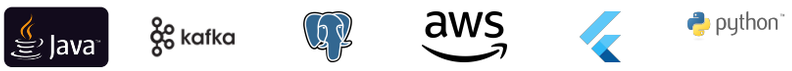


The initial canary covers a small set of restriction types and does not automate the substantive decision. Overdue-review alerts go to category owners; execution-divergence alerts go to the financial domain. Customers receive stage information rather than a calendar-based promise of release.

The operations drill includes a new restrictive order after approval, changed destination ownership and failure after sending an instruction. In the latter case, query and reconciliation precede any resend. Rollback freezes new executions through the new version and preserves audit history. A faster queue that loses grounds, evidence or appeal access is unacceptable.

CASE 03 · EXTERNAL PROPOSAL

My contribution



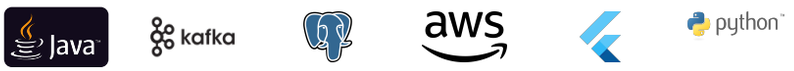
As Staff, I would help turn implicit states into a traceable protocol for appeals, review and execution. The rejected alternative is automatic release when a timer expires. It appears to solve delay but ignores active impediments and confuses a work deadline with financial authority.

As EM, I would organize specialist collaboration without promising that engineering alone can decide a regulatory question. This includes review capacity, support training and stopping criteria backed by accountable owners. My proposed value is connecting technical quality with operational responsibility, not presenting this external proposal as a past achievement.

In a leadership assessment, I would explain how to handle tension between customer urgency and a restriction that remains valid. The response is permitted information, accountable review and correct execution when authorized. I would ask the team to discuss an edge case and record its decision, including why a particular step should remain manual. This makes the project reviewable by specialists beyond engineering and avoids implying that faster software can substitute for the authority or evidence needed to make a substantive decision.

CASE 03 · EXTERNAL PROPOSAL

Handoff and decision



Local execution

python3 labs/case_03.py -v · 14 tests passed.

The reviewer follows due, decide and execute_exit in that order, observing that the first function does not change money. Then inject a mismatched balance and confirm that approval remains available for a later correct execution. The lab demonstrates a control; it is neither legal advice nor a banking closure system.

To proceed, the domain must provide a valid taxonomy, operator identity, destination-ownership confirmation and a reconciliation source. Investment depends on eligible volume and genuinely avoidable expenditure. If those contracts cannot be established, delivery should remain focused on tracking and queue quality while preserving existing execution.

- A timer does not authorize a transfer.
- A new restriction invalidates an old approval.
- Audit separates decision from execution.

Executable case lab on GitHub

CASE 04 · EXTERNAL PROPOSAL

The record and its outcome

04

Actual crop of the public source, captured 23 October 2026. The text below includes the company reply and reported outcome.

Resolved · RA 230743513

On October 31, 2025, a customer reported being unable to access the account because facial authorization depended on a broken camera. On November 3, Nubank said it had received a selfie with a code, internally validated the device and enabled its features. The customer marked the problem resolved. The source does not disclose exact duration, unavailable funds or how the alternative photo was captured.

This case does not establish a facial-recognition error. The reported barrier is physical inability to capture an image, and the response shows an assisted solution already existed. The proposal improves discovery, routing and tracking of that alternative without claiming Nubank lacked it. There is no public biometric photograph to reproduce; the capture documents the complaint and resolution. The technical discussion begins when repeating the same instruction is no longer useful.

Reclame AQUI - public complaint and response

CASE 04 · EXTERNAL PROPOSAL

Diagnosis before design

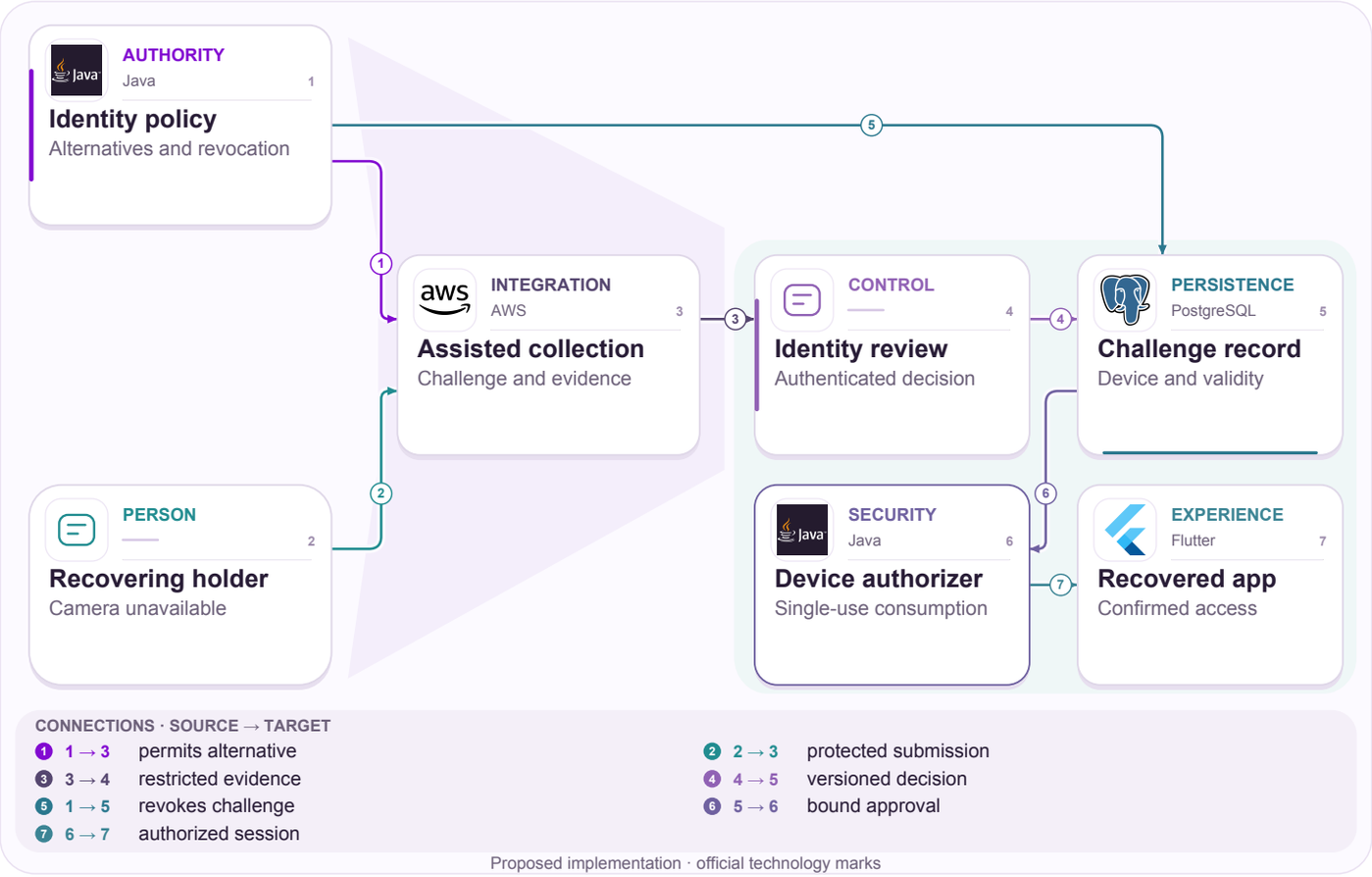
Denied permission, unavailable camera and unconfirmed identity are different reasons. Combining them in a generic message creates attempts that do not change device capability. The hypothesis is that diagnosing the barrier before capture reduces repeated routing while retaining the same identity requirement.

I would investigate local error sequences, attempts and contacts per journey without storing images in analytics. A sample of assisted recoveries would reveal where people abandon or restart. If routing is already clear and waiting occurs in specialist analysis, priority shifts to review capacity and quality. The complaint alone does not measure that distribution.

Local diagnosis must be treated as an untrusted signal. A modified client can declare that no camera exists, so this result only selects a guidance path and never lowers identity requirements. I would also investigate whether the alternative is usable by someone without another device, keeping that limitation explicit in research. The resolved case does not identify what enabled alternative capture, so the same option cannot be assumed for everyone. Accessibility review should examine actual available resources rather than treating successful completion by one person as universal usability.

CASE 04 · EXTERNAL PROPOSAL

Architecture boundaries



Independent proposed architecture; suggested technologies, not Nubank internal infrastructure.

The person submits evidence through a protected channel while policy determines acceptable alternatives. Review produces a decision bound to the challenge and intended device. Policy can also revoke the record before consumption.

The authorizer does not interpret a photograph or a support message; it consumes a valid approval once. The interface declares recovery only after authorization is confirmed. People and reviewers are roles, not branded software components.

CASE 04 · EXTERNAL PROPOSAL

Executable core

PYTHON / ORIGINAL PROPOSAL

```python
    def authorize(self, owner, device, challenge, now):
        with self.lock:
            if owner != self.owner or device != self.device:
                raise PermissionError("recovery binding")
            if type(now) is not int or now < 0:
                raise ValueError("invalid clock")
            if now >= self.expires_at:
                raise ValueError("challenge expired")
            if challenge != self.challenge:
                raise PermissionError("challenge mismatch")
            if self.state != "approved" or not self.evidence:
                raise ValueError("review required or challenge consumed")
            self.state = "authorized"
            self.challenge = None
            return {"device": self.device, "authorized": True}
```

Literal excerpt from the executed independent lab; remaining validation is in the complete file.

Recovery.authorize requires binding to the holder, device and challenge, plus a valid deadline and prior approval. Completion consumes the challenge and prevents reuse. Invalidation occurs under the same lock as precondition checking, so two concurrent calls cannot authorize using the same local evidence.

The code neither recognizes faces nor decides whether a person is legitimate. review receives an identity decision already validated by a trusted adapter in the model. This boundary matters: testing a state machine does not prove biometric security. Production evidence, operators and sessions need appropriate authentication and persistence.

Python - unittest

Executable case lab on GitHub

CASE 04 · EXTERNAL PROPOSAL

Stack implementation

DART / ORIGINAL PROPOSAL

```dart
String nextStep(String capability, String verification) {
  // Proposed copy routing; never grants account access.
  if (capability == 'camera_unavailable') {
    return 'Open the approved assisted recovery path';
  }
  if (capability == 'permission_denied') {
    return 'Review camera permission instructions';
  }
  if (verification == 'not_confirmed') {
    return 'Follow the identity review instructions';
  }
  return 'Continue the approved capture flow';
}
```

Proposed integration example, not executed; this is not Nubank internal code.

The illustrative Dart separates hardware inability from verification failure. The experience should explain the next available action without recommending shortcuts that remove controls. A person without a working camera may need an assisted channel or another approved method whose eligibility belongs to identity policy.

I would review accessibility from the first screen: predictable focus, spoken instructions and preserved progress after interruption. The alternative should not require discovering a support contact through trial and error. The excerpt was not compiled and requests no real permissions; integration requires representative-device testing and security review.

Dart - Branches

CASE 04 · EXTERNAL PROPOSAL

Contracts and ownership

Field / rule	Owner	Invariant
challenge_id	Identity	Single use
device_binding	Authorizer	Immutable context
evidence_ref	Review	No biometrics in logs
expires_at	Policy	Exclusive deadline

JSON / ORIGINAL PROPOSAL

```json
{
  "schema_version": 1,
  "challenge_id": "synthetic-one-use",
  "device_binding": "synthetic-device",
  "evidence_ref": "restricted-reference",
  "expires_at": 100,
  "state": "approved",
  "authorized": false
}
```

Illustrative proposed contract; synthetic values, not customer data.

A challenge needs identity, validity, intended device and a decision reference. The contract does not carry selfies or documents into the general UI; it transmits the minimum necessary state. Revocation must override an approval not yet consumed, and queries must reflect that without silently restarting the case.

Retention policy distinguishes biometric material, review decisions and operational logs. Each category needs appropriate access and duration. Evidence storage should not become a permanent archive for technical convenience. Ownership comes from a verified session, not a value chosen in a public request body, even though the lab accepts strings directly.

CASE 04 · EXTERNAL PROPOSAL

Adversarial verification

PYTHON / ORIGINAL PROPOSAL

```python
    def test_parallel_authorization_consumes_once(self):
        self.approve()
        def attempt(_):
            try:
                return self.auth()["authorized"]
            except (ValueError, PermissionError):
                return False
        with ThreadPoolExecutor(max_workers=4) as pool:
            result = list(pool.map(attempt, range(4)))
        self.assertEqual(result.count(True), 1)

    def test_camera_problem_is_not_identity_proof(self):
        with self.assertRaises(ValueError):
            self.auth()
        self.assertEqual(self.r.state, "collecting")
```

Literal lab tests; count and result come from an actual run.

The 14 tests verify single use under concurrency, exact expiration, a different device, wrong holder, revocation, missing evidence and misleading truthy values. The most important test confirms that declaring a camera problem authorizes nothing. Another shows that an incorrect challenge does not make a later attempt succeed without review.

Real capture, liveness, accessibility and storage tests are still missing. Those belong to different layers with their own criteria. I would add network-interruption and device-change drills during recovery, checking that an approved decision remains bound to the correct context rather than moving to another device.

Python - unittest

CASE 04 · EXTERNAL PROPOSAL

Measure the right journey

Metric	Numerator / denominator	Window and segment
Legitimate completion	Valid / eligible recoveries	7 days; barrier
Redundant attempt	Unchanged attempts / journeys	Session; device
Queue p 95	Time to review	Daily; alternative
Account takeover	Confirmed cases / recoveries	Risk window; cohort

Legitimate completion and account takeover must be read together. Higher conversion that authorizes the wrong people is a regression even if support volume falls. Collection segments physical barriers, permissions and review without making raw biometrics an analytics dimension.

The pilot retains a comparable cohort of eligible journeys and observes repeat contacts for seven days, plus fraud signals over the identity team’s chosen window. Comprehension testing includes people with capture limitations. Any confirmed increase in unauthorized access stops expansion; longer review time also requires checking whether the flow merely moved work to specialists.

CASE 04 · EXTERNAL PROPOSAL

Financial value with assumptions

Assumed scenario	Main input	Net/month	Payback (months)
Conservative	3,000; 0.2 contact; 10% review	BRL -3,000.00	None
Base	8,000; 0.4 contact; 8% review	BRL 11,280.00	9.75
Ambitious	15,000; 0.6 contact; 5% review	BRL 57,500.00	1.91

Assumptions are 3,000 / 8,000 / 15,000 monthly recoveries, 0.2 / 0.4 / 0.6 contacts avoided per journey and BRL 9.00 per contact. The alternative may add review: assume 0.10 / 0.08 / 0.05 additional reviews per journey at BRL 18.00. Operations add BRL 3,000.00 / BRL 6,000.00 / BRL 10,000.00 monthly, with hypothetical implementation of BRL 110,000.00.

The formula deducts additional review before operations. Better routing is therefore not automatically cheaper. No customer balance enters the calculation because none was published. Residual fraud must be measured as a guardrail and budgeted if an attributable difference exists; no unsupported antifraud benefit is assumed. Support capacity becomes cash savings only when realizable spending falls.

Assumed implementation budget: BRL 110,000.00. Annual net results before implementation: BRL -36,000.00 / BRL 135,360.00 / BRL 690,000.00. Every input is an assumption, not a Nubank baseline.

CASE 04 · EXTERNAL PROPOSAL

Accountable roadmap

Proposed phase	Accountable role	Delivery / exit gate
Days 0-15	Identity + accessibility	Approved alternatives
Days 16-30	Mobile + privacy	Reasons without biometrics
Days 31-60	Staff + review	Binding and revocation
Days 61-90	EM + support	Pilot with capacity

The first milestone aligns identity, mobile and accessibility on alternatives already available. The second instruments barrier reasons without collecting sensitive material. The third builds resumability and tracking in a controlled environment. Only then does a pilot measure whether people reach the correct channel with fewer repeated attempts.

The EM must reserve human-review capacity before increasing intake. Staff defines binding and revocation contracts with device owners. More routing without review capacity can worsen waiting, so acceptance covers queue quality rather than screens alone. The proposed timeline may change after discovery.

CASE 04 · EXTERNAL PROPOSAL

Rollout and operations

Rollout does not begin by disabling facial recognition. It begins by making an approved alternative discoverable and trackable. Identity on-call receives alerts for repeated use, mismatched context and consumption failures; support is instructed not to request material through improvised channels.

Drills include a challenge expiring during review, holder revocation and approval followed by connection loss. Resumption queries existing state before opening another case. Rollback preserves decisions and retains previous assisted channels. If evidence-to-device binding is uncertain, the response is renewed authorized review rather than release for convenience.

CASE 04 · EXTERNAL PROPOSAL

My contribution

As Staff, I would separate device capability from identity confidence so a UI simplification cannot contaminate authorization. I would reject a rule accepting any photograph after a camera error: a physical problem does not make evidence more trustworthy. A sound alternative requires policy, review and single-use binding.

As EM, I would coordinate mobile, identity, privacy and support around a journey the organization can operate. This includes specialist capacity and training as well as code. My proposed contribution makes security and accessibility joint quality criteria while remaining transparent about what the lab demonstrates and what requires further validation.

I would also avoid a conversion target that pressures reviewers to approve uncertain cases. The EM should track decision quality, workload and escalation while Staff keeps the protocol capable of representing waiting and additional review. Recovery may legitimately take longer when evidence is missing. The desired improvement removes unproductive repetition and improves guidance while preserving the security decision the case requires. That distinction helps a team discuss tradeoffs honestly and prevents a convenient completion metric from becoming the only definition of success.

CASE 04 · EXTERNAL PROPOSAL

Handoff and decision

Local execution

python3 labs/case_04.py -v · 14 tests passed.

The reviewer first calls authorize without review and observes rejection. Next approve a synthetic reference, change the device and observe another rejection; only the correct context consumes the challenge. This walkthrough demonstrates that capture capability and authorization are separate contracts.

Handoff requires an approved alternative-method policy, session service, restricted storage and retention plan. It also requires real-hardware testing, interruption support and fraud monitoring. Progress depends on better legitimate recovery without weakened identity. If review capacity dominates the problem, I prioritize the queue before adding more app steps.

- An unavailable camera is not identity proof.
- Expired or revoked challenges cannot authorize.
- Sensitive material stays out of analytics.

Executable case lab on GitHub

CASE 05 · EXTERNAL PROPOSAL

The record and its outcome

05

Actual crop of the public source, captured 23 October 2026. The text below includes the company reply and reported outcome.

Answered · RA 242544191

On March 6, 2026, a customer reported changing phones, not knowing or accessing the previous email and receiving no code at a newly supplied address. On March 13, Nubank explained additional checks and said a device had been authorized on March 9. It was awaiting a reply to private guidance. The record is answered, with no public final confirmation from the customer. The cause of delivery failure is unproven.

An authorized device does not by itself prove that the person completed the entire reported journey. The response may indicate meaningful progress, but it does not turn answered into resolved. This case concerns email, not SMS or a broken camera. Private messages were not inspected. The proposal uses the reported barrier to examine the contract between updated contact, delivered code, validated code and a new-device session.

Reclame AQUI - public complaint and response

CASE 05 · EXTERNAL PROPOSAL

Diagnosis before design

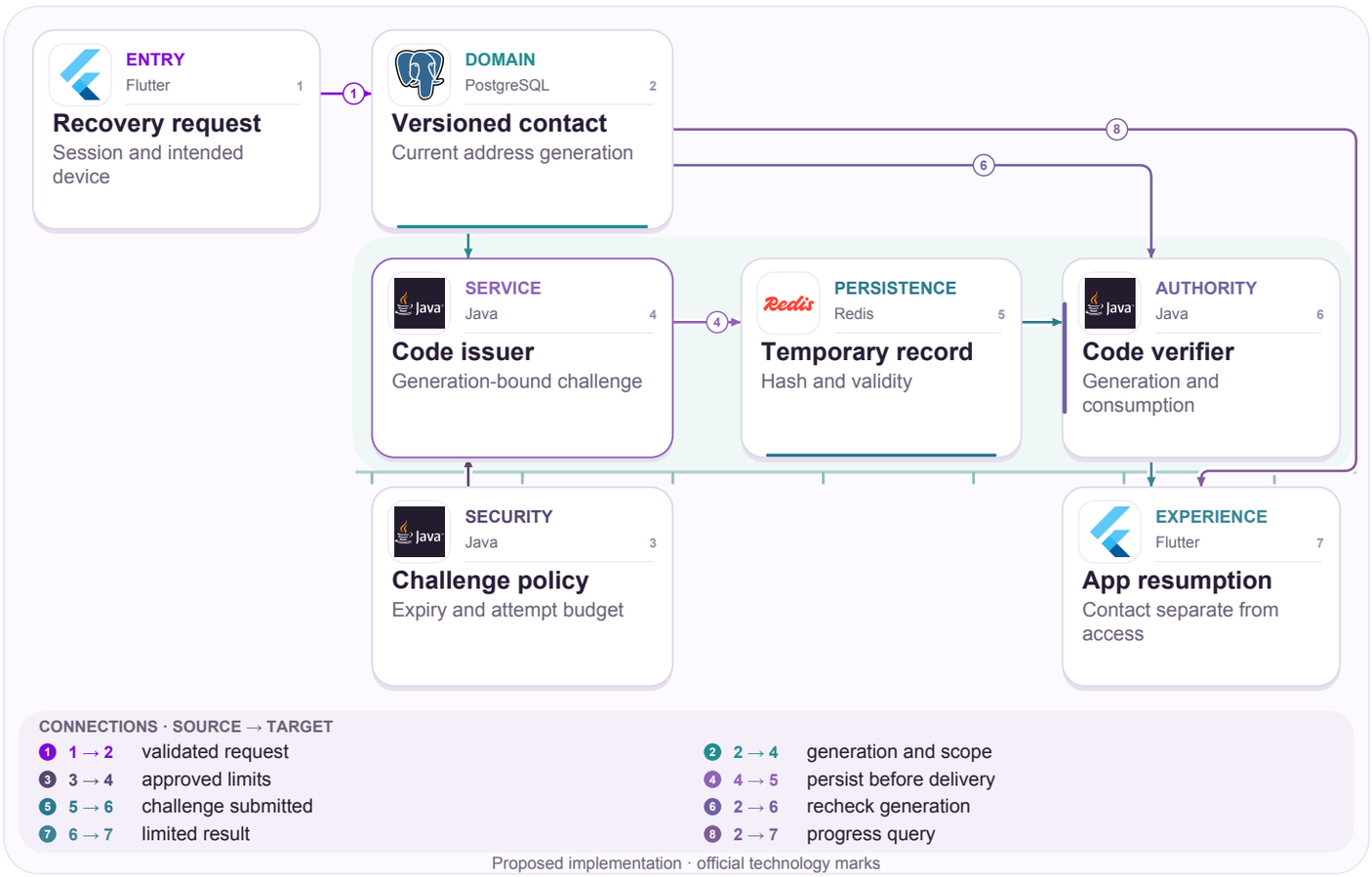
The report supports investigation of delivery, generation binding and journey comprehension, but identifies no provider, bounce or spam filter. I would trace a recovery_id across change request, issuance, provider acceptance, delivery feedback and validation. SMTP acceptance means neither reading nor identity proof.

One hypothesis is that the customer cannot distinguish the active contact from the challenge that remains valid after resending. Logs and research can refute this: if contact and challenge are consistent, the bottleneck may lie elsewhere. I would not claim a root cause from a “did not arrive” message alone.

Delivery investigation must acknowledge incomplete feedback. Some providers report acceptance without delivery confirmation; others produce delayed or duplicated events. I would document which signals are observations and which are inferences, keeping customer messages consistent with that distinction. A newly supplied address still requires identity validation before changing account contact. Research should observe whether people understand that confirming contact and recovering access can require different steps. Otherwise, a technically accurate delivery message may create the false expectation that the account should already be available.

CASE 05 · EXTERNAL PROPOSAL

Architecture boundaries



Independent proposed architecture; suggested technologies, not Nubank internal infrastructure.

The versioned contact feeds issuance and verification. Policy sets limits while the record retains the current challenge. The verifier rechecks generation before consuming a code, preventing a concurrent change from leaving an old challenge usable.

The app receives contact progress separately from access. Confirming an address is only one step; device authorization remains in the identity domain. The diagram does not treat the delivery provider as an authority on account ownership.

CASE 05 · EXTERNAL PROPOSAL

Executable core

PYTHON / ORIGINAL PROPOSAL

```python
    def verify(self, owner, generation, code, now):
        if owner != self.owner:
            raise PermissionError("recovery scope")
        if type(now) is not int or now < 0 or not isinstance(code, str):
            raise ValueError("invalid request")
        with self.lock:
            current = self.challenge
            if current is None:
                raise ValueError("challenge unavailable")
            issued_generation, digest, issued_at, expiry, attempts = current
            if type(generation) is not int or generation != self.generation:
                raise ValueError("obsolete contact generation")
            if generation != issued_generation or not issued_at <= now < expiry:
                raise ValueError("challenge expired or not yet valid")
            if attempts >= 3:
                raise ValueError("attempt budget exhausted")
            current[4] += 1
            candidate = hashlib.sha256(code.encode()).digest()
            if not hmac.compare_digest(candidate, digest):
                raise PermissionError("invalid code")
            self.verified, self.challenge = True, None
            return True
```

Literal excerpt from the executed independent lab; remaining validation is in the complete file.

ContactRecovery.verify rejects obsolete generation, invalid time, missing challenge and exhausted attempt budget. A hash compares the code without retaining its plain text in the educational record. Success consumes the challenge and sets verified to true; device_authorized remains false.

The lab accepts fixed codes for reproducible tests. A real service must generate random secrets, limit distributed abuse and protect storage. A simple hash of a short code is insufficient against database extraction; appropriate secret and key design is required. The purpose is lifecycle and binding verification rather than a production-ready authentication service.

Python - unittest

Executable case lab on GitHub

CASE 05 · EXTERNAL PROPOSAL

Stack implementation

TYPESCRIPT / ORIGINAL PROPOSAL

```typescript
// Proposed presentation contract; validate network input separately.
type RecoveryView =
  | { state: 'contact_pending'; generation: number }
  | { state: 'contact_verified'; generation: number }
  | { state: 'device_authorized'; deviceRef: string };
function accessReady(view: RecoveryView): boolean {
  return view.state === 'device_authorized';
}
function contactReady(view: RecoveryView): boolean {
  return view.state === 'contact_verified'
      || view.state === 'device_authorized';
}
```

Proposed integration example, not executed; this is not Nubank internal code.

The TypeScript contract uses an explicit union to distinguish pending contact, confirmed contact and authorized device. This helps UI review, but static types do not validate network input. The adapter must still validate schema, origin and scope before constructing the object.

Messages neither expose a complete email address nor reveal account existence to unauthenticated requesters. Resend displays a policy-consistent deadline and preserves the journey. I would reuse the existing delivery provider where possible, adding correlation and event handling instead of assuming that switching providers solves an unknown cause.

TypeScript - Narrowing

CASE 05 · EXTERNAL PROPOSAL

Contracts and ownership

Field / rule	Owner	Invariant
recovery_id	Identity	Holder scope
contact_generation	Contact	Authorized changes only
challenge_hash	Security	Never log code
delivery_state	Adapter	Accepted ≠ read

JSON / ORIGINAL PROPOSAL

```json
{
  "schema_version": 1,
  "recovery_id": "synthetic-recovery",
  "contact_generation": 2,
  "delivery_state": "accepted",
  "contact_verified": false,
  "device_authorized": false,
  "expires_at": 160
}
```

Illustrative proposed contract; synthetic values, not customer data.

Generation changes only after an authorized contact update. Each issuance replaces the prior challenge within that generation and applies cooldown. TTL belongs to the challenge while attempt limits belong to abuse policy. Production needs additional account, device and source limits that survive restarts and multiple instances.

Delivery events should distinguish accepted, bounced and expired without concluding that a message was read. The public record provides none of these events. Logs exclude codes and full addresses; investigation uses protected references. Contact change must not allow an attacker to choose a destination freely and become the holder by receiving their own code.

CASE 05 · EXTERNAL PROPOSAL

Adversarial verification

PYTHON / ORIGINAL PROPOSAL

```python
    def test_contact_change_invalidates_old_code(self):
        self.r.change_contact("a", "other@example.test", True)
        self.r.issue("a", "654321", 110, 60)
        with self.assertRaises(ValueError):
            self.r.verify("a", 1, "123456", 120)
        self.assertTrue(self.r.verify("a", 2, "654321", 120))

    def test_contact_proof_does_not_authorize_device(self):
        self.assertTrue(self.r.verify("a", 1, "123456", 120))
        self.assertTrue(self.r.verified)
        self.assertFalse(self.r.device_authorized)
```

Literal lab tests; count and result come from an actual run.

The 14 tests cover contact change with an old code, resend, attempt limits, expiration, future clock, replay and wrong holder. The state-separation test confirms that email validation does not authorize a device. This invariant prevents a delivery mechanism from becoming complete identity proof.

The suite sends no email and measures no deliverability. Provider events, cross-process limits and timing-based enumeration tests remain missing. Before production, I would add coordinated resend attacks and two sessions changing contact concurrently, plus integration with the real identity-recovery policy.

Python - unittest

CASE 05 · EXTERNAL PROPOSAL

Measure the right journey

Metric	Numerator / denominator	Window and segment
Confirmed contact	Confirmations / journeys	7 days; provider
Resumption	Completed / started resumptions	Session; version
Repeat contact	Repeat contacts / recovery_id	7 days; reason
Obsolete code	Invalid accepts / verifications	Continuous; generation
Abuse	Confirmed cases / journeys	Risk window

The funnel must show issuance, observable delivery, contact validation and authorization without treating them as equivalent. I would measure repeat contacts per recovery and time per stage, segmented by provider and app version where volume is sufficient to preserve privacy.

The experiment compares eligible journeys of similar complexity and observes legitimate recovery, messages sent and review load. Better delivery accompanied by more abuse does not pass acceptance. Expansion stops for accepted obsolete codes, account-data leakage or account takeover attributable to the flow. Baselines remain unknown until instrumentation and identity-team validation.

CASE 05 · EXTERNAL PROPOSAL

Financial value with assumptions

Assumed scenario	Main input	Net/month	Payback (months)
Conservative	10,000; Δ 0.05; 15,000 sends	BRL -450.00	None
Base	30,000; Δ 0.12; 45,000 sends	BRL 20,450.00	4.89
Ambitious	60,000; Δ 0.18; 90,000 sends	BRL 71,700.00	1.39

Assumptions are 10,000 / 30,000 / 60,000 monthly journeys, repeat-contact reductions of 0.05 / 0.12 / 0.18 and BRL 8.00 marginal cost per contact. Additional messages are 15,000 / 45,000 / 90,000 at BRL 0.03 each, plus BRL 4,000.00 / BRL 7,000.00 / BRL 12,000.00 monthly operations. Hypothetical implementation costs BRL 100,000.00.

Net benefit is journeys × avoided repeat contacts × cost, less messages and operations. Global average revenue per customer is not used to value recovered access; that would require causal incremental revenue and margin evidence unavailable in this complaint. The conservative scenario may be negative, showing that sending more messages without sufficiently reducing confusion does not justify investment through avoided cost.

Assumed implementation budget: BRL 100,000.00. Annual net results before implementation: BRL -5,400.00 / BRL 245,400.00 / BRL 860,400.00. Every input is an assumption, not a Nubank baseline.

CASE 05 · EXTERNAL PROPOSAL

Accountable roadmap

Proposed phase	Accountable role	Delivery / exit gate
Days 0-15	Identity + delivery	Event map
Days 16-30	Staff + security	Generation and limits
Days 31-60	Mobile + product	Shadow resumption
Days 61-90	EM + support	Pilot without new abuse

The roadmap starts with an inventory of available events and contact-recovery methods. Identity validates which changes require review; the delivery owner explains what callbacks actually mean. Product defines progress messages that do not reveal account data.

Shadow mode compares issued and consumed generations before changing resend or experience. The EM coordinates provider dependencies and review capacity; Staff resolves concurrency and policy persistence. If observability reveals a predominantly external problem, the plan should prioritize the adapter and communication instead of unnecessarily rebuilding authentication.

CASE 05 · EXTERNAL PROPOSAL

Rollout and operations

During canary rollout, old and new issuers must share generation control. A UI flag cannot stop an incompatible backend from accepting an old code. Migration requires compatibility or explicit exclusion of versions that do not understand the new identity.

The drill includes late delivery feedback, concurrent resend and contact change while a code is in transit. Operations must distinguish provider delay from an already-revoked challenge. Rollback preserves generations and limits rather than reactivating old secrets. Abuse alerts go to identity; aggregate delivery failures go to the channel owner.

CASE 05 · EXTERNAL PROPOSAL

My contribution

As Staff, I would propose generation binding and the separation between contact possession and device authorization. I would reject accepting any unexpired code: two contacts and two resends can leave several contradictory proofs apparently valid. The challenge must be current in the correct context.

As EM, I would organize delivery across security, messaging and mobile, including abuse tests in the definition of done. I would also ensure support sees the same progress without accessing secrets. These are proposed technical and coordination actions, not claims of authority or prior results inside Nubank.

The technical decision must withstand pressure to remove steps without understanding their purpose. I would explain which controls are necessary for risk and which repetitions result only from poorly preserved state. As EM, I would organize a support review using anonymized journeys so the people receiving customer questions participate in design. This produces a more operable solution and avoids adding another resend button without fixing context. The review should end with concrete examples of what the interface can truthfully say at each stage and when a specialist must intervene.

CASE 05 · EXTERNAL PROPOSAL

Handoff and decision

Local execution

python3 labs/case_05.py -v · 14 tests passed.

The reviewer starts with change_contact, issues a code, changes contact again and attempts to consume the previous generation. Then verify that the new contact can be confirmed without authorizing the device. Tests use example.test addresses and synthetic codes; they make no real delivery or access requests.

Handoff requires identity service, secret generation, protected storage, delivery callbacks and durable rate limiting. Proceeding requires measurable legitimate recovery and abuse guardrails. If only provider acceptance improves, without completion or fewer repeat contacts, the pilot has not demonstrated the intended value and needs renewed diagnosis.

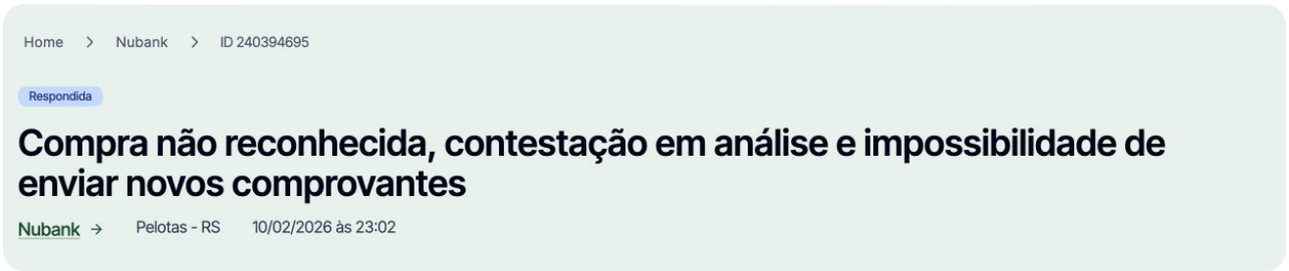
- Verified contact does not authorize a device.
- Resend invalidates the previous challenge.
- No code appears in logs.

Executable case lab on GitHub

CASE 06 · EXTERNAL PROPOSAL

The record and its outcome

06



Actual crop of the public source, captured 23 October 2026. The text below includes the company reply and reported outcome.

Answered · RA 240394695

On February 10, 2026, a customer reported disputing an unrecognized Airbnb purchase and being unable to add evidence while it was under review. On February 18, Nubank reported provisional credit, a dispute opened on February 11, cancellation of the virtual card and disabling the automatic updater. Amounts are masked. There is no public final outcome or confirmation that the attachment channel was reopened.

The concrete barrier is adding material after opening the case, not proving here whether the purchase was fraudulent. The confirmation attributed to Airbnb is in private material that was not examined. Provisional credit must remain distinct from the dispute outcome. The proposal does not promise that a card network will accept every late document; it aims to preserve material, explain its stage and provide a traceable admissibility decision.

Reclame AQUI - public complaint and response

CASE 06 · EXTERNAL PROPOSAL

Diagnosis before design

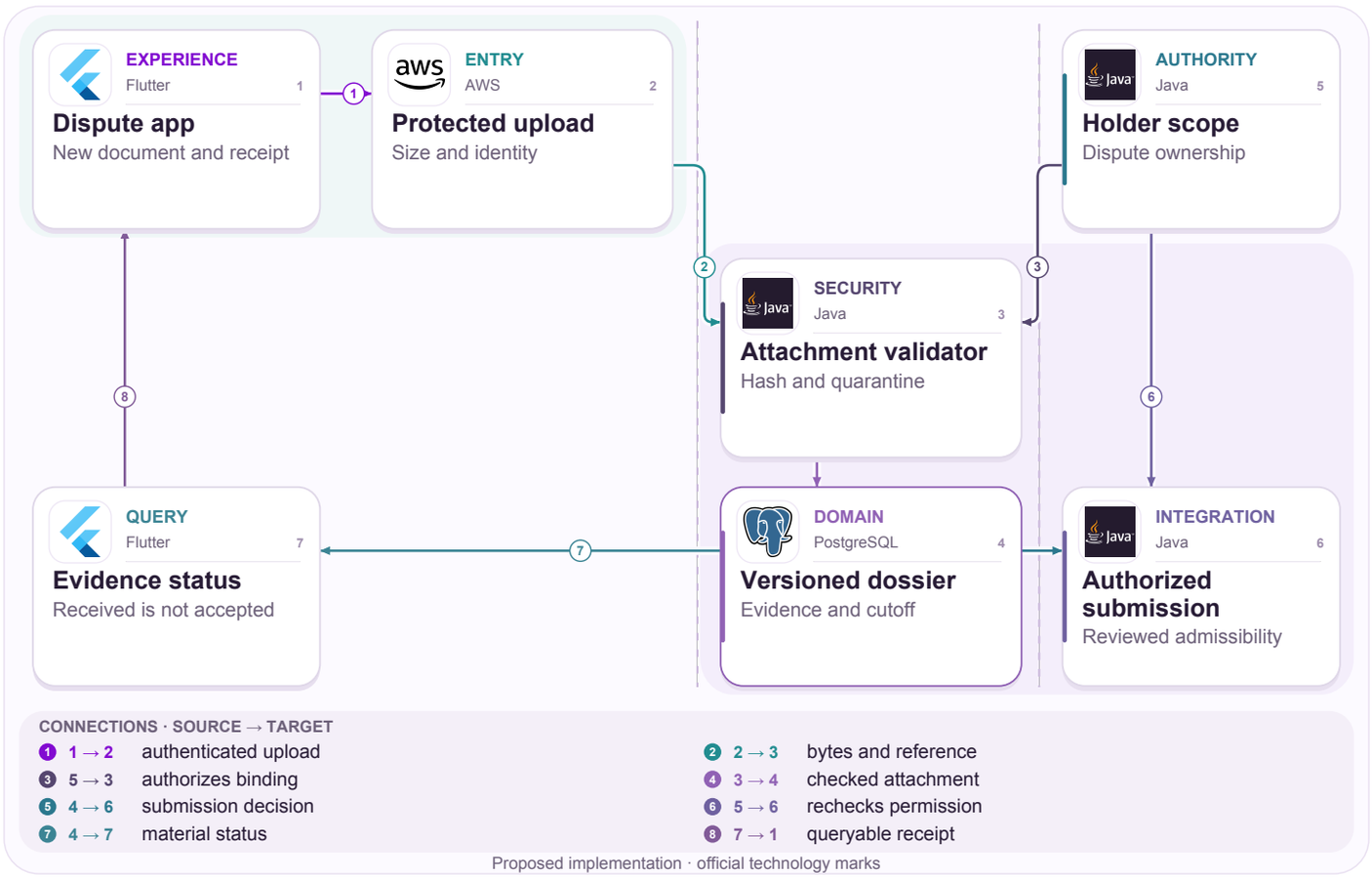
An “under review” state may be blocking every edit even though adding evidence and changing the original claim carry different risks. This is a hypothesis, not a proven app cause. I would map internal stages and external deadlines to identify where additional evidence can still be considered.

Investigation compares upload attempts, contacts requesting submission and analyst decisions. It must distinguish UI restrictions, format limits, quarantine and network cutoffs. If material was already outside the applicable deadline, improvement may mean an explanation and an allowed route rather than implying engineering can unilaterally reopen the dispute.

I would also distinguish supplementary evidence from correcting the disputed transaction itself. The former may be appended without changing case identity; the latter may require reviewing the object of the dispute. If both use one generic edit button, it becomes difficult to explain what was actually received. The proposed design records submission intent and traceability so an analyst can determine its relationship to earlier material. This avoids treating every later document as either a harmless addition or an unauthorized change when its meaning requires judgment.

CASE 06 · EXTERNAL PROPOSAL

Architecture boundaries



Independent proposed architecture; suggested technologies, not Nubank internal infrastructure.

Upload enters quarantine and retains identity before reaching the dossier. Holder scope protects both intake and submission. Dossier policy determines what can proceed after review without deleting earlier evidence.

The app response distinguishes received, being checked, available for analysis and submitted. Receiving bytes does not confirm merit or external acceptance. The design also keeps this state separate from provisional credit or a final financial decision.

CASE 06 · EXTERNAL PROPOSAL

Executable core

PYTHON / ORIGINAL PROPOSAL

```python
    def append(self, owner, upload_id, content, now):
        if owner != self.owner:
            raise PermissionError("dispute scope")
        if not upload_id or type(content) is not bytes:
            raise ValueError("upload identity and bytes required")
        if not 0 < len(content) <= 1024 * 1024:
            raise ValueError("attachment size")
        if type(now) is not int or now < 0:
            raise ValueError("clock")
        digest = hashlib.sha256(content).hexdigest()
        with self.lock:
            if upload_id in self.seen:
                previous = self.items[self.seen[upload_id]]
                if previous["digest"] != digest:
                    raise ValueError("upload identity conflict")
                return dict(previous)
            item = dict(version=len(self.items) + 1, digest=digest,
                        received_at=now, state="quarantined",
                        late=now >= self.cutoff)
            self.seen[upload_id] = len(self.items)
            self.items.append(item)
            return dict(item)
```

Literal excerpt from the executed independent lab; remaining validation is in the complete file.

Dossier.append checks scope, size and upload identity before creating a version. Repeating the same key and content returns the previous record; reusing the key with different bytes creates a conflict. A copy is returned so callers cannot accidentally mutate internal state.

Material received after cutoff is retained as late, but submit does not promise submission in that window. The scanner is a simulated boundary: a trusted boolean represents an already-verified external verdict. The lab neither scans malware nor stores cloud files. Its contract preserves versions and separates receipt from eligibility.

Python - unittest

Executable case lab on GitHub

CASE 06 · EXTERNAL PROPOSAL

Stack implementation

TYPESCRIPT / ORIGINAL PROPOSAL

```typescript
// Proposed metadata only; no storage permissions implemented here.
type EvidenceRecord = {
  dossierId: string;
  uploadId: string;
  version: number;
  originalObjectRef: string;
  digest: string;
  state: 'quarantined' | 'reviewable' | 'rejected' | 'submitted';
  late: boolean;
};
function readyForReview(item: EvidenceRecord): boolean {
  return item.state === 'reviewable';
}
```

Proposed integration example, not executed; this is not Nubank internal code.

The storage example defines references and states rather than complete permissions. In an S 3 integration, I would design restricted upload, quarantine, content assessment and release to authorized analysts. Original objects and derivatives need distinct identities so metadata removal does not destroy the original evidentiary reference.

The dispute service decides admissibility and deadline; storage does not assume that authority. Temporary URLs must not become public history links. The UI can show a safe name, size and receipt without rendering active content. This TypeScript example is illustrative and was neither executed nor connected to AWS services.

Amazon S3 - User Guide

CASE 06 · EXTERNAL PROPOSAL

Contracts and ownership

Field / rule	Owner	Invariant
upload_id + digest	Ingestion	No replacement
dossier_version	Dispute	Preserved history
scan_state	Security	Quarantine first
cutoff + rule	Operations	Receive ≠ submit

JSON / ORIGINAL PROPOSAL

```json
{
  "schema_version": 1,
  "dossier_id": "synthetic-dispute",
  "upload_id": "synthetic-upload",
  "dossier_version": 2,
  "digest": "sha256-reference",
  "state": "quarantined",
  "late": false,
  "provisional_credit_is_final": false
}
```

Illustrative proposed contract; synthetic values, not customer data.

The contract associates upload_id, dossier version, digest and receipt time. A digest detects changed bytes but does not establish that content is truthful. Substantive review remains necessary and may reject technically intact material.

The model does not allow silent attachment replacement. Corrections become new versions with an explicit relationship to the previous one. Deadlines need a source, time zone and applicable rule reviewed with card-network operations. Privacy requires separating receipt visibility from document access. Retention must consider dispute and relevant obligations rather than defaulting to unlimited storage.

CASE 06 · EXTERNAL PROPOSAL

Adversarial verification

PYTHON / ORIGINAL PROPOSAL

```python
    def test_late_evidence_is_retained_without_submission_promise(self):
        item = self.upload(now=101)
        self.d.scan(item["version"], True, True)
        with self.assertRaises(ValueError):
            self.d.submit(item["version"], "dispute-analyst", True)
        self.assertEqual(len(self.d.items), 1)
        self.assertEqual(self.d.items[0]["state"], "reviewable")

    def test_reused_upload_id_cannot_replace_evidence(self):
        original = self.upload()
        with self.assertRaises(ValueError):
            self.upload(data=b"replacement")
        self.assertEqual(self.d.items[0]["digest"], original["digest"])
```

Literal lab tests; count and result come from an actual run.

The 14 tests verify retained late material without a submission promise, identity conflict, quarantine, rejected content, scope, replay and mutation of returned data. The exact cutoff is considered late in this model and must be checked against the real rule before integration.

The replacement test protects earlier evidence while the quarantine test prevents completed upload from appearing immediately usable. The suite does not validate real file types, antivirus, accessibility or network rules. I would add malformed-file and interrupted-upload tests, plus restoring the relationship after a temporary URL expires.

Python - unittest

CASE 06 · EXTERNAL PROPOSAL

Measure the right journey

Metric	Numerator / denominator	Window and segment
Valid receipt	Linked / eligible uploads	Daily; stage
Review p 95	Receipt→admissibility time	Weekly; cutoff
Repeat contact	Attachment contacts / dossiers	7 days; reason
Integrity	Failures / audited attachments	Continuous; version
Triage cost	Minutes / eligible dossier	Weekly; material

Success is not simply more attachments. The main metric tracks eligible material received with a receipt and reviewed within a useful window. More irrelevant documents can increase cost and delay decisions, so analyst time and admissibility must also be observed.

The pilot compares disputes at similar stages and with similar reasons, reviewed by analysts familiar with deadlines. Repeat contacts are measured per dossier over seven days. No chargeback result is attributed to the improvement without an appropriate comparison because merit and merchant behavior affect outcomes. Document exposure, evidence substitution or a missed deadline immediately stops expansion.

CASE 06 · EXTERNAL PROPOSAL

Financial value with assumptions

Assumed scenario	Main input	Net/month	Payback (months)
Conservative	2,000 × 4 min × 25%	BRL -1,100.00	None
Base	6,000 × 6 min × 40%	BRL 10,200.00	13.73
Ambitious	12,000 × 8 min × 50%	BRL 46,400.00	3.02

The financial model uses analyst rework without inventing the masked purchase amount. Assumptions are 2,000 / 6,000 / 12,000 monthly dossiers, 4 / 6 / 8 minutes saved per dossier, BRL 75.00 per hour and realizable shares of 25% / 40% / 50%. Additional uploads are 3,000 / 9,000 / 18,000 at BRL 0.20 each, plus BRL 3,000.00 / BRL 6,000.00 / BRL 10,000.00 monthly operations. Hypothetical implementation costs BRL 140,000.00.

The calculation deducts file checks and operations from monetizable capacity. If capacity does not lower spending, I report hours rather than cash. Principal potentially recovered to the customer stays outside this formula. Improved dispute outcomes enter a separate analysis only after causality and the actual beneficiary are established.

Assumed implementation budget: BRL 140,000.00. Annual net results before implementation: BRL -13,200.00 / BRL 122,400.00 / BRL 556,800.00. Every input is an assumption, not a Nubank baseline.

CASE 06 · EXTERNAL PROPOSAL

Accountable roadmap

Proposed phase	Accountable role	Delivery / exit gate
Days 0-15	Dispute operations	Stages and cutoffs
Days 16-30	Security + privacy	Custody and access
Days 31-60	Staff + mobile	Receipt and quarantine
Days 61-90	EM + analysts	Pilot; triage cost

Dispute operations leads the stage and cutoff map; security defines the upload boundary; privacy reviews custody and retention. Staff connects these contracts so an apparently small app change does not create an ungoverned document channel.

The first prototype receives synthetic attachments, generates a receipt and shows analysts only released material. A limited cohort then validates usefulness and cost. The EM secures training and capacity for greater intake. If an external cutoff prevents consideration, delivery is limited to a permitted route and explanation without promising that every piece of evidence changes the result.

CASE 06 · EXTERNAL PROPOSAL

Rollout and operations

Rollout enables receipt before automating any external submission. Analysts remain accountable for admissibility; a flag cannot make every attachment accepted evidence. Security on-call watches quarantine failures while dispute operations tracks material near cutoff.

The drill injects duplicate upload, an unavailable scanner and a delayed callback. The safe state keeps the file restricted and explains pending work. Rollback closes new intake through the new flow while preserving receipts and authorized access to received material. Preserving evidence is as important as avoiding improper submission.

CASE 06 · EXTERNAL PROPOSAL

My contribution

As Staff, I would propose immutable additional evidence rather than freely reopening the entire form. This preserves the original claim and makes changes auditable. The rejected alternative is overwriting the earlier attachment: it simplifies the screen but loses context and may confuse review.

As EM, I would coordinate dispute operations, mobile, security and privacy, placing analyst capacity and training in the same plan as development. The proposed contribution turns a concrete barrier into an operable flow without promising chargeback success or attributing financial results to a code example.

A hiring discussion can explore how I would negotiate minimum scope with operations: the first gain may be a receipt and routing before automated submission. As Staff, I would document that boundary to reduce risk; as EM, I would ensure the team is not held accountable for a network outcome it does not control. The commitment would cover intact receipt, review within the applicable window and correct information, measured separately from financial victory. This helps prioritize a useful first delivery without disguising external dependencies as implementation details.

CASE 06 · EXTERNAL PROPOSAL

Handoff and decision

Local execution

python3 labs/case_06.py -v · 14 tests passed.

A reviewer submits synthetic bytes, repeats the upload and attempts to reuse its identity with different content. Then observe quarantine, review and submission as separate states. A late document remains recorded but does not automatically cross the cutoff.

Production handoff requires protected storage, a real scanner, format validation, retention policy and a card-network adapter. Acceptance covers holder-scoped access, historical integrity and messages that distinguish provisional credit from final resolution. If additional triage costs exceed avoided rework, the next step is better evidence guidance before expanding intake.

- Upload receipt does not promise external acceptance.
- A new attachment never erases the earlier one.
- Quarantined documents are not submitted.

Executable case lab on GitHub

CASE 07 · EXTERNAL PROPOSAL

The record and its outcome

07

Actual crop of the public source, captured 23 October 2026. The text below includes the company reply and reported outcome.

Answered · RA 228360515

On October 2, 2025, a customer reported three Pix transfers totaling BRL 478.55 for CCXP 25 tickets and said she notified Nubank within two hours of recognizing a scam. She requested MED updates. Nubank reported starting the process and described seven calendar days plus four for conditional reimbursement. No individual outcome or recovered amount was published. This is a historical 2025 response, not a complete description of current rules.

The public response associates absent email with an unaccepted claim or unavailable funds but does not identify which outcome occurred in this case. A customer needs an explicit outcome when evidence exists and ongoing tracking when it does not. Police reports, receipts and private chat were not inspected. The quoted amount is alleged customer exposure, not independently verified final loss or potential bank revenue.

Reclame AQUI - public complaint and response

CASE 07 · EXTERNAL PROPOSAL

Diagnosis before design

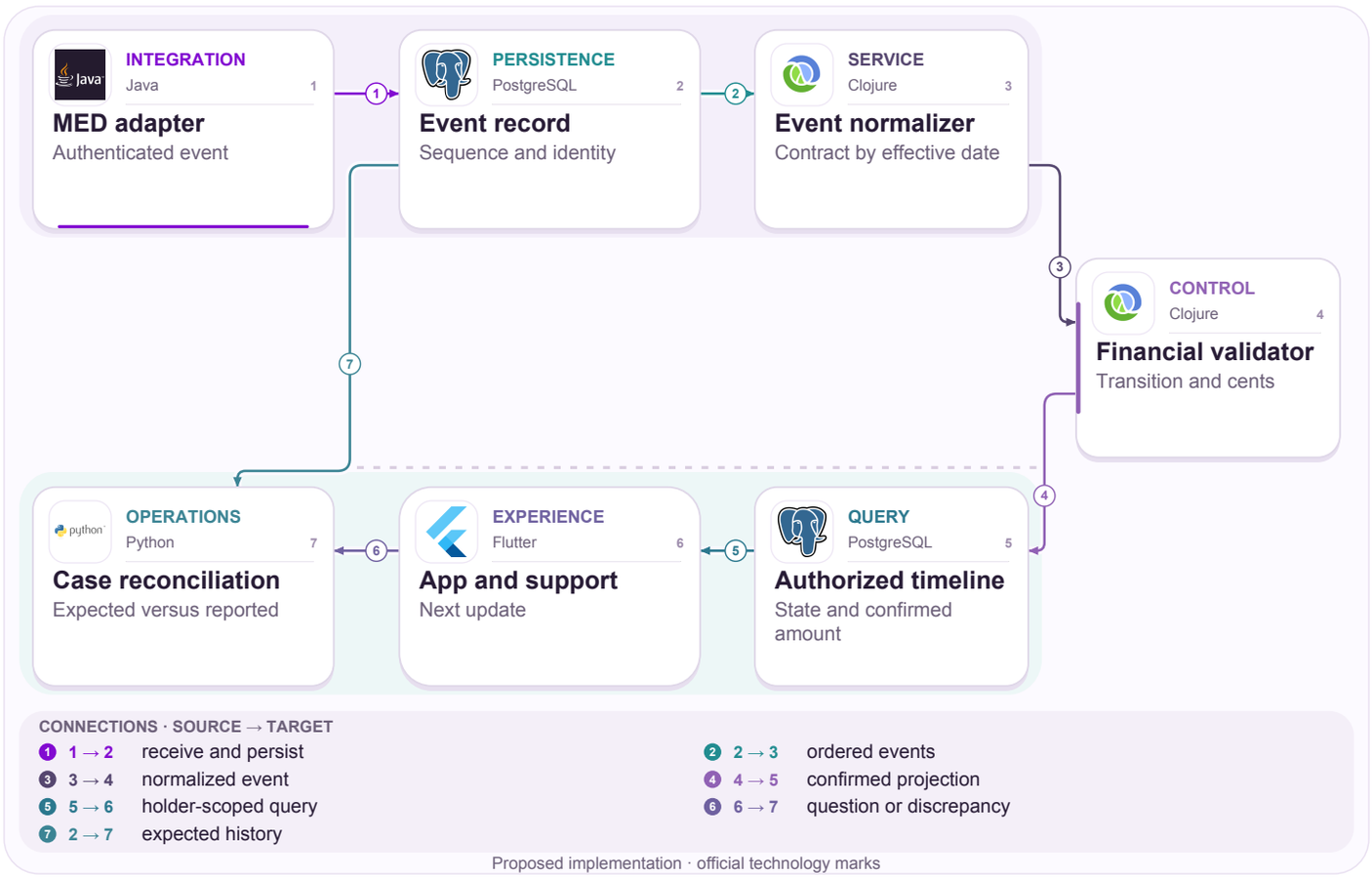
Diagnosis separates interbank progress from message delivery to the customer. Missing email does not reveal whether a case is under analysis, rejected, unfunded or affected by a communication failure. The engineering hypothesis is that explicit states and a next update reduce uncertainty-driven contacts.

I would reconstruct received events, available projection and emitted notifications per case without inferring that delay caused the receiving account to be emptied. The source does not demonstrate that causality. The episode occurred in 2025; a current implementation must map 2026 contracts and rules with Pix, risk and legal rather than judging historical conduct against later requirements.

The timeline should explain partial recovery without turning the remainder into a debt owed by the bank. It must also distinguish operational closure from definitive impossibility of any future recovery under the applicable contract. I would not fill event gaps with a probable outcome merely to simplify the screen. When information is unconfirmed, the next action is investigation and communication of the limitation, preserving the ability to update state when authorized evidence arrives. This makes uncertainty visible without inventing either a positive or negative financial promise.

CASE 07 · EXTERNAL PROPOSAL

Architecture boundaries



Independent proposed architecture; suggested technologies, not Nubank internal infrastructure.

The adapter verifies origin before persistence. The normalizer translates the effective version into an internal contract; the validator protects transitions and amounts before publishing the timeline. App and support share that authorized view.

Reconciliation compares expected history with the state explained to the customer. The lab's sequence is internal to its adapter and does not claim to reproduce official DICT fields. A gap blocks progress and requires recovery rather than inventing an outcome.

CASE 07 · EXTERNAL PROPOSAL

Executable core

PYTHON / ORIGINAL PROPOSAL

```python
    def apply(self, event_id, sequence, state, cents=0, trusted=False):
        if trusted is not True:
            raise PermissionError("verified MED adapter required")
        if type(sequence) is not int or sequence <= 0 or not event_id:
            raise ValueError("event identity")
        if type(cents) is not int or cents < 0:
            raise ValueError("integer cents required")
        payload = (sequence, state, cents)
        with self.lock:
            if event_id in self.events:
                if self.events[event_id] != payload:
                    raise ValueError("event conflict")
                return False
            if sequence != self.sequence + 1:
                raise ValueError("sequence gap or obsolete event")
            allowed = {"reported": {"analyzing"},
                       "analyzing": {"partial", "closed"},
                       "partial": {"partial", "closed"}}
            if state not in allowed.get(self.state, set()):
                raise ValueError("state transition")
            if state != "partial" and cents != 0:
                raise ValueError("refund must be explicit")
            if state == "partial" and not 0 < cents <= self.claimed - self.recovered:
                raise ValueError("invalid recovery amount")
            self.recovered += cents
            self.sequence, self.state = sequence, state
            self.events[event_id] = payload
            return True
```

Literal excerpt from the executed independent lab; remaining validation is in the complete file.

RecoveryTimeline.apply preserves each event’s identity and payload. It accepts only the next sequence; delayed events or gaps require reconciliation and controlled retry. A partial return must be positive and cannot exceed the remaining claimed amount. Closing a case adds no money.

The function checks authority before recognizing replay and rejects booleans as monetary values. The model neither blocks accounts nor calls participants and does not implement every regulatory state. It demonstrates a communication projection that conserves cents. Reopening or new rules require an explicit additional contract rather than an improvised transition.

Python - unittest

Executable case lab on GitHub

CASE 07 · EXTERNAL PROPOSAL

Stack implementation

CLOJURE / ORIGINAL PROPOSAL

```clojure
(defn recovery-event [current event]
  ;; Proposed local rule, not a DICT payload specification.
  (let [amount (:cents event)
        remaining (- (:claimed current) (:recovered current))]
    (cond
      (not (true? (:verified event))) {:error :untrusted}
      (not (integer? amount)) {:error :invalid-money}
      (not (<= 1 amount remaining)) {:error :amount-out-of-range}
      (not= (:sequence event) (inc (:sequence current)))
      {:error :sequence-gap}
      :else {:next-recovered (+ (:recovered current) amount)
             :next-sequence (:sequence event)})))
```

Proposed integration example, not executed; this is not Nubank internal code.

The proposed Clojure validates a financial event without making an external call. Its result describes a possible update; persistence must compare versions and atomically store the event and projection. A Kafka consumer, if chosen, must recognize replays and have a gap-recovery strategy.

The real adapter requires authentication and mapping to the effective official contract, reviewed with the Pix domain. Synthetic names are not presented as production fields. In Flutter, the timeline would separate confirmed amount, analysis state and next update. I would not use a progress percentage that suggests recovery probability without a validated, explainable model.

Banco Central - DICT API, integration reference

CASE 07 · EXTERNAL PROPOSAL

Contracts and ownership

Field / rule	Owner	Invariant
contract_version	Pix + legal	Explicit effective date
event_id + sequence	Adapter	Gap blocks progress
recovered_cents	Financial domain	Never exceeds claim
next_update	Operations	No refund promise

JSON / ORIGINAL PROPOSAL

```json
{
  "schema_version": 1,
  "contract_version": "synthetic-adapter-v1",
  "case_id": "synthetic-med",
  "event_id": "synthetic-partial-return",
  "sequence": 2,
  "state": "partial",
  "confirmed_cents": 20000,
  "claimed_cents": 47855
}
```

Illustrative proposed contract; synthetic values, not customer data.

The contract declares schema version and rule effective date alongside identity, sequence and confirmed amount. A case identifier grants no access; queries are bound to the holder and support's operational scope. Events carry minimal references rather than complete recipient-account data.

For partial returns, the total must reconcile with the financial source and previous events. Duplicate messages do not increase recovered money; an amount correction needs explicit semantics and approval rather than a negative number sent casually. Official deadlines and states require validation before integration; the lab is independent of that certification.

CASE 07 · EXTERNAL PROPOSAL

Adversarial verification

PYTHON / ORIGINAL PROPOSAL

```python
    def test_partial_refund_replay_never_doubles_money(self):
        self.event("analysis", 1, "analyzing")
        self.event("refund-a", 2, "partial", 20000)
        self.assertFalse(self.event("refund-a", 2, "partial", 20000))
        self.event("refund-b", 3, "partial", 27855)
        self.assertEqual(self.t.read("a")["recovered_cents"], 47855)

    def test_gap_retries_after_missing_event(self):
        with self.assertRaises(ValueError):
            self.event("refund", 2, "partial", 100)
        self.event("analysis", 1, "analyzing")
        self.assertTrue(self.event("refund", 2, "partial", 100))
```

Literal lab tests; count and result come from an actual run.

The 14 tests cover replayed partial returns, totals above the claim, sequence gaps, closure without recovery, attempted reopening, event conflict and wrong scope. The gap test shows that rejection does not consume the event: after its predecessor arrives, the same event can be accepted.

The lab also prevents silence from closing a case. It does not test actual recipient funds, scheme communication or regulatory deadlines. Integration would add replay of anonymized messages, comparison with the financial source and adapter-outage drills. The relevant guarantee is never communicating recovered money that has not been confirmed.

Python - unittest

CASE 07 · EXTERNAL PROPOSAL

Measure the right journey

Metric	Numerator / denominator	Window and segment
Update p 95	Event→query time	Daily; stage
Missing next update	Cases without date / eligible	Daily; owner
Progress uncertainty	Contacts / active cases	7 days; cohort
Amount mismatch	Mismatches / audited cases	Continuous; version
Observed recovery	Returned / eligible amount	Closure; profile

The main indicator measures delay between a confirmed event and queryable state, segmented by stage and contract version. I also track cases without a next update, uncertainty-driven contacts and displayed-versus-confirmed amount discrepancies. These signals differ from the financial recovery rate.

Evaluation uses comparable case cohorts, accounting for reporting time, profile and recoverability. A clearer timeline may reduce contacts without recovering one additional cent; that can still be useful but must be described accurately. Expansion stops for false promises, data exposure or mismatched totals. Baselines are not public and must be collected before setting targets.

CASE 07 · EXTERNAL PROPOSAL

Financial value with assumptions

Assumed scenario	Main input	Net/month	Payback (months)
Conservative	3,000 × 0.3 × BRL 9.00	BRL 3,100.00	51.61
Base	8,000 × 0.6 × BRL 9.00	BRL 33,200.00	4.82
Ambitious	15,000 × 0.8 × BRL 9.00	BRL 90,000.00	1.78

The financial scenario measures case handling without counting BRL 478.55 as a benefit. Assumptions are 3,000 / 8,000 / 15,000 monthly active cases, 0.3 / 0.6 / 0.8 contacts avoided per case and BRL 9.00 marginal cost. Assumed additional operations cost BRL 5,000.00 / BRL 10,000.00 / BRL 18,000.00 monthly, with hypothetical implementation of BRL 160,000.00.

The calculation is cases × contacts avoided × cost, less operations. Additional customer recovery is a separate measure with no assumed value in this calculation. If communication does not change the recovery mechanism, I do not attribute increased returns to it. This scenario is not added to general support or Pix scenarios because the contacts may belong to the same journeys.

Assumed implementation budget: BRL 160,000.00. Annual net results before implementation: BRL 37,200.00 / BRL 398,400.00 / BRL 1,080,000.00. Every input is an assumption, not a Nubank baseline.

CASE 07 · EXTERNAL PROPOSAL

Accountable roadmap

Proposed phase	Accountable role	Delivery / exit gate
Days 0-15	Pix + risk + legal	Effective contract
Days 16-30	Staff + operations	Semantics and reconciliation
Days 31-60	Mobile + support	Shadow timeline
Days 61-90	EM + product	Pilot; measured clarity

Discovery brings Pix, risk and legal together to map the effective contract and communication policy. The initial milestone is an event table with authority, interpretation and information permitted to the holder. Staff resolves semantics and reconciliation; product determines how to explain incomplete states without promising outcomes.

The projection runs in shadow before serving queries. Operations compares samples of partial returns and closed cases; support validates usefulness. The EM coordinates capacity and regulatory dependencies. If the adapter cannot guarantee origin or completeness, the pilot remains limited to approved general guidance and better observability rather than a speculative individual timeline.

CASE 07 · EXTERNAL PROPOSAL

Rollout and operations

The canary begins with reads and communication without changing interbank actions. An operational owner handles gaps; another confirms financial discrepancies. A next update is a planned information commitment, not a refund guarantee. If it cannot be met, the case needs a revised update time and escalation rather than silence.

The drill injects duplicated partial returns, out-of-order events, an unavailable adapter and closure with zero recovered funds. Rollback preserves history and lets the previous query path explain known states. Coexisting contract versions must be identifiable; mixing historical and current rules without effective dates creates interpretation and operating risk.

CASE 07 · EXTERNAL PROPOSAL

My contribution

As Staff, I would propose a protocol in which state and amount have supporting evidence and evolve by effective date. I would reject inferring rejection from email silence because message delivery and process decisions are different domains. I would also avoid an unsupported recovery percentage that could resemble a financial promise.

As EM, I would align Pix, risk, legal, support and mobile around a measurable goal of clarity and less rework. The priority would be making each stage explainable and operable rather than demonstrating sophisticated architecture. These are proposed contributions to the role, not claims of internal experience or gains already achieved at Nubank.

In team collaboration, I would separate communication goals from recovery goals to avoid misleading attribution of credit. The EM can encourage timely updates without demanding returns that depend on funds and other participants. Staff can help establish evidence of completeness and consistency. This alignment lets engineering demonstrate real value even when an individual financial outcome is limited by external factors. It also supports a more honest investment decision: better clarity may justify its operating cost without being presented as proof that the underlying recovery mechanism became more effective.

CASE 07 · EXTERNAL PROPOSAL

Handoff and decision

Local execution

python3 labs/case_07.py -v · 14 tests passed.

The reviewer starts analysis, applies two synthetic partial returns and replays one. The total must remain 47,855 cents. Next attempt to exceed the claim and close a case without a return; the model clearly separates completion from recovered money.

Handoff depends on a validated official contract, event authentication, persistence, reconciliation and communication policy. No local test replaces scheme certification. Proceeding requires financial coherence and measurably less uncertainty. If the project merely produces more notifications without comprehension or fewer repeat contacts, revise the experience before increasing volume.

- A duplicated return is never counted twice.
- Email silence does not close the case.
- Historical and current contracts stay separate.

Executable case lab on GitHub

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

08 · The dispute and the response

08

Actual crop of the public source, captured 23 October 2026. The text below includes the company reply and reported outcome.

On 14 January 2026, complaint 237674561 described an unrecognized loan discovered after three overdue installments. Nubank replied that the loan had been contracted in March 2024 using an authorized device and password, with no irregularity identified. It maintained its refusal. The customer closed the case on 31 January as Not resolved. These are public accounts from parties in disagreement; they do not establish fraud or consent.

What this evidence supports

An opportunity to make investigation understandable and reviewable. The loan amount, individual agreement and complete signals are not public. The capture documents the complaint and response; it is not a forensic finding.

Reclame AQUI - case 08 · 2026-01-14

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Observed access does not settle consent

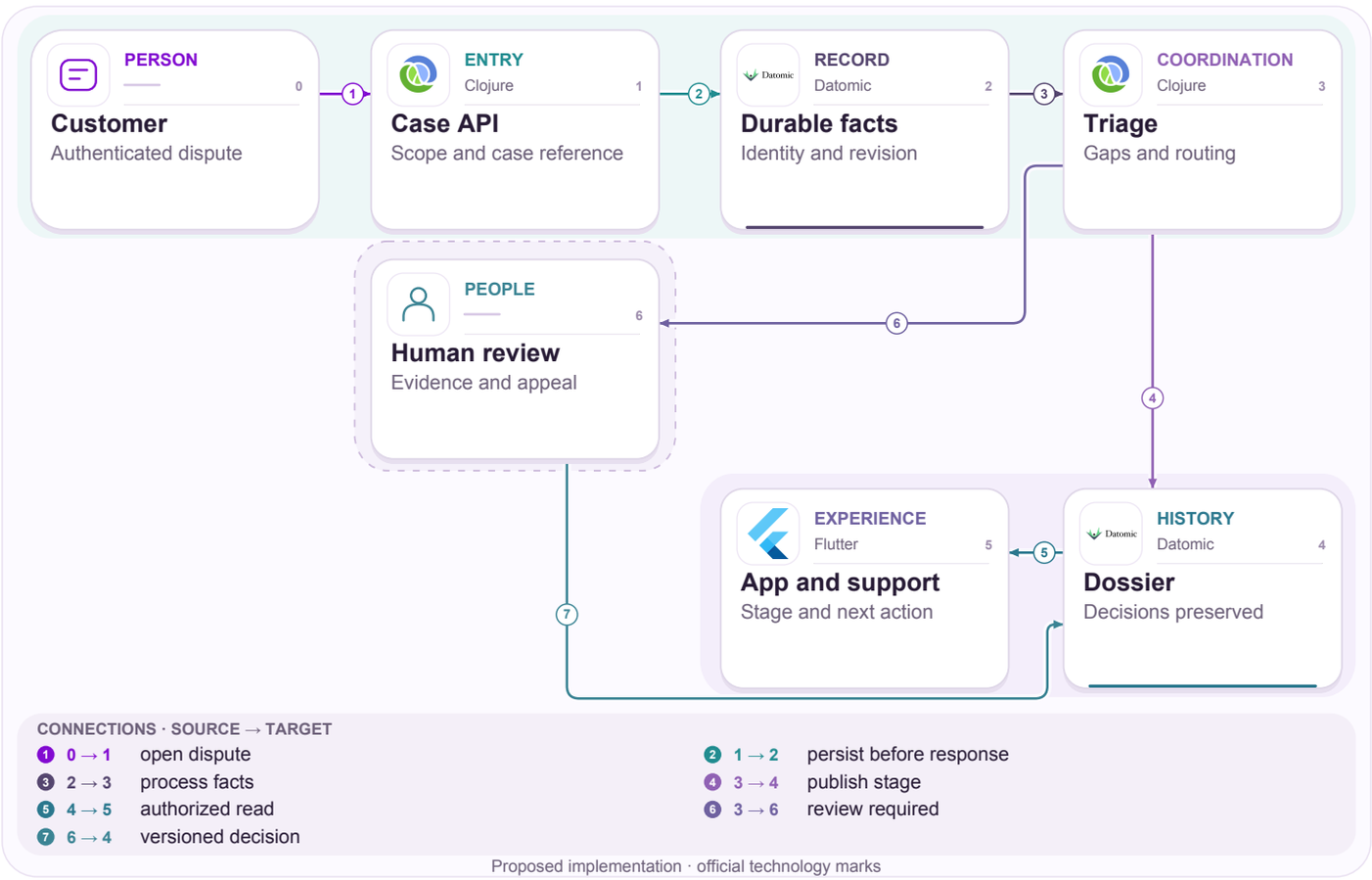
The engineering question starts at the boundary between evidence and conclusion. A record of an authorized device may matter to an investigation, but it should not be stored as a synonym for the account holder's intent. This proposal separates access facts, the policy version applied, the reviewer's interpretation and the communicated decision. A customer can then challenge a specific premise without restarting the entire conversation.

The falsifiable hypothesis is that some rework comes from incomplete dossiers or decisions whose reviewable reasoning is unclear. I would compare an authorized sample of cases with repeat contacts against what the agent could see at that time. If dossiers were already complete and understandable, the hypothesis weakens; the intervention may belong in policy, communication or another stage.

The study does not propose publishing fraud-control secrets or refunding every disputed loan. Financial decisions remain subject to approved authority and policy. The objective is consistent analysis with an owner, minimized evidence and a defined appeal. Legal and operational complexity requires specialist validation before setting deadlines or interim measures.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

A dossier with independent review



Proposed external architecture; not Nubank's internal implementation.

The API authenticates the account holder and scopes access to the case. The durable record preserves actions before acknowledgment; triage organizes gaps without deciding fraud. The dossier supplies different customer and support projections with a shared revision reference.

Human review records the decision and policy in history. An appeal adds a stage rather than erasing the first decision. The exception path requires an independent reviewer in the model. Real identity, authorization and evidence retention are necessary integrations outside the laboratory.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Core: revision and action identity

PYTHON / ORIGINAL PROPOSAL

```python
    def append(self, action_id, expected, actor, role, kind, evidence):
        if not isinstance(action_id, str) or not action_id.strip():
            raise ValueError('nonempty action identity required')
        event = dict(actor=actor, role=role, kind=kind, evidence=evidence)
        payload = json.dumps(event, sort_keys=True)
        with self.db:
            self.db.execute('BEGIN IMMEDIATE')
            old = self.db.execute('SELECT payload FROM events WHERE action_id=?',
                                  (action_id,)).fetchone()
            if old:
                if old[0] != payload:
                    raise ValueError('idempotency conflict')
                return self.history()
            history = self.history()
            if expected != len(history):
                raise ValueError('stale revision')
            allowed = {'evidence': 'analyst', 'decision': 'reviewer',
                       'appeal': 'customer', 'review': 'reviewer'}
            if kind not in allowed or allowed[kind] != role:
                raise PermissionError('role not authorized')
            if kind == 'decision' and not any(
                    e['kind'] == 'evidence' for e in history):
                raise ValueError('no evidence dossier')
            validate_transition(history, event)
            self.db.execute('INSERT INTO events VALUES(?,?,?)',
                            (expected + 1, action_id, payload))
        return self.history()
```

Literal excerpt from labs/case_08.py; exercised by local tests.

The transaction checks expected revision, action identity and authorized role. An identical retry returns history without duplicating the event; reusing the key with different content is a conflict. This contract prevents two concurrent decisions from advancing the same revision.

The laboratory represents one case and receives roles from trusted context. It neither authenticates people nor determines whether the loan is legitimate. validate_transition requires evidence before a decision and protects appeal independence. The local database demonstrates persistence; it does not represent the institution’s actual storage.

Executable case lab on GitHub

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Make the policy explicit

CLOJURE / ORIGINAL PROPOSAL

```clojure
(defn reviewable? [{:keys [evidence decision policy]}]
  (and (seq evidence)
       (string? policy)
       (every? #(contains? % :source-ref) evidence)
       (not= :device-access-only decision)))

(defn appeal-command [case-id revision reviewer]
  {:case-id case-id
   :expected-revision revision
   :action :independent-review
   :reviewer reviewer})
```

Illustrative Clojure example, not executed; integration and policy require validation.

A pure function can check structural completeness before routing the dossier. It cannot turn the presence of data into a correct verdict. The example rejects a simplified decision category based only on device access; this is a proposed rule for the study, not a known Nubank rule.

In the service, the reviewer identifier must come from authenticated identity rather than the customer's request body. Policy has an immutable version, effective period and owner. Policy changes need approved examples and operational review, because a technically correct deployment can still produce inappropriate decisions. Kafka would carry minimized references; sensitive attachments would remain in restricted storage.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Evidence and access contracts

Object	Owner	Rule
Evidence reference	Security	Hash and source; no password
Decision	Authorized reviewer	Policy and revision required
Appeal	Operations	Preserves earlier decision
Customer projection	Product	Useful reason, no secrets

JSON / ORIGINAL PROPOSAL

```json
{
  "case_id": "synthetic-08",
  "revision": 3,
  "evidence": [{"category": "device_access", "digest": "sha256:..."}],
  "decision": {"policy": "v1", "reviewer_ref": "reviewer-1"},
  "appeal": {"status": "pending", "independent_reviewer": true},
  "customer_projection": {"stage": "under_review"}
}
```

Synthetic illustrative contract; not a payload captured from the app.

A hash helps identify evidence but does not prove its truth or replace chain of custody. Retention policy determines how long the object can be consulted; the projection should not retain unnecessary copies. Administrative access needs a recorded purpose and must be auditable by an independent team.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Test appeal and recovery

PYTHON / ORIGINAL PROPOSAL

```python
    def test_appeal_requires_independent_reviewer(self):
        self.seed()
        self.d.append('a', 2, 'owner', 'customer', 'appeal', {})
        with self.assertRaises(PermissionError):
            self.d.append('r', 3, 'r1', 'reviewer', 'review', {'policy': 'v2'})
        self.assertEqual(len(self.d.history()), 3)
        self.d.append('r', 3, 'r2', 'reviewer', 'review', {'policy': 'v2'})
        self.assertEqual(self.d.history()[1]['evidence']['policy'], 'v1')

    def test_restart_and_retry_preserve_one_evidence(self):
        self.d.append('e', 0, 'a', 'analyst', 'evidence', self.ev)
        self.d.close()
        self.d = Dispute(self.path)
        result = self.d.append('e', 0, 'a', 'analyst', 'evidence', self.ev)
        self.assertEqual(len(result), 1)
        self.assertNotIn('consent', result[0]['evidence'])
```

Literal excerpts; the complete local suite passed 9 tests.

The first assertion protects appeal independence and verifies that rejection does not change history. The second closes and reopens the database, retries an action and confirms a single record. Other tests attack stale revisions, incorrect roles, missing evidence and sensitive payloads.

Tests do not assess human quality, evidence authenticity or policy fairness. Production work would add blinded sample review, cross-account access tests, permission expiry, export auditing and tool-outage exercises. A system that perfectly preserves a poor decision still needs process correction.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Measure useful review and human effort

Metric	Definition	Window / segment
Useful review	Cases with reasoned action / opened	Weekly; dispute type
Rework	Repeated minutes / eligible dossier	Monthly sample; team
Repeat contact	Repeated contacts / cases	30 days; stage
Disagreement	Materially corrected decisions / audited	Monthly; policy

The baseline is unknown. I would instrument transitions with pseudonymized case_id, revision and timestamp; work time requires operational sampling rather than queue duration alone. Case age and actual effort are different measures. Unauthorized access and audit failures are additional guardrails.

The pilot would compare equivalent teams or periods using predefined eligibility, avoiding selection of easy cases only. Independent reviewers would assess samples from both groups. Expansion would stop for data exposure, lost evidence or a material deterioration in decision consistency, even if average handling time improved.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Value: recovered capacity, not profit

Assumed scenario	Potential/year	Operations/year	Year 1 after cost
Conservative	R$ 144,000	R$ 60,000	R$ -156,000
Base	R$ 504,000	R$ 90,000	R$ 174,000
Ambitious	R$ 1,080,000	R$ 120,000	R$ 720,000

Every input is assumed: 12,000, 24,000 or 40,000 annual dossiers; 8, 14 or 18 rework minutes saved; R$ 90 fully loaded hourly cost. Valued capacity is N × minutes ÷ 60 × 90. Implementation is assumed at R$ 240,000 and annual operations at R$ 60,000, 90,000 or 120,000. The final column subtracts both in year 1 and does not represent realized cash.

In the base case, 24,000 × 14 ÷ 60 × 90 = R$ 504,000; subtracting R$ 90,000 and R$ 240,000 leaves R$ 174,000 in modeled net capacity. Year-one break-even requires approximately 15,715 equivalent dossiers. Redistributing hours does not automatically save cash. Investment depends on confirming avoidable rework and identifying a concrete use for the released capacity.

Keep financial categories separate

Disputed principal and possible fraud are excluded. Loss estimates require verified exposure, confirmation and recovery. Do not aggregate benefits with other chapters that cover the same contacts.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

A 90-day plan with accountable roles

Proposed phase	Deliverable and owner	Exit gate
Days 0-15	Risk + operations: taxonomy and sample	Facts separated from inference
Days 16-30	Staff + privacy: contract and access	Access review approved
Days 31-60	EM + operations: shadow and training	Independent sample consistent
Days 61-90	Product + risk: limited pilot	Guardrails and capacity validated

The calendar is a planning assumption. The critical dependency is authorized access to a sufficient sample, not building a screen. If evidence sources are incompatible, the first deliverable becomes identifying those gaps and assigning owners to resolve them.

The Engineering Manager coordinates capacity across lending, fraud, support and privacy; the Staff Engineer defines contracts and technical criteria. Policy review remains with its owners. Every phase ends with observable evidence and a recorded decision to continue, narrow or stop. The schedule does not promise changes dependent on external validation.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Operate without abandoning the queue

Deployment starts in shadow mode: the new tool prepares dossiers while the existing operation continues making decisions. Completeness, preparation time and discrepancies are compared without automatically changing collections or credit. The next canary exposes tracking and review to a limited cohort, with human capacity reserved for appeals.

Alerts distinguish aging queues, unavailable evidence and authorization failure. Operations owns queue response; security owns exposure; the service team owns lost or duplicated events. A projection outage neither closes the case nor deletes its trail. Customers receive an honest stage and next action.

Rollback disables the new projection and automatic routing while preserving history and active cases. An exercise should simulate database failure after a decision, permission revocation during access and return of an old revision. The runbook must explain pending-work reconciliation and continuity communication without exporting dossiers to unprotected spreadsheets.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

My contribution to the decision

As Staff, I would define the evidence model, authorization boundaries and review invariants. I would lead the consistency, retention and signal-exposure discussion using concrete disagreements. The most important decision would be to keep access records separate from interpretation of consent, preventing a technical field from carrying a conclusion it cannot support.

As Engineering Manager, I would establish joint delivery with operations and risk, an end-to-end owner and reserved review time. I would first prioritize the most frequent demonstrated rework, with explicit criteria for choosing product changes, integration or training as the next investment. These are proposed actions, not claims that I have managed Nubank teams or policies.

I would reject automatic refusal merely because the dossier contains a known device. I would also avoid exposing every signal to support. The proposal balances useful review with minimized access and accepts higher operational effort when needed to protect the quality of a sensitive decision.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Handoff for technical and operational review

- Run: python3 labs/case_08.py. The complete suite contains 9 tests and accesses no external service.
- Review Dispute.append and validate_transition, then inspect history before and after an appeal.
- Confirm that retry, key conflict, stale revision and incorrect role cannot create another decision.
- Before production: identity, account scope, retention, policy, audit and independent human review.

The laboratory supports a joint review: engineering explains the contract; operations checks whether history reconstructs the work; risk evaluates decisions needing supervision; privacy evaluates exposure. Local execution validates only the modeled invariants, not the merits of a dispute.

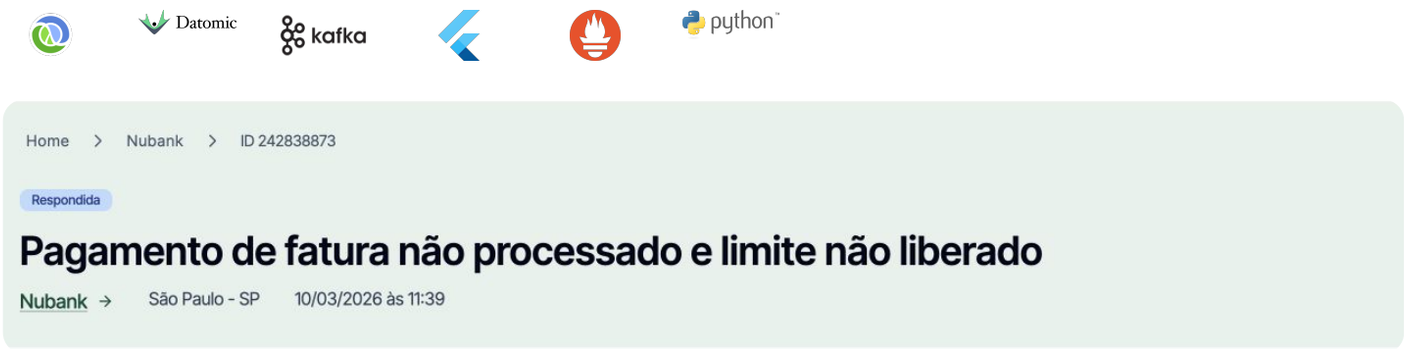
The final gate is straightforward: proceed only if the sample shows a correctable gap, independent review maintains quality and released capacity has a destination. Otherwise retain the learning and choose another intervention. The intended result is a better-grounded decision that is easier to review, without promising automatic resolution of financial disputes.

Executable case lab on GitHub

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

09 · Paid does not explain allocation

09



Actual crop of the public source, captured 23 October 2026. The text below includes the company reply and reported outcome.

Complaint 242838873, dated 10 March 2026, describes a statement marked paid while credit was not fully restored, after three support tickets. On 17 March, Nubank reported correcting the statement, recognizing the payment and settling advanced installments. It explained credit restoration through processing and financed balances. The public status is Responded; no final customer confirmation is visible. Individual amounts are masked.

Evidence boundary

The response reports a correction but does not expose the ledger, event order or technical cause. This study proposes allocation traceability rather than diagnosing the responsible system.

Reclame AQUI - case 09 · 2026-03-10

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Three states, different consequences

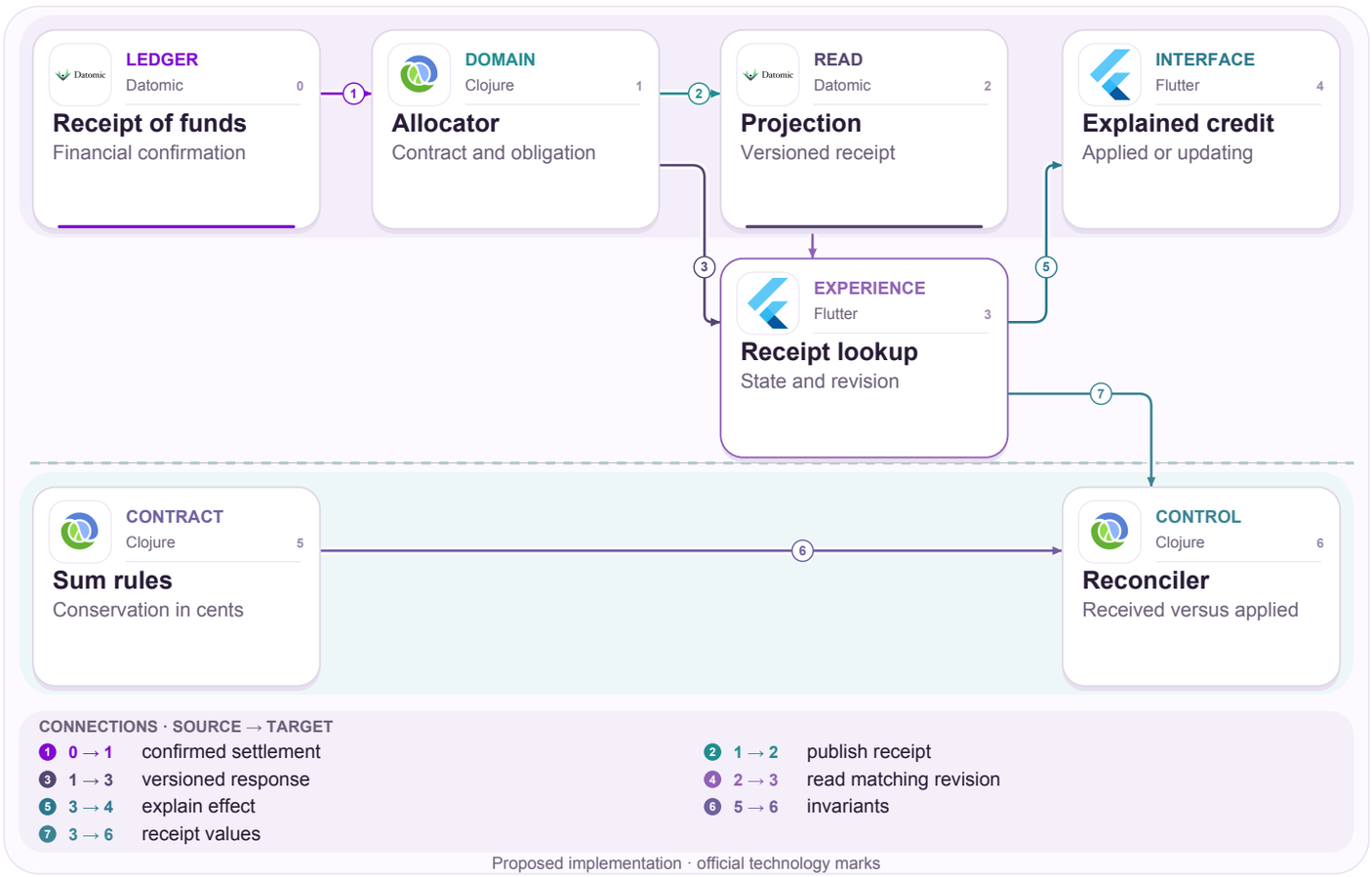
Receiving money, applying money and refreshing a credit-limit view are separate stages. An interface can correctly report a payment while leaving unanswered which obligations were settled. With financing, the effect on available credit depends on rules that a single label cannot explain. The receipt must connect receipt of funds, destination and consequence rather than merely replace a word on screen.

My hypothesis is that some repeat contact comes from the absence of this verifiable link across channels. Investigation would use the payment identifier to reconstruct confirmation, allocations, compensating entries and projection updates. I would compare what the customer saw against support and ledger records at the same revision. If obligation calculation is wrong, improved presentation is insufficient and financial correction takes priority.

The responsibility boundary includes contractual financing rules and settlement recognition. The app cannot independently choose where to move money. It should also avoid inviting a second payment while the first allocation is uncertain. The proposal makes the pending state explicit and provides tracking under the same identifier.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

A receipt linked to the ledger



Proposed external architecture; not Nubank's internal implementation.

The allocator receives financial confirmation and applies a validated plan. The canonical receipt and its projection share a revision. The reconciler compares receipt values against independent invariants; a discrepancy is not solved by hiding cents in presentation.

Available-credit updates may run at a different pace, but must expose their revision and state. The diagram summarizes responsibilities, not every integration. Financial correction requires authorized compensating entries in the ledger; the reconciler detects and routes discrepancies without independently editing balances.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Core: allocate or change nothing

PYTHON / ORIGINAL PROPOSAL

```python
    def apply(self, ident, amount, plan, crash=False):
        if not isinstance(ident, str) or not ident.strip():
            raise ValueError('nonempty payment identity required')
        if type(amount) is not int or amount <= 0:
            raise ValueError('positive integer cents required')
        if any(type(v) is not int or v <= 0 for v in plan.values()):
            raise ValueError('invalid allocation')
        encoded = json.dumps(plan, sort_keys=True)
        with self.db:
            self.db.execute('BEGIN IMMEDIATE')
            old = self.db.execute('SELECT amount,plan FROM payments WHERE id=?',
                                  (ident,)).fetchone()
            if old:
                if old != (amount, encoded):
                    raise ValueError('payment conflict')
                return self.receipt(ident)
            if sum(plan.values()) > amount:
                raise ValueError('allocation exceeds received amount')
            for target, value in plan.items():
                changed = self.db.execute('UPDATE obligations SET due=due-? '
                    'WHERE id=? AND due>=?', (value, target, value)).rowcount
                if changed != 1:
                    raise ValueError('unknown or overpaid obligation')
                self.db.execute('INSERT INTO allocations VALUES(?,?,?)',
                                (ident, target, value))
            if crash:
                raise RuntimeError('crash before commit')
            self.db.execute('INSERT INTO payments VALUES(?,?,?,?)',
                            (ident, amount, encoded, amount - sum(plan.values())))
        return self.receipt(ident)
```

Literal excerpt from labs/case_09.py; SQLite represents one local authority.

The conditional write prevents applying more than an obligation owes. Allocations and receipt share a transaction: if the second destination is invalid, the first returns to its prior state as well. Unapplied funds remain explicit.

The example receives a preselected plan and does not implement Nubank’s contractual priority. Production selection requires versioned policy, account authorization and settlement evidence. A retry is accepted only when amount and plan are identical. Reusing the key to change destination requires a different authorized action, never a silent update.

Executable case lab on GitHub

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Pure rules, transactional writes

CLOJURE / ORIGINAL PROPOSAL

```clojure
(defn allocation-valid? [{:keys [received allocations unapplied]}]
  (and (pos-int? received)
       (every? pos-int? (vals allocations))
       (<= 0 unapplied)
       (= received (+ unapplied (reduce + (vals allocations))))))

(defn allocation-event [payment-id revision plan]
  {:event-type :payment-applied
   :payment-id payment-id
   :revision revision
   :allocations plan})
```

Illustrative Clojure example, not executed; the Python laboratory is the tested artifact.

The function checks conservation but cannot prove that each obligation is eligible. That validation belongs inside the transactional boundary that knows balance, account holder and contract. Separating pure calculation from persistence helps test rules and review changes without attributing database guarantees to the function.

Event integration should persist allocation and publication intent atomically. The projection consumer deduplicates by identifier and financial revision. Transport ordering alone is insufficient: replay, reprocessing and consumer changes can redeliver the same fact. Publication failure leaves identifiable pending work, not permission to apply the payment again.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Conserve money and identity

Field	Meaning	Owner
payment_id	Stable receipt identity	Settlement
allocations	Destination in cents	Credit/statements
unapplied	Funds without a destination	Operations
revision	Source financial fact	Ledger

JSON / ORIGINAL PROPOSAL

```json
{
  "payment_id": "synthetic-payment-09",
  "currency": "BRL",
  "received_cents": 15001,
  "allocations": {"bill": 10000, "installment": 5000},
  "unapplied_cents": 1,
  "ledger_revision": 42,
  "limit_projection_revision": 41,
  "limit_state": "updating"
}
```

Synthetic example: 15,001 received, 15,000 applied and 1 unapplied.

A credit projection behind the financial revision explains that the view is still updating, without asserting that credit is already available. Identifiers must not expose accounts or contracts in public URLs. Support receives what is needed to explain allocation; third-party details and unrelated obligations remain protected. Rectification should reference the earlier fact and preserve the trail.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Partial failure must not partially settle

PYTHON / ORIGINAL PROPOSAL

```python
    def test_failure_after_first_write_rolls_everything_back(self):
        with self.assertRaises(ValueError):
            self.l.apply('p', 20000, {'bill': 10000, 'installment': 6000})
        self.assertEqual(self.l.due('bill'), 10000)
        self.assertEqual(self.l.due('installment'), 5000)
        self.assertEqual(self.l.db.execute(
            'SELECT COUNT(*) FROM allocations').fetchone()[0], 0)

    def test_restart_retry_does_not_apply_twice(self):
        first = self.l.apply('p', 12000, {'bill': 10000})
        self.l.close()
        self.l = Ledger(self.path)
        self.assertEqual(self.l.apply('p', 12000, {'bill': 10000}), first)
        self.assertEqual(first['unapplied'], 2000)
        self.assertEqual(self.l.due('bill'), 0)
```

Literal suite excerpts; 9 tests passed.

The partial-failure test attempts to settle one obligation and exceed another, verifying full rollback. The recovery test reopens the database and confirms that the same receipt cannot apply money twice. The suite also checks a residual cent, conflicting plan, fractional amount and payment against an already settled obligation.

Interest, fees, currency, reversals and cross-account authorization remain outside scope. An integration test should observe outbox and projection after failure between commit and publication, checking the receipt against an independent source. Local conservation is necessary but does not replace settlement-system reconciliation.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Metrics that follow the payment

Metric	Numerator / denominator	Window and segment
Late application	Payments beyond threshold / eligible	Daily; debt type
Unallocated funds	Unapplied cents / received cents	Daily; age
Repeat contact	Allocation contacts / payments	30 days; contract
Discrepancy	Unreconciled receipts / checked	Continuous; revision

Baselines and internal deadlines are unknown. I would first define each timestamp's origin: external confirmation, allocation commit and projection publication. Network delay must not be confused with settlement delay. Contact classification needs sampled review to distinguish a presentation question from a real financial error.

The experiment releases an explainable receipt without changing allocation rules. It compares equivalent cohorts by product, financing and amount. Expansion stops for any conservation violation, increased repeated payments or cross-account context leakage. Reducing contact must never incentivize hiding access to help.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

A financial case with two cost components

Assumed scenario	Potential/year	Operations/year	Year 1 after cost
Conservative	R$ 70,800	R$ 60,000	R$ -249,200
Base	R$ 240,000	R$ 60,000	R$ -80,000
Ambitious	R$ 676,800	R$ 60,000	R$ 356,800

Every input is hypothetical: 60,000, 120,000 or 240,000 eligible payments/year; 0.06, 0.10 or 0.14 fewer contacts per payment; R$ 18 per contact. I additionally assume 1, 2 or 3 reconciliation hours saved per thousand payments at R$ 100/hour. Implementation: R$ 260,000; annual operations: R$ 60,000. Contacts and reconciliation hours must represent different work.

The base case is 120,000 × 0.10 × 18 + 120 × 2 × 100 = R$ 240,000 in annual capacity. Year one is R$ 240,000 - 60,000 - 260,000 = -R$ 80,000. Break-even requires 160,000 payments equivalent to the base assumption. This supports a pilot or reuse of components rather than automatic approval of the full project.

Do not convert balances into revenue

Restored credit and received principal are not profit. Capacity becomes cash only through concrete spending reductions. Exclude anticipation rework already measured in case 14.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

A reconciliation-led roadmap

Proposed phase	Owner and deliverable	Gate
Days 0-15	Statements + credit: allocation map	Synthetic cases reconcile
Days 16-30	Staff: receipt and outbox	Failures do not duplicate application
Days 31-60	Operations: shadow reconciliation	Differences classified
Days 61-90	EM + product: receipt canary	Contacts fall; integrity holds

The main dependency is a trustworthy financial reference against which to compare the new view. Without it, a dashboard can merely make an error more persuasive. In the first phase, accounting and credit approve examples for partial payments, financing, anticipation and compensation, with amounts in cents and explicit revisions.

The 90-day calendar is a planning scenario. The EM reserves support and operations capacity during the canary; the Staff Engineer defines failure tests and the correction contract. If reference quality fails the gate, the next delivery is reconciliation and remediation rather than visual expansion. Every exception needs an owner and a destination for unapplied funds.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Rollback preserves financial writes

Launch starts with shadow reading. The projection calculates a receipt and compares it with the ledger without guiding customers or changing allocations. A canary then exposes the allocation explanation to eligible accounts. The old view remains an identified fallback, without mixing total and detail revisions.

Alerts combine unapplied-fund age, projection delay and sum violations. Platform owns publication; credit and statements own financial interpretation; operations coordinates customer resolution. An error queue should carry IDs and reasons rather than unnecessary complete financial data in broadly accessible logs.

Disabling the new receipt does not undo confirmed payments. Rollback affects presentation and routing, followed by reconciliation of pending work. Exercises include crashes before and after commit, a delayed consumer and concurrent correction. Throughout recovery, the tool must distinguish received from applied money for customers and support alike.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

My contribution on two fronts

As Staff, I would center the design on conservation and identity. I would choose the transaction boundary, define compensation and document which reads may be eventually consistent. I would reject inferring available credit from a paid-statement label because it replaces a financial contract with a visual approximation.

As Engineering Manager, I would bring together teams that may see different parts of the same payment. I would establish an end-to-end resolution owner and an investigation agreement using shared IDs. The plan must include support training and exception-handling capacity, not only a mobile component.

I would also advocate an economic decision proportional to the evidence. The base scenario does not recover implementation cost in the assumed first year. If an existing receipt API can be extended, the cost changes and must be recalculated. My role is to expose that tradeoff and protect journey integrity without presenting potential capacity as realized savings.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

A receipt review walkthrough

- Run python3 labs/case_09.py: 9 local tests, with no banking integration.
- Read Ledger.apply and receipt; reproduce two-destination allocation with one residual cent.
- Fail the second destination and confirm that the first does not remain settled.
- Before production: real settlement, account scope, contractual rules, outbox and compensating entries.

A reviewer must be able to explain where every cent came from and where it went. A receipt showing only a total does not satisfy the contract. The local demonstration permits direct inspection of payments, obligations and allocations, but does not cover all rules of a real card or financing product.

Proceeding requires a reconciled sample, demonstrated customer understanding and operations able to resolve differences. If conservation breaks, the gate is to stop and correct the financial authority. If writes are correct and only the view is confusing, the explanation pilot can proceed with lower risk and cost.

Executable case lab on GitHub

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

10 · Duplication with confirmed resolution

10

Actual crop of the public source, captured 23 October 2026. The text below includes the company reply and reported outcome.

In complaint 241843991, dated 27 February 2026, the customer described an error message and repeated NuPay payment, followed by two R$ 900 transactions and R$ 81.17 interest. On 4 March, Nubank reported crediting the duplicate and interest; the customer confirmed resolution. The technical cause is not established.

Public figure	Interpretation caution
4 installments of R$ 245.29	Stated sum: R$ 981.16
R$ 900 + R$ 81.17	Analytical sum: R$ 981.17
Difference: R$ 0.01	Do not invent a rounding adjustment

Resolved historical case

The one-cent discrepancy belongs to the account. The exact adjustment entry was not published; the response confirms categories rather than a numeric total.

Reclame AQUI - case 10 · 2026-02-27

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Timeout is not definitive failure

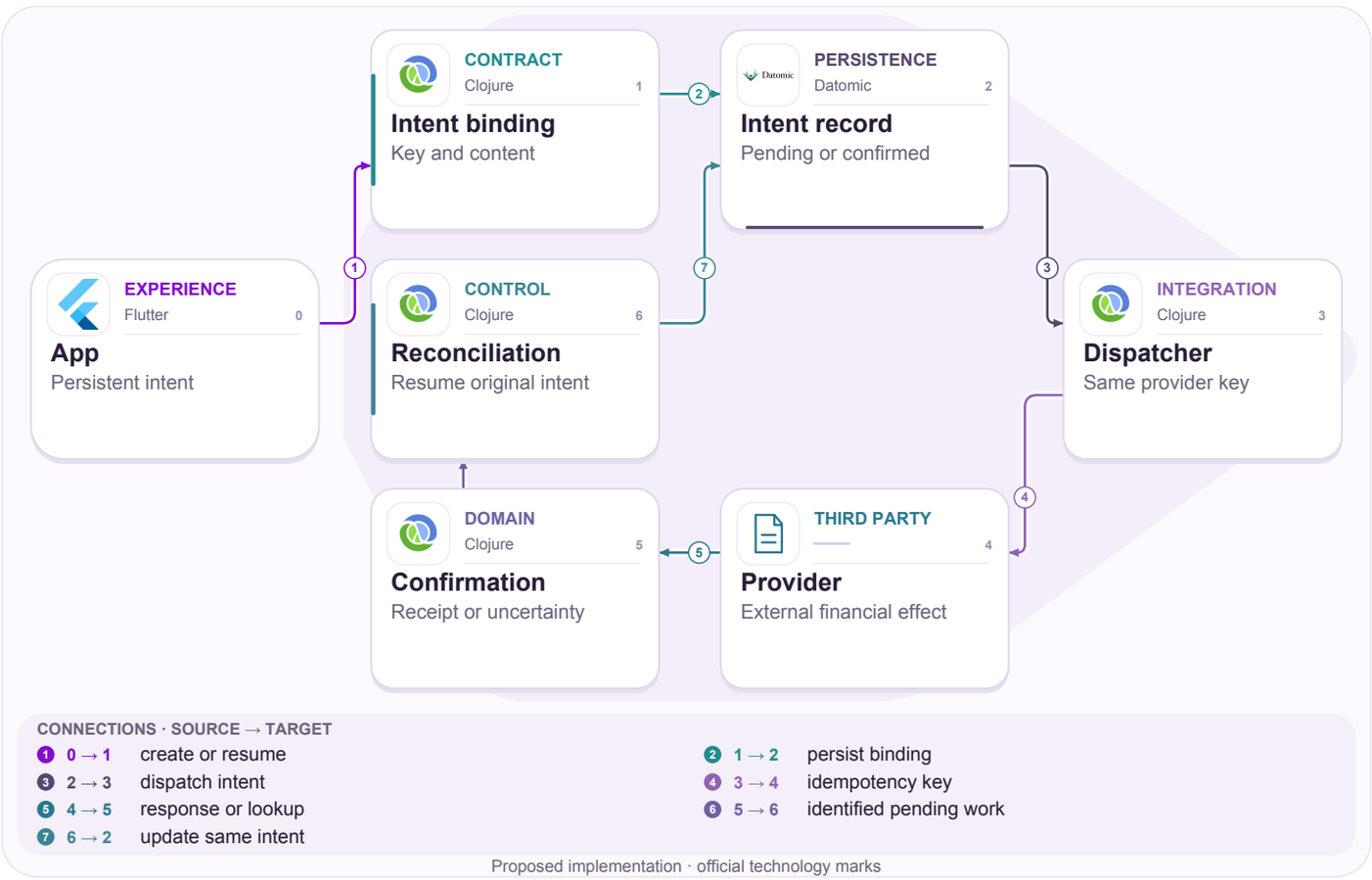
An interruption can occur after a partner accepts payment but before the app receives confirmation. During that interval, saying that nothing happened can induce another financial intent. This chapter hypothesizes that an explicit unknown-outcome state, paired with lookup of the original attempt, reduces unintended repetition without preventing legitimate new purchases.

Investigation must reconstruct identifiers, timestamps and contracts across client, service and provider. Retrying one intent must be distinguished from a deliberate second purchase. Amount, merchant and temporal proximity support analysis but are insufficient identity: two equal purchases may be legitimate. The public source cannot establish which mechanism was present or absent in the incident.

Control should begin before dispatch: create and persist intent, bind content to a key and recover state after app restart. If the provider lacks idempotency or reliable lookup, the guarantee weakens. The experience must then acknowledge uncertainty and initiate reconciliation rather than promise exactly one distributed execution.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

One intent crosses uncertainty



Proposed external architecture; not Nubank's internal implementation.

The record binds key, amount, currency and recipient before dispatch. The provider retains the same key across attempts. Confirmation and reconciliation update the original intent, allowing the app to recover the receipt after losing the response.

The model uses two databases to represent the service/provider boundary. No distributed transaction spans them. The property explicitly depends on provider deduplication; without that capability, the design requires a different operational strategy and cannot retain the same promise.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Core: persist before dispatch

PYTHON / ORIGINAL PROPOSAL

```python
    def pay(self, key, amount, merchant, timeout_after_provider=False):
        if not isinstance(key, str) or not key.strip():
            raise ValueError('nonempty intent identity required')
        if type(amount) is not int or amount <= 0 or not merchant:
            raise ValueError('invalid intent')
        payload = json.dumps([amount, merchant])
        with self.db:
            self.db.execute('BEGIN IMMEDIATE')
            old = self.db.execute('SELECT payload,receipt FROM intents WHERE key=?',
                                  (key,)).fetchone()
            if old and old[0] != payload:
                raise ValueError('intent conflict')
            if old and old[1]:
                return old[1]
            self.db.execute('INSERT OR IGNORE INTO intents VALUES(?,?,NULL)',
                            (key, payload))
        receipt = self.provider.charge(key, payload)
        if timeout_after_provider:
            raise TimeoutError('provider committed; local outcome unknown')
        with self.db:
            self.db.execute('UPDATE intents SET receipt=? WHERE key=?',
                            (receipt, key))
        return receipt

    def state(self, key):
        row = self.db.execute('SELECT receipt FROM intents WHERE key=?',
                              (key,)).fetchone()
        return 'absent' if row is None else ('confirmed' if row[0] else 'unknown')
```

Literal excerpt from labs/case_10.py; two local databases and a synthetic provider.

Intent is recorded before the external call. The injected timeout occurs after provider commit but before the local receipt is saved. Recovery uses the same key and receives the existing receipt, demonstrating that missing a response does not authorize another effect.

A local transaction should not remain open while waiting on the network: this creates locks without making the partner an atomic participant. The laboratory exposes that separation. In production, payload includes account scope and contract; integration must address key retention, expiry and provider compatibility.

Executable case lab on GitHub

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

State should guide the next action

DART / ORIGINAL PROPOSAL

```dart
sealed class PaymentState {}
class Unknown extends PaymentState {
  final String intentId;
  Unknown(this.intentId);
}
class Confirmed extends PaymentState {
  final String receiptId;
  Confirmed(this.receiptId);
}

String nextAction(PaymentState state) => switch (state) {
  Unknown() => 'Check the existing payment',
  Confirmed() => 'Open receipt',
  _ => 'Continue',
};
```

Illustrative Dart, not executed; states and copy require product and accessibility review.

The interface offers lookup of the existing attempt when the outcome is unknown. The wording is only an example: it must explain what is being checked without asserting debit or failure before confirmation. A new-purchase button must represent explicit new intent rather than a disguised retry.

Local persistence helps app recovery, but the server remains the reference. Reinstallation, device changes or cleared storage must not automatically create another charge. After session recovery, the client queries authorized recent intents. Recipient information should enable recognition while minimizing sensitive detail and never exposing integration credentials.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Identity before heuristics

Contract	Invariant	Boundary
Same key	Same content and receipt	Defined scope and retention
Different content	Explicit conflict	No silent update
New intent	May repeat the amount	Own consent
Correction	Principal/charge linkage	Policy and reconciliation

JSON / ORIGINAL PROPOSAL

```json
{
  "intent_id": "synthetic-intent-10",
  "currency": "BRL",
  "amount_cents": 90000,
  "merchant_ref": "synthetic-merchant",
  "payload_digest": "sha256:...",
  "provider_key": "synthetic-intent-10",
  "state": "unknown",
  "next_action": "query_existing_intent"
}
```

Synthetic contract in cents; not a NuPay capture.

A digest does not replace field validation and must not serve as the sole authorization. Financial correction is a separate operation linked to the original effect; it neither deletes intent nor turns a confirmed attempt into an event that never occurred. Interest and principal need distinct trails for complete, auditable resolution.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Fail after the external effect

PYTHON / ORIGINAL PROPOSAL

```python
    def test_timeout_restart_and_retry_produce_one_charge(self):
        with self.assertRaises(TimeoutError):
            self.c.pay('intent1', 90000, 'merchant', True)
        self.assertEqual(self.c.state('intent1'), 'unknown')
        self.assertEqual(self.charges(), 1)
        self.c.db.close()
        self.c = IntentClient(self.path / 'client.db', self.p)
        self.assertEqual(self.c.pay('intent1', 90000, 'merchant'),
                         'provider:intent1')
        self.assertEqual(self.charges(), 1)
        self.assertEqual(self.c.state('intent1'), 'confirmed')

    def test_same_amount_can_be_two_explicit_purchases(self):
        self.c.pay('intent1', 90000, 'merchant')
        self.c.pay('intent2', 90000, 'merchant')
        self.assertEqual(self.charges(), 2)
```

Literal excerpts; 9 tests passed, including recovery after timeout.

The most important adverse scenario has already produced a provider effect. The assertion verifies one charge after reopening the client and repeating the call. Another test permits two equal purchases with different intents, preventing a protection that indiscriminately blocks legitimate payments.

The suite also checks amount conflicts, changed recipients, repeated timeouts and invalid cents. It does not cover a real network, partner key retention, interest or merchant cancellation. The integration contract must test these behaviors with the provider, especially a late response after the journey has moved to support.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Measure ambiguity and confirmed effects

Metric	Definition	Collection / segment
Unknown outcome	Intents without outcome / eligible	Events; provider and version
Confirmed duplicate	Unintended repetitions / intents	Review; reason
Reconciliation time	p95 to definitive outcome	By age and partner
False block	Legitimate purchases blocked / reviewed	Independent sample

Current rates are unknown. A network exception cannot automatically count as duplication. Confirmation requires linking intents and examining effects while preserving legitimate equal purchases. Complete-correction measurement checks principal and associated charges without assuming that every adjustment is a bank loss.

The experiment begins with shadow intent recovery and failure tests. The canary compares mobile versions and compatible partners while holding authorization policy constant. Any new duplication caused by the control, blocked legitimate payment or lost traceability stops expansion. Conversion alone is insufficient evidence of success.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

A strong case does not prove high ROI

Assumed scenario	Potential/year	Operations/year	Year 1 after cost
Conservative	R$ 4,050	R$ 36,000	R$ -211,950
Base	R$ 27,000	R$ 36,000	R$ -189,000
Ambitious	R$ 108,000	R$ 36,000	R$ -108,000

Annual assumptions: 30,000, 60,000 or 100,000 ambiguous intents; confirmed duplication in 0.5%, 1% or 2%; incremental prevention of 30%, 50% or 60%. This yields 45, 300 or 1,200 avoided incidents. I assume R$ 50 in support/reconciliation and R$ 40 in irrecoverable charges per incident, totaling R$ 90. The complaint's amounts are not used as average bank cost.

Assumed implementation: R$ 180,000; annual operations: R$ 36,000. In the base case, 300 × 90 = R$ 27,000; year one is R$ 27,000 - 36,000 - 180,000 = -R$ 189,000. Break-even requires 2,400 actually prevented incidents in year one. None of these scenarios independently justifies full investment. A smaller correction reused across a platform may have different economics and needs a separate estimate.

Separate flows from losses

Refunded principal is not automatically a bank expense. Operational capacity, absorbed charges and customer money are distinct categories. The real incident supplies neither population frequency nor realized benefit.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

A partner-dependent roadmap

Proposed phase	Owner and deliverable	Gate
Days 0-15	Payments: provider contract	Idempotency/lookup verified
Days 16-30	Staff + mobile: persistent intent	Restart resumes same operation
Days 31-60	Operations: failures and shadow	External effect reconciled
Days 61-90	EM + product: partner canary	No duplicates or false blocks

The key dependency is knowing what the partner actually guarantees and how long it retains a key. If lookup cannot distinguish pending from absent, the design must preserve that ambiguity. The 90-day schedule is proposed planning, not a commitment by Nubank or its provider.

The EM negotiates integration, testing and support capacity; the Staff Engineer documents guarantees for each failure scenario. Mobile delivery must wait for the server contract. Better wording without state recovery can merely conceal the issue. The economic gate is explicit too: start with a limited change when scenarios do not support a new platform.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Operate uncertainty without repetition

The canary needs an unknown-intent queue with age, partner and next step. Alerts carry authorized investigation references rather than complete payment data. Payments owns the effect, integration owns the external contract and support owns continuity of customer communication.

The runbook starts by looking up existing intent and provider evidence. It does not tell an operator to create another charge to see whether it works. Late confirmation must update the same case and prevent duplicate correction. A refund request requires its own authorized workflow; it is not an automatic consequence of timeout.

Rollback disables the new experience but preserves recorded intents and confirmations. I would simulate response loss, duplicate callbacks, reordered responses and unavailable lookup. I would also test recovery after app reinstallation. The stopping condition is any scenario in which the displayed state induces a financial action incompatible with what is known.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

My contribution to the contract

As Staff, I would make local guarantees and external dependencies explicit. I would reject exactly-once as a slogan for the entire journey. The design must identify transaction boundaries, deduplication and uncertainty. I would also define retry versus new intent so duplicate prevention does not obstruct legitimate purchases.

As Engineering Manager, I would coordinate mobile, payments, partner and support around reproducible failure scenarios. The team needs time for exercises and reconciliation, not merely a visual delivery target. I would put project cost and evidence of incidence into the same prioritization decision.

The rejected alternative is automatically blocking any payment with the same amount and recipient within a short window. That rule can create false positives and does not solve response loss. Identified intent offers a more precise contract when the partner cooperates. Where it does not, that limitation must remain visible in the design and operational plan.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Review a single effect under an assumption

- Run python3 labs/case_10.py: 9 local tests with no financial calls.
- Inspect Provider.charge and IntentClient.pay; identify the two separate commits.
- Inject timeout_after_provider = True and observe unknown state with an existing charge.
- Resume with the same key; then create a separate explicit intent and verify two legitimate purchases.

The demonstrated guarantee depends on correct deduplication by the synthetic provider. The laboratory does not prove that a real partner offers equivalent retention, scope or lookup. Production requires contractual tests, authentication, currency, expiry rules and independent effect reconciliation.

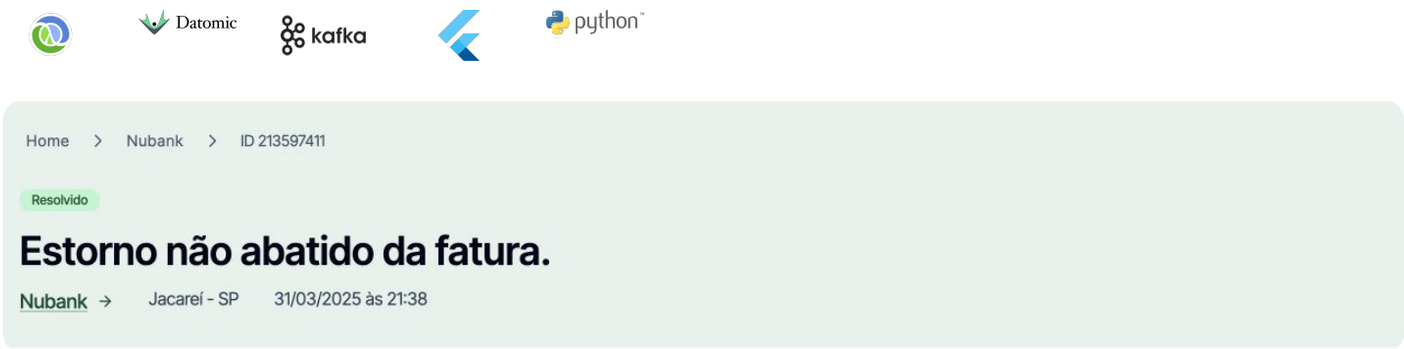
Proceed only when the external contract is confirmed, messaging guides the correct action and the pilot demonstrates fewer repeats without blocking valid purchases. Otherwise limit the change to information recovery and assisted operations. The contribution is to make uncertainty manageable and prevent a lost response from becoming another financial obligation.

Executable case lab on GitHub

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

11 · Visible credit, disputed balance

11



Actual crop of the public source, captured 23 October 2026. The text below includes the company reply and reported outcome.

In complaint 213597411, dated 31 March 2025, the customer described three refunds whose sum seemed correct but whose deduction from the bill was unclear. Nubank said they were correctly applied and explained purchases minus early payments minus refunds. The customer confirmed resolution on 8 April. The source establishes neither an accounting defect nor the specific action that resolved it.

Chapter scope

Investigate understanding and read consistency. Individual amounts are not public. The proposal does not presume withheld money; it starts with explaining every credit and its corresponding balance.

Reclame AQUI - case 11 · 2025-03-31

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

A correct total can remain opaque

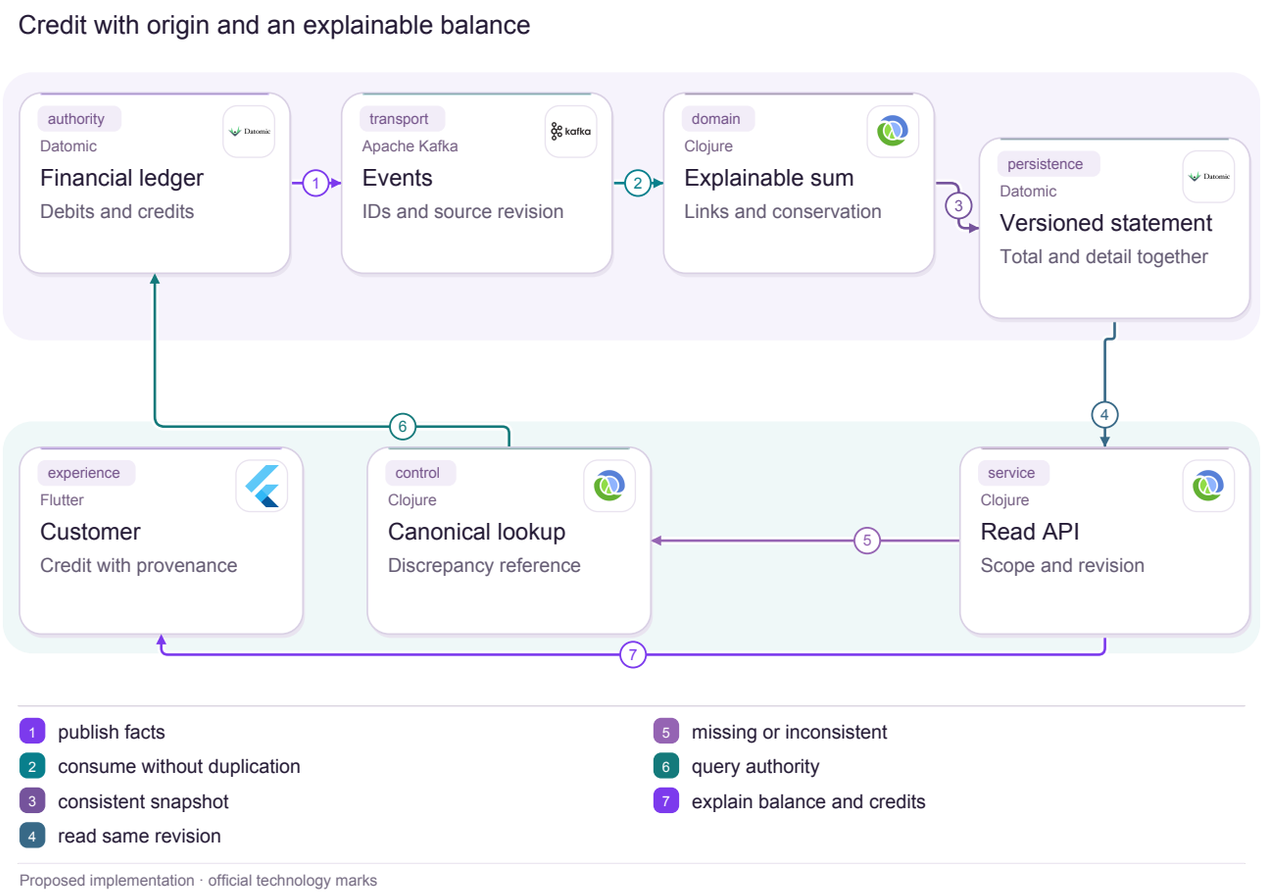
Customers need to recognize the original purchase, identify all associated credits and understand the difference between an open balance and a closed document. Three green rows do not necessarily explain how much the amount due changed. The proposal is a navigable calculation breakdown with explicit links between refund, purchase and statement revision.

The falsifiable hypothesis is that decomposition reduces interpretation errors when credit has already been correctly recorded. I would test concrete tasks: finding the canceled purchase, adding partial refunds and explaining the balance after an early payment. If participants still struggle, I would investigate language, grouping or revision discrepancies rather than automatically blame customer attention.

There is also an accounting boundary: a presentation model cannot correct the ledger to make its sum look consistent. The financial source retains authority. The app should demonstrate the calculation for the queried revision and offer investigation when it cannot reconcile it. Closing documents and open views need reference dates without promising identical values across different periods.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Total and detail share a revision



Proposed external architecture; not Nubank's internal implementation.

Events preserve ledger identity and revision. The aggregator links refunds to purchases and builds a snapshot whose total and detail share one revision. Customers can expand the explanation without receiving a second sum computed by independent rules.

Canonical lookup handles missing or inconsistent data; it is not permission to mix values from different revisions. The interface indicates when the reference changes. The design does not cover interest, currency exchange or every entry in a real product; these require integration with financial authority.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Core: identity and refund boundaries

PYTHON / ORIGINAL PROPOSAL

```python
    def append(self, entry):
        if not isinstance(entry.id, str) or not entry.id.strip():
            raise ValueError('nonempty entry identity required')
        if type(entry.cents) is not int or entry.cents <= 0:
            raise ValueError('positive integer cents required')
        if entry.kind not in {'purchase', 'refund', 'payment'}:
            raise ValueError('unknown entry type')
        with self.lock:
            old = self.entries.get(entry.id)
            if old:
                if old != entry:
                    raise ValueError('entry identity conflict')
                return
            if entry.kind == 'refund':
                purchase = self.entries.get(entry.parent)
                if not purchase or purchase.kind != 'purchase':
                    raise ValueError('missing purchase provenance')
                used = sum(e.cents for e in self.entries.values()
                           if e.kind == 'refund' and e.parent == entry.parent)
                if used + entry.cents > purchase.cents:
                    raise ValueError('refund exceeds purchase')
            elif entry.parent:
                raise ValueError('unexpected parent')
            self.entries[entry.id] = entry

    def snapshot(self):
        with self.lock:
            return tuple(self.entries.values())
```

Literal excerpt from labs/case_11.py; an in-memory model with a local lock.

Each entry has immutable identity. A refund must reference an existing purchase, and principal refunds cannot exceed that purchase in the model. Identical retries do not create a second reduction. A snapshot returns a tuple of immutable objects.

That boundary is deliberate: additional compensation, interest and other categories need their own types rather than a disguised oversized refund. The laboratory has no disk persistence or distributed consumption. The lock demonstrates exclusion within one local authority; a real service needs an equivalent transaction or conditional write.

Executable case lab on GitHub

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

One shared sum function

CLOJURE / ORIGINAL PROPOSAL

```clojure
(defn explain-statement [entries]
  (let [sum-kind (fn [kind]
                   (reduce + (map :cents
                     (filter #(= kind (:kind %)) entries))))
        purchases (sum-kind :purchase)
        refunds (sum-kind :refund)
        payments (sum-kind :payment)
        signed (- purchases refunds payments)]
    {:purchases purchases
     :refunds refunds
     :payments payments
     :amount-due (max 0 signed)
     :credit-balance (max 0 (- signed))}))
```

Illustrative, unexecuted Clojure; simplified rules equivalent to the laboratory.

The reducer separates purchases, refunds and payments. When the signed balance is negative, it presents a statement credit balance rather than a negative amount due. This preserves financial meaning when the visible payable total is bounded at zero. Both quantities remain reconcilable.

App, PDF and support should consume the same projection or a shared calculation contract with an explicit revision. Copying the formula across three clients increases divergence risk. Real implementation must add accounting-approved categories and cross-channel parity tests. The function does not decide posting dates or whether a merchant actually returned funds.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Provenance is part of the contract

Element	Rule	Privacy
entry_id	Stable identity	No account in public ID
parent	Original purchase for refund	Authorized owner only
revision	Consistent total and detail	Avoid excess history
credit_balance	Credit distinct from amount due	Accessible text

JSON / ORIGINAL PROPOSAL

```json
{
  "statement_id": "synthetic-statement-11",
  "revision": 4,
  "currency": "BRL",
  "purchases_cents": 10000,
  "refunds_cents": 10000,
  "payments_cents": 0,
  "amount_due_cents": 0,
  "refund_entries": ["r0", "r1", "r2"],
  "source_purchase": "p"
}
```

Synthetic three-refund example; no amount was extracted from the complainant's account.

A many-to-one relationship is necessary when money returns in parts. Purchase and credit totals must not be duplicated merely because they appear in both summary and detail. A short support reference locates the same revision without asking customers to resend every document or expose unrelated purchase information.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Test sums, revisions and concurrency

PYTHON / ORIGINAL PROPOSAL

```python
    def test_three_refunds_reduce_exactly_once(self):
        for i, cents in enumerate((3333, 3333, 3334)):
            refund = Entry('r' + str(i), 'refund', cents, 'p')
            self.s.append(refund)
            self.s.append(refund)
        result = explain(self.s.snapshot())
        self.assertEqual(result['refunds'], 10000)
        self.assertEqual(result['amount_due'], 0)
        self.assertEqual(result['revision'], 4)

    def test_old_snapshot_does_not_change_after_new_credit(self):
        old = self.s.snapshot()
        self.s.append(Entry('r', 'refund', 5000, 'p'))
        self.assertEqual(explain(old)['amount_due'], 10000)
        self.assertEqual(explain(self.s.snapshot())['amount_due'], 5000)
```

Literal excerpts; 9 local tests passed.

The synthetic split 3,333 + 3,333 + 3,334 demonstrates exact conservation of 10,000 cents. Replaying each credit leaves the result unchanged. An old snapshot stays stable after a new refund, explaining why two legitimate revisions can display different balances.

Other tests reject missing purchase provenance, excess refund, identity conflict and invalid cents. The concurrency test attempts two 6,000 refunds against 10,000 and accepts only one. Real contractual rules, closing, tax and interest are not tested. These limits should be visible before using the example as an implementation reference.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Success means demonstrated understanding

Metric	Numerator / denominator	Window and segment
Understanding	Correctly answered tasks / tasks	Sessions; accessibility
Interpretive contact	Questions on applied credit / refunds	30 days; refund type
Parity	Reconciled snapshots / compared	Daily; channel and revision
Provenance	Credits with valid origin / credits	Continuous; integration

Baselines are unknown. The experiment uses equivalent tasks and comparable groups with the same financial calculation. It measures accuracy and time without confusing a faster screen with better understanding. Screen readers must announce sign, value and linkage; color cannot be the sole signal of credit.

Contact classification is audited so a financial error is not mislabeled as customer confusion. The canary stops for sum divergence, mixed revisions or worse understanding among assistive-technology users. Detail clicks indicate use rather than success. Final validation asks participants to explain the balance in their own words.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Economics depend on avoided questions

Assumed scenario	Potential/year	Operations/year	Year 1 after cost
Conservative	R$ 16,000	R$ 24,000	R$ -128,000
Base	R$ 120,000	R$ 24,000	R$ -24,000
Ambitious	R$ 400,000	R$ 24,000	R$ 256,000

Annual assumptions: 200,000, 500,000 or 1 million eligible refunds; 0.005, 0.015 or 0.025 fewer interpretive contacts per refund; R$ 16 per contact. Assumed implementation: R$ 120,000; operations: R$ 24,000/year. The calculation values support capacity, not credit returned to the customer.

In the base case, 500,000 × 0.015 × 16 = R$ 120,000. After operations and implementation, year one is -R$ 24,000. Break-even requires 600,000 refunds equivalent to the base assumption. If the product already has a common projection, incremental investment may be lower; the estimate must be redone without automatically carrying over the assumed benefit.

Do not count the same journey twice

Exclude duplication contacts covered in case 10 and allocation contacts in 09. Released hours become cash only through concrete spending changes. This study has observed no Nubank incidence rate or saving.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

A calculation-and-language roadmap

Proposed phase	Owner and deliverable	Gate
Days 0-15	Accounting + design: scenarios	Sums and questions approved
Days 16-30	Staff: projection and links	Total/detail share revision
Days 31-60	Mobile + accessibility: prototype	Assistive understanding validated
Days 61-90	EM + product: experiment	Accuracy improves; sums hold

The 90-day period is a planning assumption. Representative examples of partial refunds, payments and closing are needed first, using authorized access or synthetic data. Without that coverage, an attractive screen may work only for the simple case and fail on the journey that generates contact.

Staff defines revision and invariants; accounting approves semantics; design validates language; the EM protects research and iteration capacity. Acceptance is not a checklist of rendered components. Customers must recognize credit origin and understand balance, while support reproduces the explanation using the same reference.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Operations with an identifiable fallback

The projection first runs in shadow and compares sums with the financial reference. During the canary, calculation detail appears on demand for a limited cohort. If revision changes during lookup, the interface announces the update and recomputes the complete set rather than replacing only the total while retaining old details.

Alerts cover orphan credits, excess principal refunds and cross-channel revision divergence. The projection team investigates processing; accounting resolves semantics; support distinguishes questions from errors. Logs carry references rather than complete purchase descriptions or personal documents.

Rollback hides the new breakdown and retains the canonical balance, preserving IDs for investigation. An exercise should simulate repeated events, refund arrival before purchase and an older PDF revision. Reconciliation must explain why an event was rejected or awaited a dependency. A credit must never be silently discarded because it arrived out of order.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

My contribution to trust in the sum

As Staff, I would define a financial representation preserving sign, origin and revision. I would avoid three independent calculations for app, PDF and support. The technical choice is an explainable projection with a canonical source and explicit out-of-order policy, without making the interface a financial authority.

As Engineering Manager, I would organize delivery with accounting, experience, accessibility and operations. I would reserve time to observe users interpreting difficult scenarios, because understanding does not automatically emerge from a correct table. The backlog would cover language and support as well as processing and telemetry.

I would reject solving the issue solely by changing refund color or adding an applied badge. That strengthens a claim without making it verifiable. The tradeoff is enough detail without overwhelming the initial screen: a simple summary, progressive explanation and a total rigorously linked to revision. These are proposed contributions, not claims of previous work at the bank.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Handoff: reproduce the explanation

- Run python3 labs/case_11.py: 9 tests with synthetic in-memory data.
- Read Entry, Statement.append, snapshot and explain; observe debit, credit and payable balance.
- Repeat three refunds, retain an old revision and compare it with the new one.
- Before production: persistence, event ordering, accounting categories, authorization and cross-channel parity.

A reviewer should explain why a credit reduces balance and why an old revision remains different. The solution does not depend on hiding history or teaching customers to manually redo every accounting calculation. The laboratory demonstrates a small contract that product and accounting can discuss.

Proceed only if the projection reconciles, language improves understanding and assistive guardrails hold. Financial error requires ledger correction first. If the issue is interpretive, an experiment can validate a smaller improvement. The case is stronger when it respects that distinction, including the fact that the historical complaint was resolved.

Executable case lab on GitHub

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

12 · The later response changes the context

12

Actual crop of the public source, captured 23 October 2026. The text below includes the company reply and reported outcome.

Complaint 230126691, dated 23 October 2025, describes delivery messages without receipt and later a fourth unsuccessful attempt. Nubank reports canceling unreceived cards and, on 10 November, a new delivery, activation and use. The page remains In reply, without final customer confirmation. The card must not be described as still missing.

Media and evidence

The customer mentions home video, but no public URL was verified in the captures. The book uses the real page as documentary evidence; it invents no recording, signature or delivery photo.

Reclame AQUI - case 12 · 2025-10-23

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

A supplier states a fact; the product decides a state

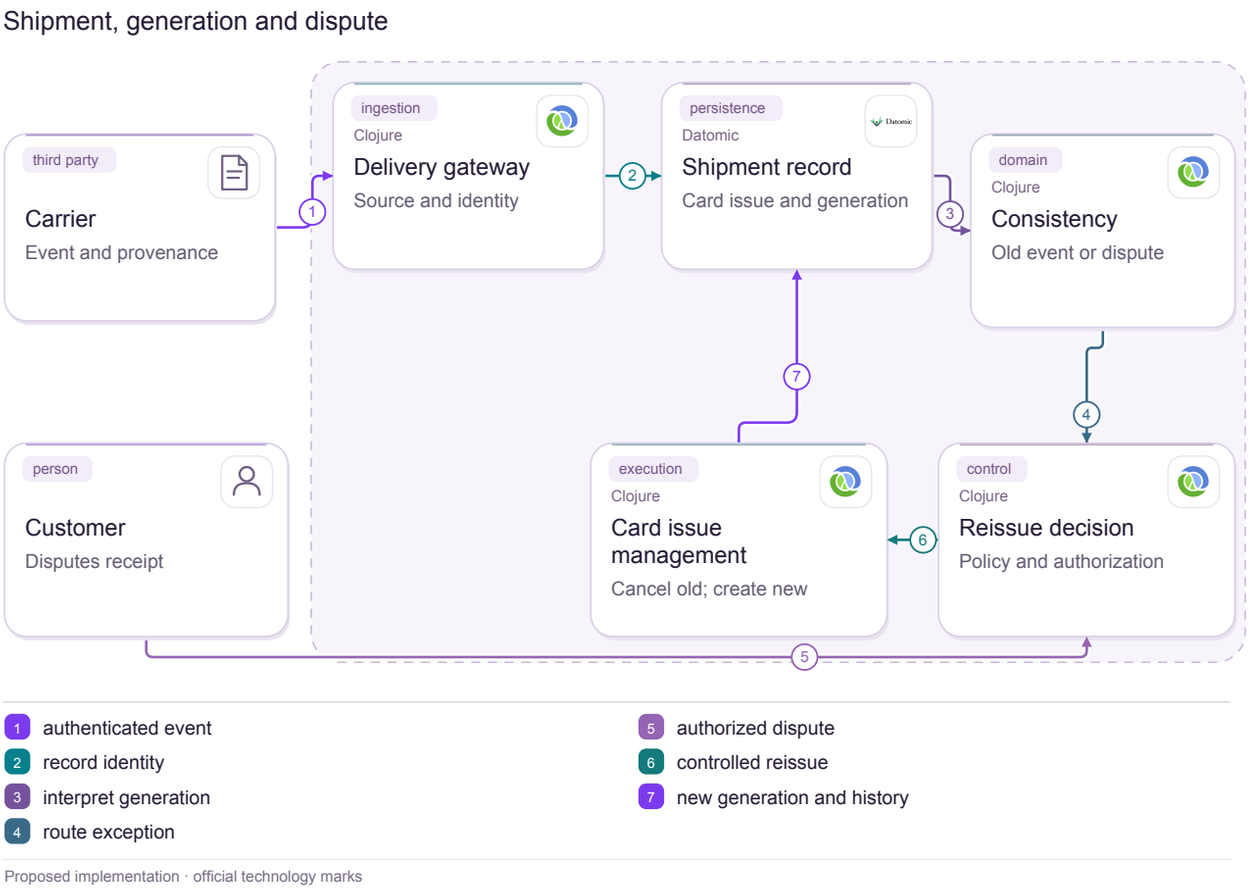
A carrier event may reliably record what the carrier declared while remaining insufficient to establish possession by the recipient. The journey should preserve that distinction. Declared delivery, disputed delivery and secure activation have different authorities; collapsing them into one status prevents handling disagreement without overwriting information.

The hypothesis is that some repeated reissues can be avoided when events, card issues and shipments have their own identities and an end-to-end exception owner. I would reconstruct authorized journeys by card generation, distinguishing initial loss, replacement order, reissue and late events. If recurrence has another origin, such as address or route performance, the plan must address that demonstrated operational cause.

The study does not conclude that a supplier falsified delivery or that the bank used a particular integration. Investigation should seek proof, provenance, time and escalation policy with minimum access. Customers should not need to publicly expose an address, signature or home video to obtain tracking and support.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

The old card cannot govern the new one



Proposed external architecture; not Nubank's internal implementation.

The gateway preserves event identity and shipment reference. The record links each shipment to a card generation. Consistency checks route old events and disputes to controlled decisions without silently changing the current card.

The customer disputes receipt through an authenticated channel. Reissue creates a new generation and cancels the earlier card under policy; it is not an automatic consequence of any delay. Secure activation remains a separate contract. The external provider receives neither an internal technology logo nor authority to activate the card.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Core: generation and persistent dispute

PYTHON / ORIGINAL PROPOSAL

```python
    def event(self, ident, generation, kind, scope):
        if not isinstance(ident, str) or not ident.strip():
            raise ValueError('nonempty event identity required')
        if type(generation) is not int or generation < 1:
            raise ValueError('positive integer generation required')
        required = {'delivered': 'carrier', 'dispute': 'customer',
                    'activate': 'customer_verified'}
        if kind not in required or required[kind] != scope:
            raise PermissionError('scope mismatch')
        payload = json.dumps([generation, kind, scope])
        with self.db:
            self.db.execute('BEGIN IMMEDIATE')
            old = self.db.execute('SELECT payload FROM events WHERE id=?',
                                  (ident,)).fetchone()
            if old:
                if old[0] != payload:
                    raise ValueError('event conflict')
                return self.state()
            current, carrier, disputed, active = self.state()
            if generation > current:
                raise ValueError('future generation; event not consumed')
            if generation == current:
                if kind == 'delivered':
                    carrier = 'delivered'
                elif kind == 'dispute':
                    disputed, active = 1, 0
                elif kind == 'activate':
                    if disputed:
                        raise ValueError('dispute requires separate resolution')
                    active = 1
                self.db.execute('UPDATE card SET carrier=?,disputed=?,active=? '
                                'WHERE id=1', (carrier, disputed, active))
            self.db.execute('INSERT INTO events VALUES(?,?)', (ident, payload))
        return self.state()
```

Literal excerpt from labs/case_12.py; synthetic local state for one account.

An old-generation event is recorded but cannot alter the current card. Repeated delivery also cannot clear a dispute. The model distinguishes carrier declaration, dispute and activation, preventing a webhook from automatically turning delivery into authorized use.

The supplied scope is trusted context injected by the server, not a freely submitted user credential. Partner signatures and real activation factors are not implemented. A dispute blocks activation in this model; operational resolution needs a separate workflow, while reissue creates another controlled generation.

Executable case lab on GitHub

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Normalize without losing provenance

CLOJURE / ORIGINAL PROPOSAL

```clojure
(defn accept-delivery [state event]
  (cond
    (not= (:generation state) (:generation event))
    (assoc state :last-event-result :old-generation)

    (= :delivered (:type event))
    (assoc state :carrier-state :delivered)

    (= :customer-dispute (:type event))
    (assoc state :disputed? true :active? false)

    :else state))
;; Carrier delivery does not clear :disputed? or set :active?.
```

Illustrative Clojure, not executed; not carrier or Nubank code.

The reducer does not discover physical truth from a status. It applies precedence and preserves the dispute. Normalization should retain the original event under restricted access alongside the common product state. Otherwise, details needed for investigation disappear during translation across suppliers.

Integration authenticates origin, limits replay and validates shipment/account linkage. An incoming timestamp cannot order the entire card lifecycle: clocks differ and suppliers may resend old information. Generation and identity provide stronger protection against an update for a canceled card appearing to confirm the new one.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

A physical-to-digital contract

Field	Authority	Rule
carrier_state	Carrier	Declaration with provenance
customer_disputed	Authenticated customer	Replay cannot clear it
card_generation	Card management	Old issue cannot overwrite
card_active	Secure activation	Independent from delivery

JSON / ORIGINAL PROPOSAL

```json
{
  "card_generation": 2,
  "shipment_ref": "synthetic-shipment-12",
  "carrier_state": "delivered",
  "customer_disputed": true,
  "card_active": false,
  "event_id": "carrier-event-17",
  "evidence_ref": null,
  "next_action": "investigate_delivery"
}
```

Synthetic state: declared delivery, open dispute and inactive card.

Delivery evidence may contain an address, signature or third-party image. Its reference should support purpose-bound access, expiry and auditing. The app presents enough information for customer action without publishing recipient data. Logistics remains responsible for obtaining partner evidence rather than transferring that integration burden to the customer.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Replays cannot close the exception

PYTHON / ORIGINAL PROPOSAL

```python
    def test_old_delivery_does_not_replace_new_generation(self):
        self.d.event('d1', 1, 'delivered', 'carrier')
        self.d.reissue(1, 'operator')
        self.d.event('late', 1, 'delivered', 'carrier')
        self.assertEqual(self.d.state(), (2, 'pending', 0, 0))

    def test_replayed_delivery_cannot_clear_dispute(self):
        self.d.event('d1', 1, 'delivered', 'carrier')
        self.d.event('c1', 1, 'dispute', 'customer')
        self.d.event('d1', 1, 'delivered', 'carrier')
        self.d.event('d2', 1, 'delivered', 'carrier')
        self.assertEqual(self.d.state(), (1, 'delivered', 1, 0))
        with self.assertRaises(ValueError):
            self.d.event('a1', 1, 'activate', 'customer_verified')
```

Literal excerpts; 10 laboratory tests passed.

The first test shows that old confirmation cannot mark generation two as delivered. The second replays the original webhook and sends another delivery event: the dispute remains open and activation blocked. That property matters more than a happy sequence of messages.

The suite also checks restart, event conflict, incorrect role, stale-generation reissue and cancellation of previous activation. It does not test physical delivery, cryptographic signatures or possession evidence. Integration exercises must validate real suppliers, expiry policies and failures between canceling an old card and issuing a new one.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Measure possession and exceptions, not just dispatch

Metric	Numerator / denominator	Window and segment
Disputed delivery	Disputes / declared deliveries	Weekly; route and partner
Repeated reissue	New cards / replacement journey	30 days; generation
Exception time	p95 to an owned decision	Daily; reason
Invalid card active	Canceled-card activations / attempts	Continuous; integration

The baseline is unknown. Operational data should connect journey, shipment and generation without exporting addresses to general telemetry. Time to confirmed possession may depend on activation or another approved secure signal; it should not be inferred solely from a carrier event.

The pilot compares equivalent routes and suppliers while controlling seasonality and shipping changes. Fewer reissues count as improvement only if possession or resolution is confirmed. Expansion stops for unauthorized activation, automatic dispute closure or excessive personal-data collection. Cutting cost by refusing necessary replacement is an adverse outcome, not success.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Avoided shipping has an economic boundary

Assumed scenario	Potential/year	Operations/year	Year 1 after cost
Conservative	R$ 3,900	R$ 24,000	R$ -110,100
Base	R$ 15,600	R$ 24,000	R$ -98,400
Ambitious	R$ 46,800	R$ 24,000	R$ -67,200

Annual assumptions: 50,000, 100,000 or 150,000 eligible shipments; avoided reissues in 0.2%, 0.4% or 0.8% of journeys. That means 100, 400 or 1,200 avoided reissues. I assume R$ 9 per card, R$ 18 shipping and R$ 12 incremental support, totaling R$ 39. No unit cost comes from the complaint.

Assumed implementation: R$ 90,000; operations: R$ 24,000/year. In the base case, 400 × 39 = R$ 15,600; year one is R$ 15,600 - 24,000 - 90,000 = -R$ 98,400. Break-even requires approximately 2,924 avoided reissues in year one. These scenarios do not independently support broad investment; they favor a focused pilot or extension of an existing workflow.

Benefit remains unproven

One customer’s reported count is not a population rate. Do not treat every dispute as fraud or every reissue as avoidable. Support already included in full logistics cost cannot be added again.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

A roadmap with supplier and operations

Proposed phase	Owner and deliverable	Gate
Days 0-15	Logistics: events and route map	Identities and sources checked
Days 16-30	Staff + cards: generation contract	Old issue cannot alter current
Days 31-60	Operations + partner: investigation	Exception has owner and evidence
Days 61-90	EM: limited-route pilot	Possession improves; security holds

The calendar is a 90-day hypothesis. The critical dependency is supplier cooperation, including event quality and evidence access. If that source is unavailable, the first delivery should expose the gap and define an alternative rather than create false certainty in the app.

The EM coordinates logistics, cards, support and partner capacity. Staff defines generation, identity and transitions. Replacement policy needs operational approval and must not emerge as a side effect of a technical retry. Every gate includes a disputed journey, legitimate delivery and a late event for a canceled card.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

A disputed-delivery runbook

Support opens a case linked to shipment and generation, preserving the customer's statement. The operator checks provenance and the next authorized action. Reissue without diagnosis is not the only button: investigation, an alternative channel and cancellation of an exposed card require clear policy and ownership.

Alerts separate missing updates, aging disputes and generation-incompatible events. Logistics owns partner follow-up; cards owns cancellation and activation; the service team owns event loss. Sensitive information stays in restricted tools rather than broadly visible tickets.

Rollback disables new interpretation while retaining generation and dispute history. It never reactivates canceled cards. Exercises include a repeated webhook after reissue, delivery assigned to the wrong card and unavailable proof. Reconciliation keeps customers informed without treating a tracking status as conclusive possession evidence.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

My contribution crosses the physical boundary

As Staff, I would define identity that survives reissue, carrier changes and late events. The central decision is separating declared delivery from secure activation while preserving dispute as its own fact. Architecture must tolerate an imperfect physical world without pretending message ordering resolves delivery truth.

As Engineering Manager, I would establish an owner for the complete exception and an operational forum with the supplier. The team must track resolution and cost per journey, not just cards dispatched. I would reserve capacity for exceptions and align the evidence required to move between stages.

I would reject automatically closing a case when a carrier resends delivered. I would also avoid collecting home videos as a default requirement. The tradeoff is sufficient investigation with minimal data and proportionate cost. Investment should follow verified routes and causes without extrapolating one complaint to the entire operation.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Handoff: review the current generation

- Run python3 labs/case_12.py: 10 tests with a local database and synthetic events.
- Read Delivery.event and reissue; identify who may declare delivery, dispute and activate.
- Create generation two and resend a generation-one event; the current card must remain unchanged.
- Before production: partner signature, identity, possession evidence, retention and dispute resolution.

The local demonstration verifies precedence and persistence, not whether a parcel arrived. Reviewers should recognize that boundary and request supplier contract tests before approving integration. They should also verify that support can reconstruct every card issue without confusing a canceled card with the current one.

Proceed only when investigation has an owner, events are correctly linked and activation security remains intact. If operational evidence shows a route problem, the intervention may be logistical. Engineering’s contribution is ensuring that the product represents that reality clearly rather than hiding it behind one status.

Executable case lab on GitHub

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

13 · Two amounts, different interpretations

13

Actual crop of the public source, captured 23 October 2026. The text below includes the company reply and reported outcome.

Complaint 244706833, dated 30 March 2026, describes a declined R$ 300 purchase despite a displayed R$ 800 value. Nubank replied that the image showed a configured virtual-card cap, not approved credit, and that purchases consumed the global line. Status is Responded. The source does not establish a 3DS failure or improper authorization.

Image referenced, not available

The response mentions a customer screenshot, but public AX exposes no address. Published evidence is the real page. The response's R$ 400/R$ 100 example is hypothetical, not this account's balance.

Reclame AQUI - case 13 · 2026-03-30

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Setting a cap does not grant credit

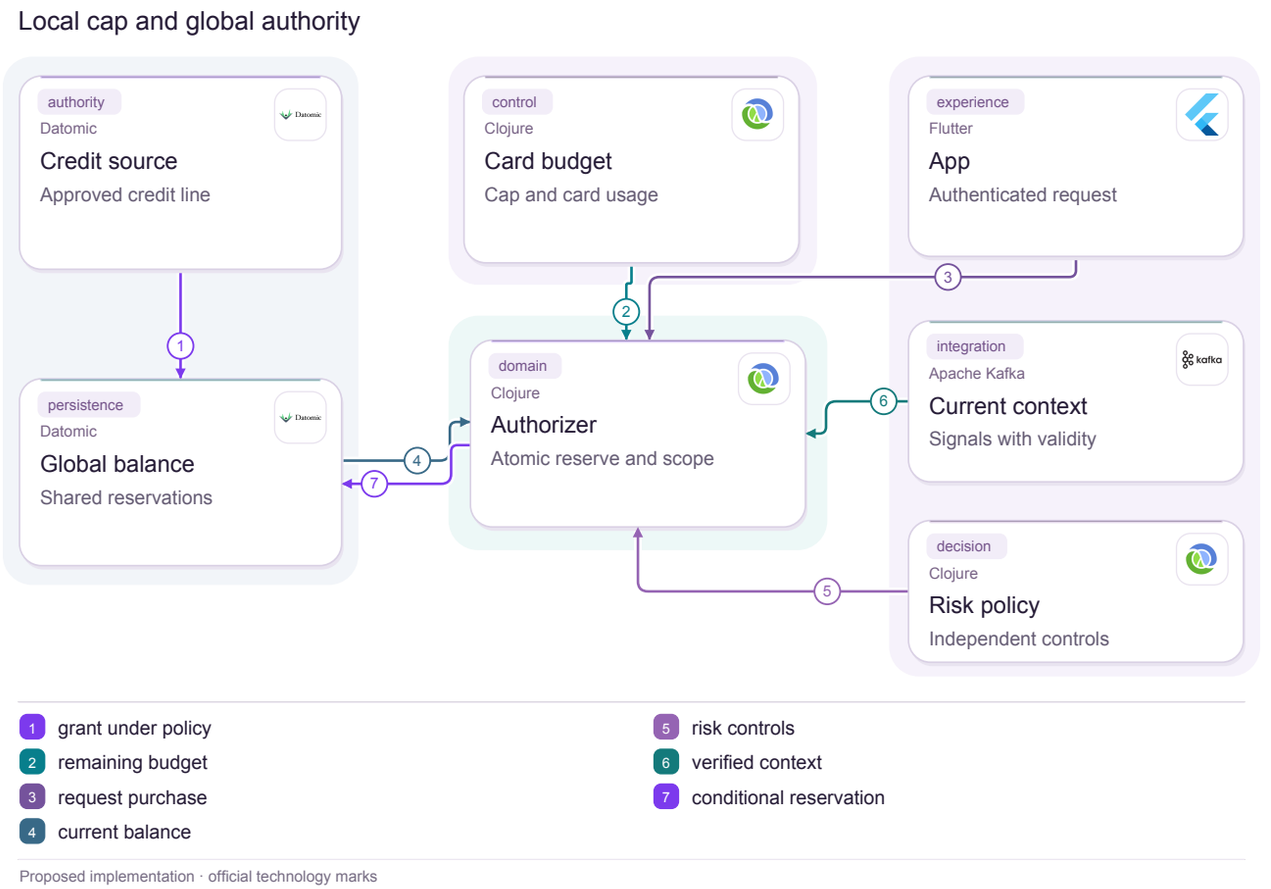
The product combines at least three concepts: approved credit, remaining global availability and a restriction chosen for a virtual card. A configurable number can look like a purchase promise when wording and hierarchy fail to explain what it controls. This proposal separates those facts and explains the tighter constraint without suggesting that moving a control grants more credit.

The falsifiable hypothesis is that some attempts and contacts result from this confusion rather than an authorization defect. I would observe users explaining what happens when they increase the virtual cap and compare their explanation with subsequent action. I would also investigate freshness: a correct view can become stale when another card consumes shared credit.

Authorization remains independent. Even with available credit, other controls may decline a purchase. The interface should provide current, sufficient information rather than an absolute approval guarantee. The study does not recommend removing risk controls, increasing credit or changing policy to manufacture a better conversion rate.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Credit is shared; reservation is atomic



Proposed external architecture; not Nubank's internal implementation.

The credit source defines the grant; global balance records shared reservations. Card budget adds a restriction while policy and context retain separate controls. The authorizer validates scope and reserves against current authority.

The screen is not a participant in that transaction: its preview can age. The diagram emphasizes authorization, while explainable reads consume versioned projections. Editing a virtual cap never changes global credit. Real risk validation is outside the laboratory and remains necessary for production.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Core: two cards, one balance

PYTHON / ORIGINAL PROPOSAL

```python
    def reserve(self, owner, ident, card, cents):
        if not isinstance(ident, str) or not ident.strip():
            raise ValueError('nonempty reservation identity required')
        if type(cents) is not int or cents <= 0:
            raise ValueError('integer cents required')
        with self.db:
            self.db.execute('BEGIN IMMEDIATE')
            available = self.preview(owner, card)
            old = self.db.execute('SELECT card,cents FROM holds WHERE id=?',
                                  (ident,)).fetchone()
            if old:
                if old != (card, cents):
                    raise ValueError('reservation conflict')
                return ident
            if cents > available:
                raise ValueError('insufficient global credit or card budget')
            self.db.execute('UPDATE account SET free=free-? WHERE owner=?',
                            (cents, owner))
            self.db.execute('UPDATE cards SET spent=spent+? WHERE id=?',
                            (cents, card))
            self.db.execute('INSERT INTO holds VALUES(?,?,?)', (ident, card, cents))
        return ident

    def set_cap(self, owner, card, cap):
        if type(cap) is not int or cap < 0:
            raise ValueError('integer cap required')
        with self.db:
            if self.db.execute('UPDATE cards SET cap=? WHERE id=? AND owner=?',
                               (cap, card, owner)).rowcount != 1:
                raise PermissionError('not owned')
```

Literal excerpt from labs/case_13.py; local SQLite reservations with integer cents.

The transaction reads global credit and card budget, checks reservation identity and only then reduces both. Concurrent purchases on different cards cannot spend the same balance. Account-holder authorization precedes return of an idempotent receipt, preventing a key from becoming a way to inspect another person’s operation.

The cap is separate configuration. The laboratory models a synthetic cumulative budget rather than the real product’s period. Reservation expiry, settlement, reversal and risk analysis are not implemented. Preview helps explain, but the financial decision revalidates everything inside the transaction.

Executable case lab on GitHub

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

An honest app preview

DART / ORIGINAL PROPOSAL

```dart
class CreditView {
  final int globalAvailableCents;
  final int virtualCapCents;
  final int virtualSpentCents;
  final DateTime asOf;
  CreditView(this.globalAvailableCents, this.virtualCapCents,
      this.virtualSpentCents, this.asOf);

  int get previewCents {
    final remaining = virtualCapCents - virtualSpentCents;
    final bounded = remaining < globalAvailableCents
        ? remaining : globalAvailableCents;
    return bounded < 0 ? 0 : bounded;
  }
  // A display estimate never replaces server authorization.
}
```

Illustrative, unexecuted Dart; the Python laboratory validates authority, not this component.

Preview takes the smaller of global credit and remaining card budget, bounded at zero. The formula is meaningful only when units, currency and periods are compatible. A real implementation must state those conditions and display its reference time rather than combine a monthly budget with usage from another window.

The component must distinguish chosen cap from available credit in text, without relying on color. Decline messages use safe, actionable reasons without exposing fraud controls. An estimate must not become a guaranteed-approval statement. Server revalidation protects against concurrency; state explanation prevents that protection from appearing arbitrary.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Credit and budget contracts

Field	What it represents	What it does not
global_available	Credit after reservations	Approval promise
virtual_cap	Configured restriction	New credit grant
virtual_spent	Usage in defined window	All financed balances
as_of	Read reference	Permanent state

JSON / ORIGINAL PROPOSAL

```json
{
  "account_ref": "synthetic-account-13",
  "card_ref": "virtual-a",
  "currency": "BRL",
  "global_available_cents": 20000,
  "virtual_cap_cents": 80000,
  "virtual_spent_cents": 30000,
  "preview_cents": 20000,
  "as_of": "synthetic-revision-2",
  "authorization_guaranteed": false
}
```

Synthetic values; not the private balance from the complaint.

The interface must not present one account’s data under another session. Cache scope needs account holder, card and revision. Safe decline reasons form a contract between risk and product; detailed logs remain restricted. Public disclosure of fraud signals is unnecessary to explain that virtual cap and approved credit are different concepts.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Concurrency attacks the authority

PYTHON / ORIGINAL PROPOSAL

```python
    def test_two_cards_cannot_overspend_shared_credit(self):
        barrier = threading.Barrier(2)
        results = []
        def attempt(card):
            client = Credit(self.path)
            barrier.wait()
            try:
                client.reserve('owner', 'hold-' + card, card, 30000)
                results.append('ok')
            except ValueError:
                results.append('declined')
            finally:
                client.db.close()
        threads = [threading.Thread(target=attempt, args=(c,)) for c in ('a', 'b')]
        for thread in threads:
            thread.start()
        for thread in threads:
            thread.join()
        self.assertCountEqual(results, ['ok', 'declined'])
        self.assertEqual(self.c.preview('owner', 'a'), 20000)
```

Literal test; the full suite passed 10 tests.

Two threads open separate connections and compete for a synthetic R$ 500, attempting R$ 300 reservations each. Only one can complete. The result does not depend on whether a screen showed enough credit beforehand. This demonstrates why mobile calculation cannot replace financial authority.

Other tests cover increasing a cap without creating credit, lowering budget, retry, incorrect owner, stale preview, key conflict and restart recovery. The suite does not test risk, card networks or reservation expiry. Those controls require their own tests and cannot be removed to reproduce the simplified example.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Understanding without weaker controls

Metric	Definition	Window and segment
Understanding	Users explaining cap correctly / tested	Sessions; accessibility
Confusion contacts	Contacts for this reason / eligible	30 days; mobile version
Decline after lookup	Credit declines / post-read attempts	Daily; read age
Outside policy	Incompatible approvals / audited	Continuous; risk control

Baselines are unknown. The first metric needs a comprehension task rather than a satisfaction question. The third distinguishes legitimately changed balances from stale reads. I would collect only necessary events and categories without recording sensitive purchase details in experience telemetry.

The experiment holds authorization policy constant and changes explanation only. Cohorts are segmented by virtual-card use and version. Expansion stops for cross-account leakage, misleading approval promises or increased approvals outside policy. Fewer attempts can be positive when customers now understand that a purchase does not fit available credit.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Value without treating declined purchases as revenue

Assumed scenario	Potential/year	Operations/year	Year 1 after cost
Conservative	R$ 6,400	R$ 12,000	R$ -65,600
Base	R$ 32,000	R$ 12,000	R$ -40,000
Ambitious	R$ 96,000	R$ 12,000	R$ 24,000

Annual assumptions: 100,000, 250,000 or 500,000 eligible users; 0.004, 0.008 or 0.012 fewer contacts per user; R$ 16 per contact. Assumed implementation: R$ 60,000; operations: R$ 12,000/year. The calculation covers support capacity only, without converting purchase volume into margin.

The base case is 250,000 × 0.008 × 16 = R$ 32,000. Year one is R$ 32,000 - 12,000 - 60,000 = -R$ 40,000. Break-even requires 562,500 users equivalent to the base assumption. The ambitious scenario yields R$ 24,000 modeled net capacity in year one, but depends on an unmeasured effect and cohort.

The calculation I do not make

I do not treat R$ 300 attempted as lost revenue or R$ 800 as approved credit. Purchase contribution analysis would require verified eligibility and margin plus a separate causal experiment. Group ARPAC cannot replace those inputs.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

A terminology-and-consistency roadmap

Proposed phase	Owner and deliverable	Gate
Days 0-15	Design + credit: terms and tasks	User separates cap from credit
Days 16-30	Staff + mobile: projection and revision	Cache respects account and period
Days 31-60	Risk + engineering: reserve tests	Concurrency preserves balance
Days 61-90	EM + product: limited experiment	Understanding improves; policy unchanged

The 90-day horizon is proposed planning. A small wording change can be validated before broader integration if it does not conceal staleness. The financial dependency is understanding card-usage semantics and reservation authority; product must not invent a simplified calculation without that agreement.

The EM coordinates design, mobile, cards, credit and risk. Staff defines which data may be eventually consistent and which decision must be atomic. Each phase includes accessibility and session-switch behavior. Success combines correct interpretation and security rather than seeking approval at any cost.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Stale reads must not become fictional credit

Deployment starts with shadow comparison between preview and authority, recording age and reason for differences. The canary changes presentation for a limited cohort. When lookup fails, the interface retains an identified reference or reports unavailability; it does not substitute configured cap as an equivalent value.

Alerts separate projection delay, scope error and reservation inconsistency. The financial team owns balance; mobile owns associating responses with the correct session; risk owns permitted communication reasons. Support receives an explanation consistent with the same revision without unrestricted access to internal signals.

Rollback restores the visual experience while preserving confirmed reservations. Exercises include concurrent cards, a cap lowered below usage, account switching during lookup and an old response arriving after a new one. Authorization must never depend on rendering order. Any committed balance needs reconciliation, not cache clearing as a financial correction.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

My contribution: clarity with authority

As Staff, I would separate configuration, projection and authorization. I would define reservation identity, global consistency and read scope. The central technical decision is keeping shared credit in an authority that arbitrates concurrency while experience provides an honest, understandable estimate.

As Engineering Manager, I would align product, credit and risk on terminology and experiment before delivery. The team needs to observe users rather than merely approve screenshots. I would protect accessibility, support and decline-reason analysis time, because improvement depends on language as much as integration.

I would reject relaxing controls to offset poor explanation with a higher approval rate. I would also avoid showing only the minimum number without its origin: that can solve one attempt while leaving confusion intact. The tradeoff is enough information for understanding while preserving risk boundaries and visual simplicity. No proposed contribution assumes my authority over bank policy.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Handoff: prove a cap cannot create balance

- Run python3 labs/case_13.py: 10 local tests, including two concurrent connections.
- Inspect Credit.preview, reserve and set_cap; raise the cap and observe unchanged global credit.
- Retry a reservation and query as another owner; no duplicate debit or receipt leakage is allowed.
- Before production: risk, card networks, expiry, settlement, reversals and real authorization.

Laboratory amounts are synthetic examples. The complaint's real account remains private and must not be reconstructed by inference from a configured cap. A reviewer should verify the financial property and separately check that the interface explains constraints without promising success.

The final decision requires improved understanding, semantically correct projections and no policy regression. If evidence shows wording alone is poor, prioritize a small improvement. If concurrent balance contention exists, address authority before presentation. The project is strongest when intervention matches the demonstrated problem rather than maximizing new services.

Executable case lab on GitHub

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

14 · The company acknowledges late execution

14

Actual crop of the public source, captured 23 October 2026. The text below includes the company reply and reported outcome.

In complaint 238574883, dated 23 January 2026, the customer describes difficulty advancing installments and conflicting guidance. Nubank acknowledges receiving payment and executing late; it reports applying installment 23 on 2 February, with 13 remaining. It explains payment conditions and says support met its deadline. Status is Responded, without final customer confirmation.

What remains unknown

No amount, rate, calculation breakdown or technical cause is public. The acknowledgment concerns late execution; it does not establish that every attempt or insufficient-funds rule was wrong.

Reclame AQUI - case 14 · 2026-01-23

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Receiving funds is not executing early repayment

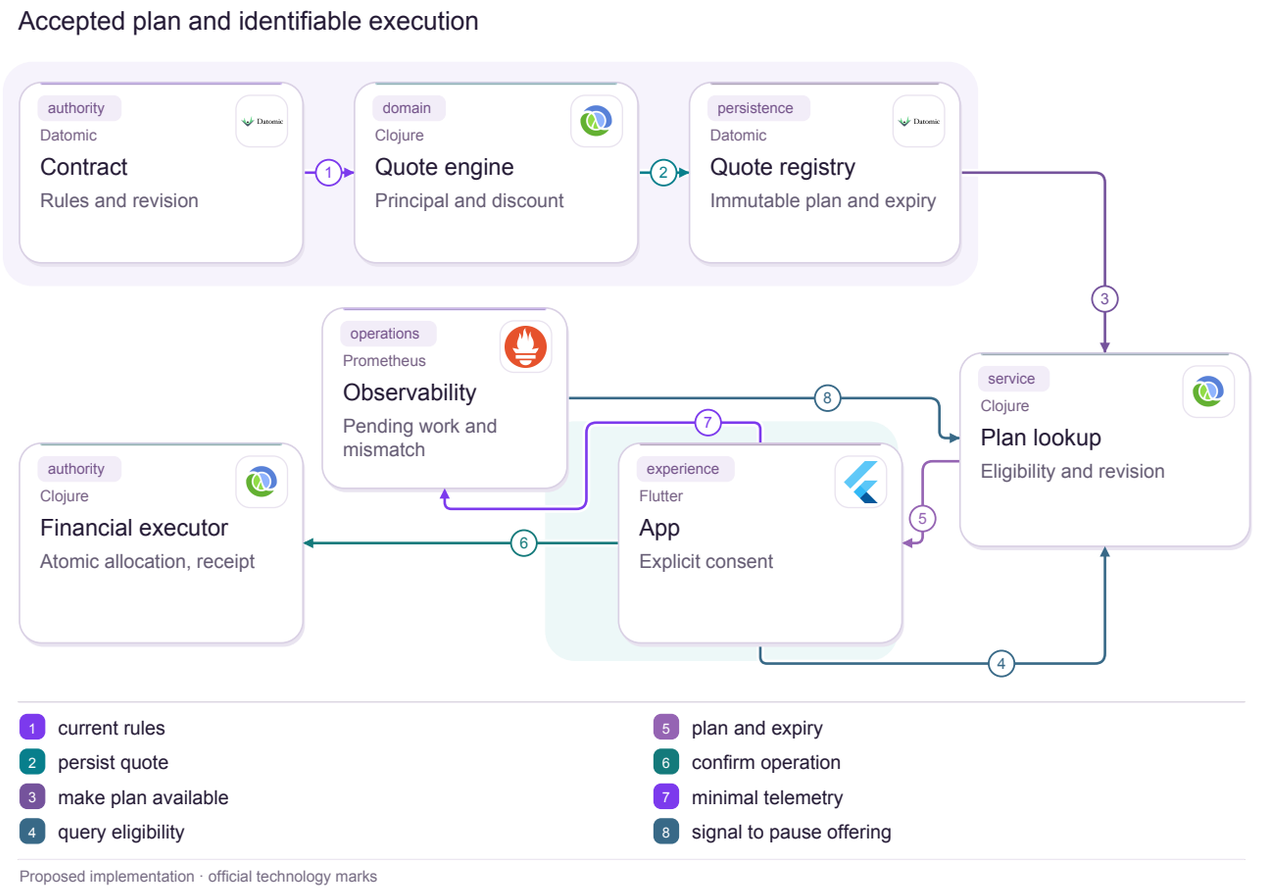
An early-repayment journey involves a discount offer, installment selection, payment and financial application. These events can occur at different times under different rules. The product must distinguish a request, a valid quote, received funds and an obligation actually settled. Without that distinction, a debit can appear to complete an operation still awaiting allocation.

The hypothesis is that a versioned quote linked to the final receipt and exception queue reduces funded operations without understandable execution. I would reconstruct a sample by quote_id and payment_id, comparing rules, validity, receipt and application. I would not assume an interface-only problem: calculation or execution discrepancies belong at the financial authority.

The laboratory proposes a synthetic atomic package of current bill and early repayment. It does not describe Nubank's current contractual priority. Real rules for partial application, cancellation and discount reversal need validation before implementation. The model supports discussion of the boundary between received money, consent to a plan and an obligation actually settled.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

A quote is a contract with expiry



Proposed external architecture; not Nubank's internal implementation.

The quote engine uses contract rules and revision to publish an immutable plan. The app queries validity and confirms an identified operation. The financial executor revalidates and applies within its authority, producing its own receipt; lookup does not replace that decision.

Observability signals pending work and discrepancies to the responsible operation, which may pause new offers. The diagram does not authorize a monitor to edit balances. Real settlement, contractual pricing and ledger integration remain necessary; the laboratory reduces them to one account and synthetic amounts.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Core: revalidate before applying

PYTHON / ORIGINAL PROPOSAL

```python
    def execute(self, ident, now, crash=False):
        with self.db:
            self.db.execute('BEGIN IMMEDIATE')
            row = self.db.execute('SELECT revision,bill,principal,discount,'
                'expires,received,executed FROM quotes WHERE id=?',
                (ident,)).fetchone()
            if not row:
                raise ValueError('unknown quote')
            revision, bill, principal, discount, expires, received, done = row
            needed = bill + principal - discount
            if done:
                return dict(applied=needed, unapplied=received - needed)
            current = self.db.execute('SELECT revision FROM contract').fetchone()[0]
            if now >= expires or revision != current:
                raise ValueError('expired or stale quote; funds remain unapplied')
            if received < needed:
                raise ValueError('full quoted amount not received')
            self.db.execute('UPDATE contract SET revision=revision+1,'
                            'settled=settled+?', (principal,))
            self.db.execute('UPDATE quotes SET executed=1 WHERE id=?', (ident,))
            if crash:
                raise RuntimeError('crash before commit')
        return dict(applied=needed, unapplied=received - needed)
```

Literal excerpt from labs/case_14.py; transactional execution in cents.

A quote is applied only when revision, validity and received amount agree. Failure before commit rolls back the complete execution. Once completed, a retry returns the original receipt even after expiry; it does not execute another early repayment.

Funds received against an expired or insufficiently funded quote remain recorded for resolution rather than disappearing or silently moving to another price. The laboratory does not implement automatic refunds. Clock and settlement facts must come from trusted server sources, and the supplied discount needs approved contractual calculation.

Executable case lab on GitHub

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Pure calculation does not replace a transaction

CLOJURE / ORIGINAL PROPOSAL

```clojure
(defn executable? [quote contract now received]
  (let [required (+ (:bill-cents quote)
                    (- (:principal-cents quote)
                       (:discount-cents quote)))]
    (and (= (:revision quote) (:revision contract))
         (< now (:expires-at quote))
         (>= received required))))

;; Run this check inside the same authority as financial application.
;; The clock, contract and settlement evidence are server-owned.
;; A completed operation returns its original receipt on retry.
```

Illustrative, unexecuted Clojure; synthetic rules for contract review.

The function expresses execution conditions, but evaluating it outside the transaction leaves a window for contract change. The service must compare revision and apply the change within the same authority. Required value must include all approved components rather than omit the current bill to make the offer look cheaper.

A material price or plan change requires renewed confirmation. Revalidation does not mean silently recalculating and debiting. History retains the accepted quote, cancellation reason and application receipt. Clojure can keep calculation deterministic; financial integration and event publication remain responsible for atomicity, durability and reconciliation.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Separate plan, funding and receipt

Object	Identity	Invariant
Quote	quote_id + revision	Immutable plan and expiry
Funding	payment_id	Same payment never adds twice
Execution	quote_id	One application after validation
Exception	Linked case	Funds always have an explicit destination

JSON / ORIGINAL PROPOSAL

```json
{
  "quote_id": "synthetic-quote-14",
  "contract_revision": 1,
  "currency": "BRL",
  "current_bill_cents": 5000,
  "principal_cents": 10000,
  "discount_cents": 1000,
  "required_cents": 14000,
  "received_cents": 14000,
  "execution_state": "pending",
  "expires_at": 100
}
```

Synthetic example: R$ 50 current bill + R$ 100 principal -R$ 10 discount = R$ 140.

The discount is not a bank saving to maximize; it is part of the contractual obligation to the customer. Presentation should separate principal, charges, discount and required total. Identifiers support investigation without exposing a complete contract in logs. The financial owner defines retention and compensation; support receives an accessible calculation and next action.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Crash and expiry challenge the plan

PYTHON / ORIGINAL PROPOSAL

```python
    def test_received_is_not_executed_and_crash_rolls_back(self):
        self.r.receive('p', 'q', 14000)
        with self.assertRaises(RuntimeError):
            self.r.execute('q', 90, crash=True)
        self.assertEqual(self.r.db.execute('SELECT settled FROM contract')
                         .fetchone()[0], 0)
        self.r.db.close()
        self.r = Repayment(self.path)
        self.assertEqual(self.r.execute('q', 90)['applied'], 14000)
        self.assertEqual(self.r.execute('q', 200)['applied'], 14000)
        self.assertEqual(self.r.db.execute('SELECT settled FROM contract')
                         .fetchone()[0], 10000)
```

Literal test; the complete suite passed 9 tests.

The test receives funds and fails after writes but before commit. The obligation remains unsettled; after restart, execution completes once. Retrying after expiry returns the same result, demonstrating the difference between recovering a receipt and accepting an expired quote.

Other tests cover partial funding, expiry, duplicate funding, conflict, a quote made stale by another execution, one-cent overpayment and invalid discount. Real interest, external automatic debit and refund are not implemented. An integration suite must validate entries against accounting and exercise compensation for operations that cannot execute.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Measure the interval customers cannot see

Metric	Definition	Window and segment
Execution time	p95 from eligible funding to application	Daily; payment type
Funded but pending	Funded unexecuted operations / eligible	Daily; age and reason
Discount mismatch	Cents differing from approved plan	Continuous; contract version
Explained cancellation	Cases with reason and destination / canceled	Weekly; channel

Baselines are unknown. Funding needs a financial timestamp rather than the moment a button was tapped. Quotes expiring before payment must also be distinguished from delayed execution after confirmed eligibility. This segmentation prevents one indicator from combining behaviors with different causes and owners.

The pilot retains approved contractual calculation and compares previous tracking against explicit plan and receipt. Expansion into new execution requires independent financial validation. I would stop for duplicate application, incorrect discount, funds without traceable destination or misleading cancellation communication. Faster completion cannot compensate for a cent applied to the wrong contract.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Value rework, not removed discounts

Assumed scenario	Potential/year	Operations/year	Year 1 after cost
Conservative	R$ 14,400	R$ 30,000	R$ -165,600
Base	R$ 67,200	R$ 30,000	R$ -112,800
Ambitious	R$ 192,000	R$ 30,000	R$ 12,000

Annual assumptions: 30,000, 70,000 or 120,000 eligible early repayments; absolute incident reduction of 0.3%, 0.6% or 1%. I assume R$ 140 rework and R$ 20 incremental support per incident, totaling R$ 160. That yields 90, 420 or 1,200 avoided occurrences. Implementation: R$ 150,000; operations: R$ 30,000/year, both assumed.

In the base case, 420 × 160 = R$ 67,200. Year one is R$ 67,200 - 30,000 - 150,000 = -R$ 112,800. Break-even requires 187,500 early repayments equivalent to the base assumption. The ambitious scenario yields R$ 12,000 modeled net capacity, but incidence and effect must be measured. The plan should start small without evidence of sufficient scale.

Calculation boundaries

Do not infer loss from installment counts in the complaint. A correct contractual discount is not avoidable cost. Exclude incidents and contacts already counted in case 09; released capacity is not guaranteed cash.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

A roadmap starting with accounting

Proposed phase	Owner and deliverable	Gate
Days 0-15	Credit + accounting: rules	Calculation examples approved
Days 16-30	Staff: quote and execution	Atomic expiry/revision checks
Days 31-60	Operations: shadow reconciliation	Received funds have traceable destination
Days 61-90	EM + product: limited pilot	Correct discounts; fewer repeat contacts

The 90-day horizon is proposed planning. The critical dependency is agreement on contractual rules, rounding and application priority. Mobile teams should not infer those conditions from support messages. Approved examples must include insufficiency, expiry, overpayment and contract changes between offer and execution.

The EM brings financing, statements, automatic debit, support and accounting into one delivery. Staff defines technical boundaries and compensation strategy. Every phase ends with evidence that communicated state matches financial state. If calculation is unvalidated, the next step is closing that gap rather than accelerating rollout.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Operate a quote that did not complete

The canary starts with shadow quotes and receipts compared against the financial source. The team follows funded-but-unexecuted operations segmented by expiry, insufficiency and stale revision. Every category has an owner and next action; a generic processing status does not replace a resolution plan.

Alerts cover age, amount and discount discrepancy. Credit owns rules; financial platform owns application; operations owns resolution; product owns communication. The runbook forbids applying payment under another plan without authorization. Any refund or compensation remains linked to original funding and quote.

Rollback pauses new offers or the new experience while preserving receipts and received funds. Exercises include crash after commit, expiry during payment, concurrent automatic debit and an old response. Recovery begins with financial authority rather than the screen. Every pending operation must end in valid application or an approved, traceable destination for the money.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

My contribution: make the promise executable

As Staff, I would define what accepting a quote means and which authority guarantees execution. I would separate pure calculation, consent, funding and application, with identity and revision at every boundary. I would reject a fix that only changes visual status while the financial operation remains unexecuted.

As Engineering Manager, I would coordinate the teams sharing this journey and expose dependencies in the plan. Support needs the accepted plan and exception reason; accounting needs reconcilable entries; mobile needs stable explanatory semantics. Delivery finishes only when all three work together.

The central tradeoff is offer validity versus friction. Keeping a quote indefinitely can undermine financial rules; unexplained expiry can frustrate customers. I would choose transparent revalidation with renewed consent for material changes. Cost and opportunity would be reassessed in the pilot without claiming gains or authority I do not possess.

PROPOSTA EXTERNA / EXTERNAL PROPOSAL

Handoff: received funds have a destination

- Run python3 labs/case_14.py: 9 local tests and synthetic values.
- Review quote, receive and execute; observe funding separately from application.
- Inject a pre-commit crash, reopen and retry; principal can settle only once.
- Before production: contractual pricing, trusted clock, authorization, settlement, ledger and controlled refunds.

Reviewers should inspect both the happy path and an operation that cannot complete. The laboratory preserves received money in an explicit pending state; it does not claim that this is enough for a real experience. Resolution needs policy, a validated deadline and communication as well as a transaction.

Proceed only when plan, execution and receipt reconcile, the customer understands the required total and operations can handle expiry and insufficiency. If automation is unsupported, retain traceable human assistance. The desired result is a financial promise that can be explained and fulfilled, and an exception that does not disappear between teams.

Executable case lab on GitHub

EXTERNAL PROPOSAL · CASE 15

The incident and its outcome

15

Actual crop of the public source, captured 23 October 2026. The text below includes the company reply and reported outcome.

On April 28, 2026, complaint 247218209 described missing Money Boxes and unavailable withdrawals, without a useful support estimate. In its April 30 response, Nubank acknowledged a temporary disruption affecting some customers, stated that assets remained protected, and reported a successful withdrawal from the account on April 28 at 20:46:01. The public status is Answered, with no final customer evaluation. No amount, verified individual duration or evidence of lost assets is disclosed.

The engineering question is how to preserve trustworthy guidance while a dependency is degraded. Olhar Digital reproduced an incomplete screen during this episode and reported recovery; that does not identify an internal cause or quantify affected customers. The following design is my proposal, not a reconstruction of the incident.

Reclame AQUI · 247218209
Olhar Digital · 28/04/2026

EXTERNAL PROPOSAL · CASE 15

Reading a balance does not authorize withdrawal

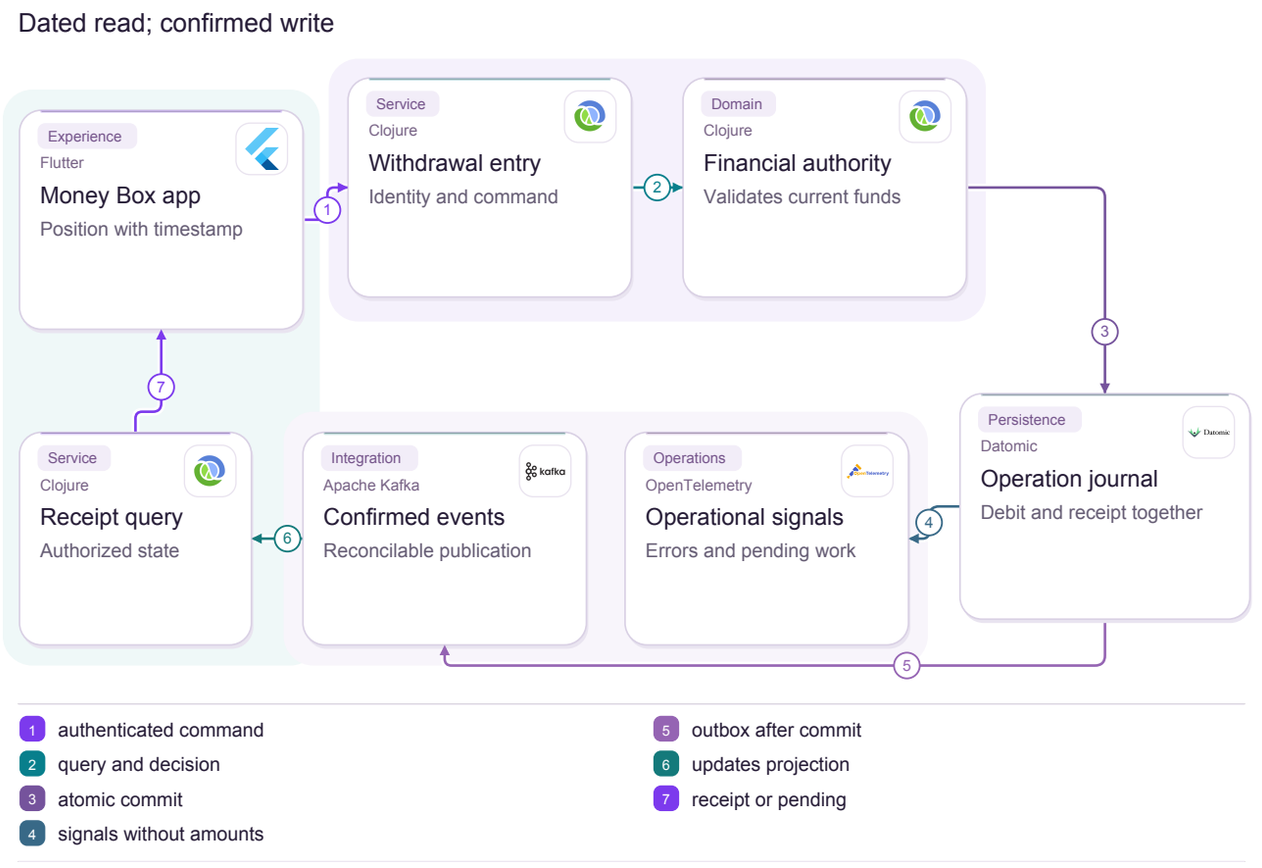
I would separate three questions often compressed into one screen: does the investment exist, which position was last verified, and can a withdrawal execute now? A failed query does not answer the other two negatively. Displaying zero because a position did not load confuses availability with ownership; allowing withdrawal against stale information creates another risk. The degraded state must explain the distinction without exposing internal details.

My falsifiable hypothesis is that an auxiliary read could still provide a dated position when withdrawal authority is unavailable. I would inspect existing contracts, shared dependencies, latency budgets and correlated failure history. If reads and writes share an unavoidable outage, an isolated cache may help guidance but cannot restore liquidity. That distinction determines the investment and belongs in the product review.

Investigation would start with synthetic attempts and authorized traces: query started, result unknown, command accepted and receipt confirmed. I would not request customer statements to reproduce the problem. The journey ends when the person receives financial confirmation or an honest explanation of unavailability, with a tracking path and a next update.

EXTERNAL PROPOSAL · CASE 15

The receipt closes the loop



Proposed external architecture; technologies illustrate choices, not confirmed internal systems.

Only the financial authority may commit a debit. The journal must persist the result and command identity in one transaction. The proposed outbox publishes already confirmed facts; if transport stops, the customer queries the receipt through an authorized path. Observability receives reduced signals, without amounts or account identifiers in labels.

The dated interface position remains advisory and never crosses this boundary as authorization. The lab represents only local commitment and receipt replay. Custody, settlement, partial outages and service-to-service authorization still require their own contracts and distributed tests.

EXTERNAL PROPOSAL · CASE 15

One attempt, one commitment

PYTHON / ORIGINAL PROPOSAL

```python
    def withdraw(self, actor, key, cents, snapshot, now, ledger_online=True):
        with self.lock:
            for value in (actor, key, snapshot.owner):
                identifier(value)
            if type(now) is not int or now < 0:
                raise ValueError('invalid clock')
            if actor != self.owner or snapshot.owner != actor:
                raise PermissionError('account scope')
            if not key or type(cents) is not int or cents <= 0:
                raise ValueError('invalid command')
            command = (actor, cents)
            if key in self.receipts:
                prior_command, receipt = self.receipts[key]
                if prior_command != command:
                    raise ValueError('idempotency conflict')
                return receipt
            if not ledger_online:
                raise RuntimeError('authority unavailable')
            if not 0 <= now - snapshot.observed_at <= 30:
                raise ValueError('snapshot expired')
            if snapshot.version != self.version:
                raise ValueError('refresh required')
            if cents > self.balance:
                raise ValueError('insufficient funds')
            self.balance -= cents
            self.version += 1
            receipt = (key, cents, self.version)
            self.receipts[key] = (command, receipt)
            return receipt
```

Literal excerpt from the executed lab; imports and initialization are in the full file.

The operation requires a matching owner, positive integer amount and a recent snapshot of the same version. The 30-second clock is an educational assumption, not a Nubank deadline. The advisory balance does not replace authority: with the ledger unavailable, no debit occurs. After success, replaying the same key retrieves the receipt even if the snapshot expired; this prevents a lost response becoming a second withdrawal.

The lock covers one process. In production I would replace this section with a conditional transaction in the durable authority, preserving the same observable semantics. The app does not manufacture trusted snapshots; the test object represents a service-issued read. A receipt also requires access control before deduplication.

Executable case lab on GitHub

EXTERNAL PROPOSAL · CASE 15

Dart for an understandable state

DART / ORIGINAL PROPOSAL

```dart
enum WithdrawalState { loading, datedPosition, unavailable, confirmed }

String guidance(WithdrawalState state, String observedAt) {
  return switch (state) {
    WithdrawalState.loading => 'Loading current position',
    WithdrawalState.datedPosition => 'Last verified: $observedAt',
    WithdrawalState.unavailable => 'Withdrawal unavailable; track status',
    WithdrawalState.confirmed => 'Withdrawal confirmed; view receipt',
  };
}

bool mayRequestFreshAuthorization(WithdrawalState state) =>
    state == WithdrawalState.datedPosition;
```

Illustrative Dart example, unexecuted; demonstration copy requires localization.

The interface may offer another authorization attempt without claiming that withdrawal is available. I would present verified amount, timestamp and unavailability as separate facts; screen readers would receive the same distinction. Confirmed state comes from a receipt, not a completed animation. Text and color should reinforce meaning, never carry the explanation alone.

The proposed Clojure service owns the decision while Flutter presents the contract. I would assess existing components before introducing any technology. Deployment starts with unknown state and a queryable receipt, because a new screen without a consistent financial contract merely moves confusion elsewhere in the journey.

EXTERNAL PROPOSAL · CASE 15

Identities that survive a retry

Field	Rule	Owner
operation_id	Unique per intent	Financial service
position_version	Monotonic per position	Custody
observed_at	Verifiable instant	Read source
receipt_id	Only after commit	Ledger

JSON / ORIGINAL PROPOSAL

```json
{
  "schema": "withdrawal-status.v1",
  "operation_id": "synthetic-operation",
  "state": "UNKNOWN",
  "position_version": 42,
  "observed_at": "2026-01-01T12:00:00Z",
  "receipt_id": null,
  "next_action": "QUERY_RECEIPT"
}
```

Proposed contract, synthetic data; schema unexecuted.

UNKNOWN is neither rejection nor confirmation. A consumer can query the same identifier without creating a new financial intent. A changed amount requires a new key; reusing the previous one must fail explicitly. I would define receipt retention against existing obligations and use a much shorter cache lifetime, avoiding deletion of durable evidence when a projection expires.

The owner is resolved from authenticated identity, not supplied as authority in JSON. Logs retain an opaque reference, error class and version. Reconciliation compares committed commands and published events, with restricted permission to investigate discrepancies and without copying balances into operational dashboards.

EXTERNAL PROPOSAL · CASE 15

Failures that must not become debits

PYTHON / ORIGINAL PROPOSAL

```python
    def test_cache_does_not_authorize(self):
        with self.assertRaises(RuntimeError):
            self.w.withdraw(self.s.owner, 'a', 1000, self.s, 101, False)
        self.assertEqual(self.w.balance, 10_000)
        self.assertEqual(self.w.receipts, {})

    def test_retry_after_commit_returns_receipt(self):
        first = self.w.withdraw(self.s.owner, 'a', 1000, self.s, 101)
        retry = self.w.withdraw(self.s.owner, 'a', 1000, self.s, 900, False)
        self.assertEqual(first, retry)
        self.assertEqual(self.w.balance, 9000)

    def test_parallel_commands_cannot_spend_same_version(self):
        def attempt(key):
            try:
                return self.w.withdraw(self.s.owner, key, 7000, self.s, 101)
            except ValueError:
                return None
        with ThreadPoolExecutor(max_workers=2) as pool:
            results = list(pool.map(attempt, ['a', 'b']))
        self.assertEqual(sum(x is not None for x in results), 1)
        self.assertEqual(self.w.balance, 3000)
```

Literal test excerpt; all 13 lab tests passed locally.

All thirteen tests passed locally. Beyond these excerpts, the suite covers a wrong owner, conflicting key, future timestamp, exact validity boundary, insufficient funds and invalid monetary types. The race uses two intents against one version; only one may commit funds. The replay test distinguishes an already completed operation from a new request during unavailability.

In the integrated environment I would add a crash after commit but before response, a delayed outbox and replica recovery. The criterion would be one financial movement and a recoverable receipt. This suite measures neither availability, network latency nor custody capacity, and a green run does not authorize real money.

EXTERNAL PROPOSAL · CASE 15

Metrics that include unknown state

The business baseline is unknown. I would measure confirmed completion: receipts within 60 seconds divided by valid intents, daily, segmented by app version and product. The 60-second window is proposed for discussion, not a published SLA. An intent without a receipt remains unknown until reconciliation and is reported separately, preventing it from disappearing from the denominator.

The second metric is useful guidance: degraded attempts showing a dated position and next step divided by degraded attempts, hourly. The third is repeat contact within 24 hours per affected intent, using pseudonymous linkage and limited retention. The fourth is receipt conflict per committed command; any confirmed financial discrepancy stops the pilot. Sampled traces explain failures without high-cardinality labels.

The experiment compares equivalent journeys under simulated failures, then a small cohort, without deliberately causing financial unavailability. The initial proposed target is reduced contact without increasing unknown outcomes or delaying confirmation. If messaging reduces contact through abandonment or prevents assisted access, the result does not count as success.

EXTERNAL PROPOSAL · CASE 15

Value without treating balances as revenue

The public case does not disclose an amount. The proposed financial evaluation considers support capacity, not Money Box assets. Exclusively for planning, I assume 20,000 affected intents per month, eight minutes per avoided contact, BRL 60 per loaded hour, BRL 5,000 monthly operating cost and BRL 120,000 implementation cost. None of these inputs is Nubank data.

Assumed scenario	Absolute reduction	Hours/month	Net value/month
Conservative	2 pp	53.33	-BRL 1,800
Base	5 pp	133.33	BRL 3,000
Ambitious	10 pp	266.67	BRL 11,000

Monthly formula: intents × absolute contact reduction × minutes ÷ 60 × hourly cost, less operations. Operating break-even requires a 3.125-percentage-point reduction, equivalent to 625 avoided contacts. Recovering implementation would require 40 months in the base scenario and 10.91 in the ambitious scenario, only if freed capacity converts fully into cash; the conservative scenario has no payback under these assumptions.

Finance would need to validate marginal cost, seasonality and how freed hours are used. Redeployed capacity may improve support without reducing expense. I would not add this benefit to other support cases without eliminating overlapping contacts, nor estimate asset loss or recovered revenue from the complaint.

EXTERNAL PROPOSAL · CASE 15

A 90-day plan with verifiable exits

Days 0-15: the Investments Staff owner maps withdrawal authority with Custody and SRE; Product defines messages for stale positions and unknown outcomes. The deliverable is a contract with failure examples and data classification. Exit requires agreement on who may declare completion, including external dependencies and states without a trustworthy estimate.

Days 16-30: backend implements idempotent lookup and a journal in a test environment while Mobile validates assisted reading and safe retries. QA injects response loss after commitment. Days 31-60: SRE runs state shadowing and reconciliation; expansion requires no discrepancy in the known dataset, complete telemetry and reviewed support documentation.

Days 61-90: the Engineering Manager coordinates a journey-level pilot, cost review and training. Investments accepts integrity; Product accepts comprehension; SRE accepts recovery. Ninety days is a proposed horizon, conditional on existing contracts. Lack of reliable transactional authority blocks pilot writes even if the visual experience is ready.

EXTERNAL PROPOSAL · CASE 15

Operating when a dependency fails

I would start in shadow mode, calculating the state that would be shown without changing financial authorization. The first cohort receives only queries and explanations; writes progress once receipt recovery has been exercised. A kill switch may remove the new presentation, but must never erase committed operations or turn receipts into generic errors.

The Investments on-call handles differences between journal and position; SRE handles availability and publication lag; Support receives a readable state and the agreed next update. One drill stops the projection while keeping authority available; another stops authority while preserving reads. Both must preserve the distinction between a known position and the current ability to withdraw.

I would stop expansion for an incompatible receipt, another owner's position, or unknown attempts rising beyond a limit agreed after baseline measurement. Reconciliation gives each pending operation an owner and deadline. A service-restored announcement would not substitute for individual confirmation that a command completed.

EXTERNAL PROPOSAL · CASE 15

My contribution: trust under failure

As Staff, I would document authority, freshness and idempotency decisions before choosing mechanisms. I would bring the team together to compare three alternatives: a global incident message alone, an advisory cache with receipts, and a withdrawal queue for later execution. My initial preference would be the second because it improves guidance while keeping authorization close to current state. A queue changes timing expectations and requires specific consent; it would not enter scope silently.

As Engineering Manager, I would negotiate capacity across Mobile, Investments, SRE and Support, preventing each team from shipping a different state. Weekly review would examine unknown operations, blocked dependencies and test learning, with documented decisions and explicit owners. The contribution I propose is reduced ambiguity between teams, not perfect availability.

I would present a reproducible test, a known limitation and an economic hypothesis to the review group one at a time. This proposal demonstrates my decision and coordination method; it does not claim involvement in the incident or privileged knowledge of Nubank architecture.

EXTERNAL PROPOSAL · CASE 15

Handoff: from lab to decision

Run python3 labs/case_15.py -v. Start at Withdrawals.withdraw and follow snapshot, receipts and the ownership check. Change the service from available to unavailable and observe that the balance does not change; then replay a completed operation with the clock advanced and verify that the receipt remains identical. These steps make the key boundary auditable within minutes.

The acceptance checklist includes: no debit without current authority; no duplicate execution on retry; no cross-owner receipt lookup; clearly dated stale positions; trackable unknown outcomes; and reconciliation of delayed publication. Integration still requires real authentication, transactional persistence, custody and settlement policies, trusted clocks, migration and disaster tests.

I would recommend moving only to an integrated prototype if the authority owner accepts this contract and failure tests are reproducible. If a dated query leads people to believe withdrawal is guaranteed, communication design returns to research. Delivery quality includes knowing when to stop an apparent improvement that creates an incorrect financial interpretation.

Executable case lab on GitHub

EXTERNAL PROPOSAL · CASE 16

A contradictory answer, a calendar

16

Actual crop of the public source, captured 23 October 2026. The text below includes the company reply and reported outcome.

Complaint 250210183, dated June 1, 2026: a Nu+ customer said she transferred investments and was told by chat that she was ineligible, without explanation. On June 9, Nubank confirmed the Treasury investment transfer and eligibility, explaining ten installments starting in June. Status: Answered, with no public confirmation of the first credit. The complaint precedes the first scheduled due date, June 10, so it does not establish overdue cashback.

The consulted rules define a window, net amount and maintenance conditions. The opportunity is to give the customer and Support the same time-aware, versioned explanation. We do not know the individual transferred amount or the original record consulted by the agent; this proposal does not attribute the disagreement to a particular engine.

Reclame AQUI · 250210183
Regulamento oficial da campanha

EXTERNAL PROPOSAL · CASE 16

Three questions before declaring an error

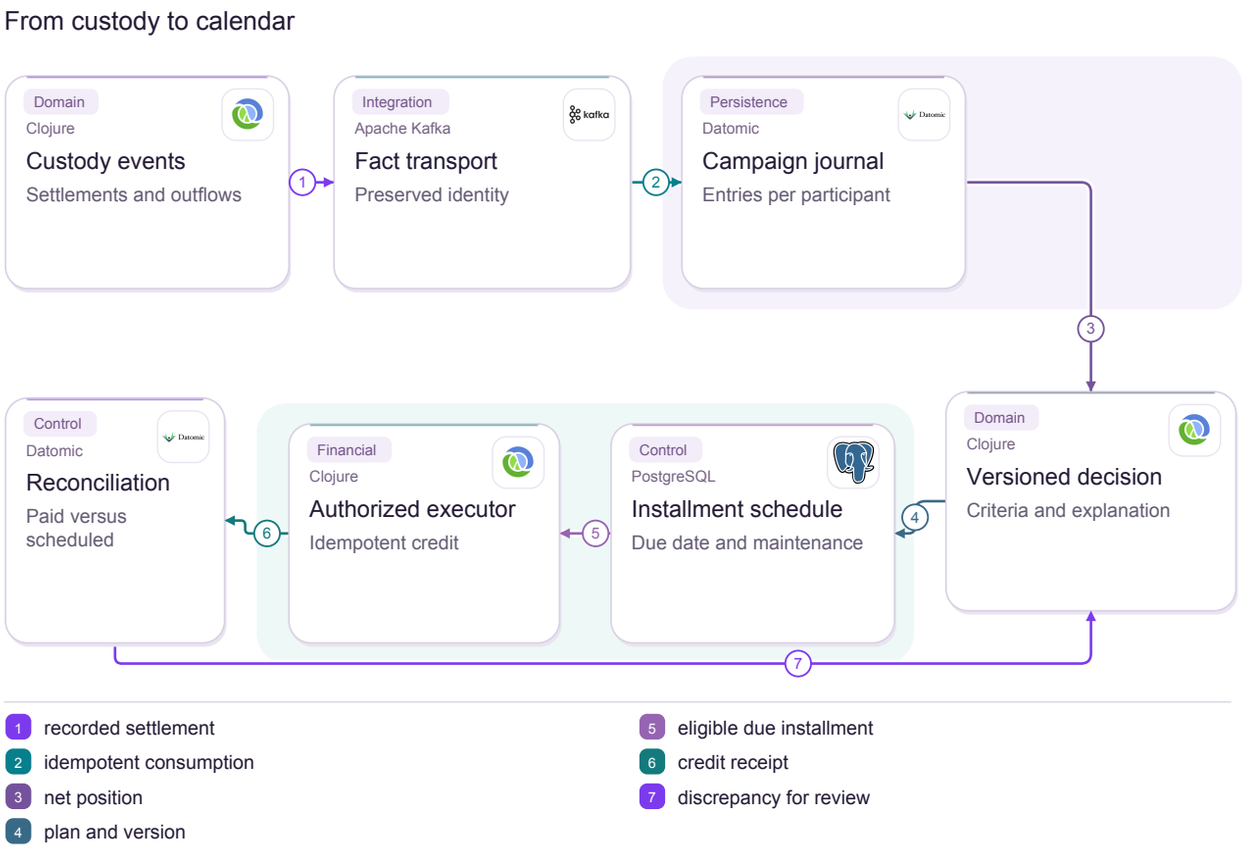
I would distinguish eligibility, due date and completed payment. Someone can be eligible in June, have an installment that is not yet due and have no posted credit. If these states become one flag, Support may interpret absent credit as exclusion from the campaign. The hypothesis to test is inconsistent meaning across channels, without assuming that the financial calculation was wrong.

Investigation would follow a synthetic example from custody completion to benefit. It would establish which date counts for the campaign, how outflows offset inflows and which rule revision was accepted. A transfer requested within the window but completed later may need a different answer; the system should show the event used and a review path without exposing third-party holdings.

I would interview Investment Operations and Support about exceptions before modeling automation. The aim is not to turn every complaint into a credit, but to provide a challengeable decision with reproducible facts. Channel disagreement should open a review of the same evidence rather than start another conversation without context.

EXTERNAL PROPOSAL · CASE 16

An explainable benefit projection



Proposed external architecture; technologies illustrate choices, not confirmed internal systems.

The source supplies custody facts; the decision component applies a campaign revision; the schedule separates due dates from payments. The executor produces credit only with authorization and current conditions. Reconciliation returns discrepancies for plan review without retroactively changing issued receipts. App and Support would query the same projection, omitted here to keep the main flow visible.

Kafka and Datomic are illustrative choices. An existing implementation may fulfill these responsibilities with other components. The lab represents simplified Nu+ rules only, not every asset, exception or actual maintenance criterion.

EXTERNAL PROPOSAL · CASE 16

Calendar before the complaint

PYTHON / ORIGINAL PROPOSAL

```python
    def schedule(self, actor, active_subscription):
        with self.lock:
            identifier(actor)
            if actor != self.owner:
                raise PermissionError('account scope')
            if active_subscription is not True:
                return {'state': 'INELIGIBLE', 'total_cents': 0, 'dues': ()}
            net = sum(e.signed_cents for e in self.events.values()
                      if date(2026, 3, 1) <= e.settled <= date(2026, 5, 29))
            total = min(max(net, 0) // 3_000_000 * 18_000, 180_000)
            dues = tuple(date(2026 + (5 + i) // 12, (5 + i) % 12 + 1, 10)
                         for i in range(10)) if total else ()
            return {'state': 'ELIGIBLE' if total else 'BELOW_THRESHOLD',
                    'total_cents': total, 'dues': dues}
    def pay(self, actor, installment, today, balance_maintained=True,
            active_subscription=True):
        with self.lock:
            plan = self.schedule(actor, active_subscription)
            if type(installment) is not int or not 0 <= installment < 10:
                raise ValueError('installment range')
            if installment in self.paid:
                return self.paid[installment]
            if type(today) is not date or not plan['dues']:
                raise ValueError('date or eligibility invalid')
            if today < plan['dues'][installment]:
                raise ValueError('not due')
            if balance_maintained is not True:
                raise ValueError('maintenance review required')
            if self.frozen_total not in (None, plan['total_cents']):
                raise ValueError('plan changed; reconcile')
            self.frozen_total = plan['total_cents']
            self.paid[installment] = self.frozen_total // 10
            return self.paid[installment]
```

Literal excerpt from the executed lab; imports and initialization are in the full file.

The calculation uses integer cents, net settled inflows within the window and ten due dates. The Nu+ example applies BRL 180 per BRL 30,000 block, capped at BRL 1,800, following the consulted rules. The complaint's individual amount remains unknown. No payment is allowed before its due date or without explicit subscription and maintenance checks.

After the first credit, a change in the calculated total blocks a new installment and requires reconciliation. Replaying the same installment returns its previous receipt without another effect. The model does not implement retroactive corrections or proportional benefit suspension: those policies must be agreed before replacing its conservative block.

Regulamento oficial

Executable case lab on GitHub

EXTERNAL PROPOSAL · CASE 16

One explanation shared by channels

CLOJURE / ORIGINAL PROPOSAL

```clojure
(defn explain-installment [{:keys [eligible due paid today]}]
  (cond
    paid {:state :paid :next-action :view-receipt}
    (not eligible) {:state :review :next-action :view-reasons}
    (.isBefore today due)
    {:state :scheduled :next-action :view-calendar :due due}
    :else {:state :due :next-action :track-payment}))

;; Inputs come from authorized, versioned domain services.
;; This function explains; it never posts a financial credit.
```

Illustrative Clojure, unexecuted; not Nubank's rules engine.

The explanation should show the reason and next action, not just a green or red color. Before the due date, the calendar explains absent credit. After the due date, a missing receipt moves into tracking. When eligibility is challenged, the review should receive the events and rule used without asking the person to retell the story.

I would deploy this function in a domain presentation layer consumed by the app and an authorized support console. Financial authority would remain separate. Copy would be localized and tested for not-yet-due, below-minimum and under-review scenarios. The interface must not infer fraud, noncompliance or customer error from incomplete data.

EXTERNAL PROPOSAL · CASE 16

Identifiable rule, event and installment

Record	Identity	Protection
Custody	event_id + owner	Authenticated source
Decision	rule_version + cutoff	Historical replay
Installment	campaign + account + index	Unique credit
Review	decision_id + reason	Restricted access

JSON / ORIGINAL PROPOSAL

```json
{
  "schema": "campaign-decision.v1",
  "rule_version": "nuplus-demo-v1",
  "eligibility": "ELIGIBLE",
  "installment": 0,
  "due_date": "2026-06-10",
  "payment_state": "NOT_YET_DUE",
  "receipt_id": null
}
```

Proposed contract; synthetic fixture, not a real API response.

Each decision records the factual cutoff. Late events are not silently backdated over paid installments; they trigger an identified review with impact analysis. Maintaining investments is a condition separate from subscription. The campaign owner defines the rule and Operations validates custody evidence; engineering ensures reproducible execution.

The rules available today may differ from historical communication. I would archive the applicable revision and its hash with appropriate legal access. This chapter restricts its example to Nu+, avoiding mixed Ultravioleta figures that differ between the response and public material versions.

EXTERNAL PROPOSAL · CASE 16

Test the date, not only the sum

PYTHON / ORIGINAL PROPOSAL

```python
    def test_complaint_date_is_before_first_due_date(self):
        plan = self.c.schedule('owner', True)
        self.assertEqual(plan['state'], 'ELIGIBLE')
        self.assertEqual(plan['dues'][0], date(2026, 6, 10))
        with self.assertRaises(ValueError):
            self.c.pay('owner', 0, date(2026, 6, 1))
        self.assertEqual(self.c.paid, {})

    def test_outflow_changes_net_eligibility(self):
        out = Transfer('out', 'owner', date(2026, 5, 29), -1)
        self.c.record(out, 'custody')
        self.assertEqual(self.c.schedule('owner', True)['total_cents'], 0)

    def test_concurrent_payment_is_once_per_installment(self):
        def pay(_):
            return self.c.pay('owner', 0, date(2026, 6, 10))
        with ThreadPoolExecutor(max_workers=4) as pool:
            results = list(pool.map(pay, range(20)))
        self.assertEqual(set(results), {1800})
        self.assertEqual(sum(self.c.paid.values()), 1800)
```

Literal test excerpt; all 15 lab tests passed locally.

The executed suite also tests custody origin, conflicting identity, window, cap, negative net balance, inactive subscription and maintenance. The complaint date does not authorize the first installment. Event reads and writes share the lock; a concurrent test exercises that contract. A late change after the first credit requires plan review instead of silently recalculating payments.

The lab does not demonstrate full compliance with the rules or verify any real person’s eligibility. In the integrated environment I would test custody replay, subscription changes near the due date and executor response loss. Finance and Investments must approve exception fixtures before the suite can act as a delivery gate.

EXTERNAL PROPOSAL · CASE 16

Observable agreement and punctuality

All baselines are unknown. The primary metric would be channel agreement: simultaneous queries returning the same decision and version divided by comparable pairs, daily and by campaign. Another would be explanation completeness: decisions with a reason, rule and next action divided by served decisions. Collection occurs in the domain without recording holdings in app analytics.

Timeliness requires eligible due installments as the denominator: reconciled credits within an agreed operating window divided by those installments, per due date. Repeat contact within seven days per challenged decision measures effort, segmented by reason. Duplicate payment per installment is the financial guardrail: any confirmed occurrence pauses expansion and opens reconciliation even if screen agreement is excellent.

I would first propose a comprehension test using synthetic dates and cross-channel shadowing. The initial aim would be to remove contradictions explained by the calendar, not increase eligibility. Expansion depends on financial reconciliation and a sufficient sample of challenged cases; absence of complaints does not prove understanding.

EXTERNAL PROPOSAL · CASE 16

A promised benefit is an obligation, not profit

BRL 180 per BRL 30,000 describes a public Nu+ rule, not this complaint's individual benefit or bank revenue. To evaluate the proposal, I assume 12,000 participants per month, 1.5 repeat contacts avoided per removed dispute, a BRL 18 loaded cost per contact, BRL 4,000 monthly operations and BRL 90,000 implementation. All are hypothetical inputs.

Scenario	Absolute reduction	Avoided contacts	Net value/month
Conservative	1 pp	180	-BRL 760
Base	3 pp	540	BRL 5,720
Ambitious	6 pp	1,080	BRL 15,440

The formula is participants × absolute dispute reduction × contacts per case × cost per contact, less operations. Monthly break-even requires a 1.235-percentage-point dispute reduction. If all capacity value became cash, implementation would pay back in 15.73 months in the base scenario and 5.83 in the ambitious scenario; there is no conservative payback. A finite campaign may end sooner, requiring assessment of reuse without counting it in advance.

I would not treat denied bonuses as savings or transferred assets as profit. The decision depends on actual marginal cost, remaining campaign duration and demonstrated ability to reuse the service for other offers.

EXTERNAL PROPOSAL · CASE 16

Deliver a decision that can be reviewed

Days 0-15: Investment Product and the campaign owner assemble applicable rules, boundary examples and the payment calendar. Operations confirms custody settlement semantics; Legal validates communication without delegating contractual interpretation to code. Exit is an approved scenario matrix including a request before the due date and a transfer settled outside the window.

Days 16-30: backend publishes versioned decisions in a sandbox; Support tests the same result in its console. Days 31-60: the Staff engineer monitors shadowing against existing decisions, classifies discrepancies and requires resolution before touching payments. Data validates denominators; Security reviews access to holdings and documents.

Days 61-90: the EM coordinates a limited offer, first in explanations and then the executor, with Finance accepting reconciled installments. This is a proposed horizon, not a Nubank commitment. Without a reliable rule revision or custody proof, the decision remains under human review; a project deadline does not justify inferring eligibility.

EXTERNAL PROPOSAL · CASE 16

Rule changes without rewriting history

A new version starts in shadow mode over fixtures and authorized facts. Differences are classified as correction, policy change or late data. The campaign owner approves the category; engineering does not apply a new rule to previous credits for technical convenience. A rollback removes the new decision component but preserves decisions, receipts and open cases.

Investments on-call receives custody discrepancies, Finance receives duplicate installments or incompatible amounts, and Support receives incomplete explanations. I would exercise out-of-order events, two executors on a due date and subscription changes between query and payment. Review must record which condition was valid at the financial decision time.

Operations also needs a human calendar: holidays, closing and exception owners. The lab uses calendar dates and does not claim to resolve that policy. If the operating contract does not exist, showing a precise estimate creates a false promise; the interface should present an approved deadline and its scope.

EXTERNAL PROPOSAL · CASE 16

My contribution: reduce interpretations

As Staff, I would lead the design of a reproducible decision record with a rule, factual cutoff and shared explanation. I would reject a solution based solely on copying the rules into chat: that helps reference but does not explain which participant facts were considered. I would also reject automatic bonuses when channels disagree; contradiction requires review, not improvised financial authorization.

As Engineering Manager, I would organize one backlog across Product, Custody, Finance and Support while separating rule and execution owners. I would measure time blocked on policy decisions and remove dependencies before pushing implementation speed. The central artifact would be a small set of scenarios accepted by everyone, traceable to the contract.

This is a contribution I propose bringing to the role: translating commercial ambiguity into testable contracts without pretending engineering alone decides customer entitlements. I do not claim to have implemented campaigns at Nubank or to know its internal operations.

EXTERNAL PROPOSAL · CASE 16

A guided benefit review

Run python3 labs/case_16.py -v. Start with Campaign.record, move to schedule and compare June 1 with June 10 in pay. Insert a one-cent outflow into the threshold example and observe the decision; then try processing the same installment in parallel. Finally change the position after the first payment and confirm that the next installment requires review.

Acceptance includes authenticated origin, event deduplication, a defined inclusive window, no credit before due date, checked subscription and maintenance, unique installment identity and changed-plan review. Persistence, complete asset rules, real event signatures, banking payments, an operating calendar and correction policies remain missing. The Clojure and JSON excerpts are unexecuted proposals.

I would proceed if the campaign and Finance owners accept the matrix and can reproduce a challenged decision. I would stop if the team can explain only the current balance without reconstructing the applied rule. The ability to answer why a decision occurred is part of the product, especially when an offer has a deadline and installment payments.

Executable case lab on GitHub

EXTERNAL PROPOSAL · CASE 17

Confirmed redemption, disputed understanding

17

Actual crop of the public source, captured 23 October 2026. The text below includes the company reply and reported outcome.

In complaint 245086475, dated April 3, 2026, the customer said she understood points and cashback as separate benefits and requested reversal after redeeming cash. On April 6, Nubank stated that the balance was unified and the choice confirmed and irreversible, with 19,071.36 points redeemed for BRL 577.91 on April 1, 2026. It denied an operational failure. Status: Answered, without final evaluation. The page proves neither unauthorized activity nor the interface used.

My proposal addresses interpretation risk before confirmation while respecting the public irreversibility constraint. It does not promise to undo the transaction or treat the received amount as proven loss. The technical challenge is to bind an understandable explanation to the balance and quote that will be consumed.

Reclame AQUI · 245086475
Termos Ultravioleta

EXTERNAL PROPOSAL · CASE 17

Two units do not create two assets

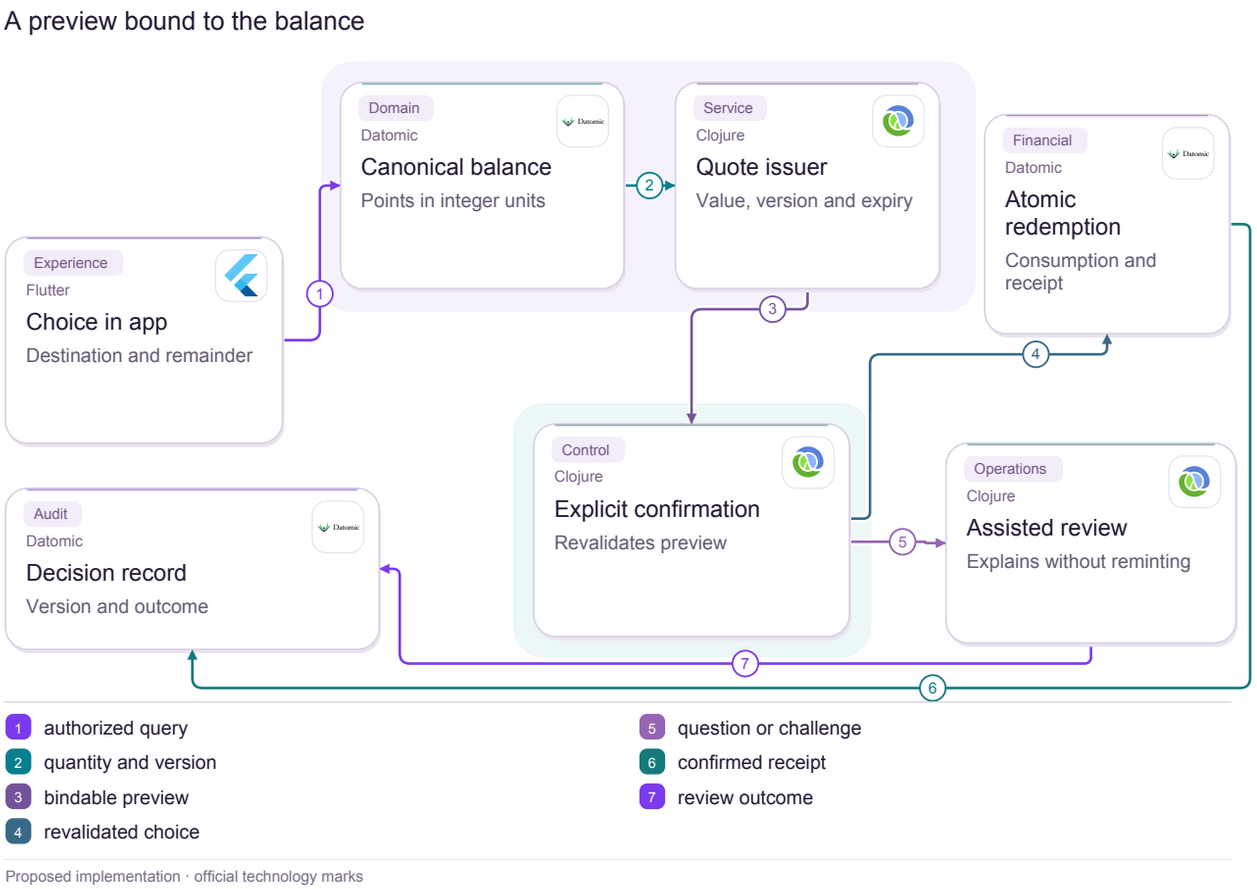
The public fact is a disagreement between expectation and the company's stated rule. The interface cause is unknown: without the historical screen, we cannot assert labels, hierarchy or missing confirmation. My hypothesis is that presenting alternatives as two independent quantities can create an additive mental model. It is testable with prototypes and tasks without attributing that design to the actual app version.

I would start by asking participants to explain what will remain after each choice before allowing a simulated redemption. The correct outcome is understanding exclusivity, destination, quantity consumed and remaining balance, not merely locating a button. Research would include screen-reader users and different familiarity levels with points, avoiding a sample limited to experienced loyalty-program users.

In the domain I would look for a canonical unit and an identified quote. Switching presentation between cash and points should not duplicate a balance, and changing the quote during confirmation must not silently change the commitment. Comprehension evidence informs the experience; only current financial authorization permits benefit consumption.

EXTERNAL PROPOSAL · CASE 17

The preview is part of the commitment



Proposed external architecture; technologies illustrate choices, not confirmed internal systems.

Confirmation receives a quote from an authorized service, including balance version and expiry. Redemption consumes the canonical unit and persists a receipt atomically. Assisted review may explain or route an exception under policy; it receives no connection that automatically recreates points. The record combines decision and outcome under restricted access.

The lab deliberately executes only the cash alternative. Miles would require another contract and partner confirmation, not an improvised multiplication. Logos represent proposed responsibility choices without claiming where Nubank holds its benefit balance.

EXTERNAL PROPOSAL · CASE 17

Consume exactly the accepted preview

PYTHON / ORIGINAL PROPOSAL

```python
    def redeem(self, actor, quote_id, choice, key, now):
        with self.lock:
            for value in (actor, quote_id, key):
                identifier(value)
            if type(now) is not int or now < 0:
                raise ValueError('invalid clock')
            if actor != self.owner:
                raise PermissionError('account scope')
            if choice != 'cash' or not key:
                raise ValueError('explicit supported choice required')
            command = (quote_id, choice)
            if key in self.receipts:
                old, receipt = self.receipts[key]
                if old != command:
                    raise ValueError('idempotency conflict')
                return receipt
            quote = self.quotes[quote_id]
            if now >= quote.expires_at:
                raise ValueError('quote expired')
            if quote.balance_version != self.version:
                raise ValueError('balance changed; preview again')
            if quote.points_minor > self.points:
                raise ValueError('insufficient points')
            self.points -= quote.points_minor
            self.cash += quote.cash_cents
            self.version += 1
            receipt = (quote_id, quote.points_minor, quote.cash_cents)
            self.receipts[key] = (command, receipt)
            return receipt
```

Literal excerpt from the executed lab; imports and initialization are in the full file.

Points use hundredths of a point and reais use cents, both integers. The fixture reproduces only the quantities published by the company, without inferring a universal commercial rate. The quote service is a simulated trusted dependency; the client cannot supply an arbitrary conversion value. An expired quote, changed version or unsupported choice produces no financial effect.

The same key returns the same receipt after commitment. Another key for the same quote encounters an incompatible balance version and fails. This is an execution property, not proof of informed consent: understanding the choice requires product and accessibility evaluation as well as backend atomicity.

Executable case lab on GitHub

EXTERNAL PROPOSAL · CASE 17

Dart: consequence before the button

DART / ORIGINAL PROPOSAL

```dart
class RedemptionPreview {
  final int pointsConsumedMinor;
  final int cashReceivedCents;
  final int pointsRemainingMinor;
  final String quoteId;
  final DateTime expiresAt;
  const RedemptionPreview(this.pointsConsumedMinor,
      this.cashReceivedCents, this.pointsRemainingMinor,
      this.quoteId, this.expiresAt);
}

String consequenceLabel(RedemptionPreview preview) =>
    "This choice consumes the same benefit balance. "
    "Review the amount and remaining points before confirming.";
```

Illustrative Dart, unexecuted; monetary formatting and localization omitted.

I would organize the preview around one main sentence and three quantities: what leaves, what arrives and what remains. The unselected alternative appears as a use of the same balance, without presenting itself as a second asset. Confirmation must keep this information available when a keyboard, enlarged text or screen reader changes navigation.

A repeated generic warning can become noise. I would compare a concrete consequence with an additional confirmation box, measuring comprehension and errors rather than clicks. When the quote expires, the interface returns to preview and requires fresh confirmation; it does not update numbers behind an already accepted button. Irreversibility must be communicated without promising an operational exception.

EXTERNAL PROPOSAL · CASE 17

Units the contract cannot confuse

Field	Unit	Invariant
points_minor	1/100 point	Nonnegative integer
cash_cents	1/100 BRL	Authorized quote
balance_version	Revision	Revalidated at commit
quote_id	Opaque reference	Single financial use

JSON / ORIGINAL PROPOSAL

```json
{
  "schema": "redemption-preview.v1",
  "quote_id": "synthetic-quote",
  "choice": "cash",
  "points_consumed_minor": 1907136,
  "cash_received_cents": 57791,
  "points_remaining_minor": 0,
  "balance_version": 1,
  "irreversible_after_confirmation": true
}
```

Explanatory fixture of reported quantities; proposed contract unexecuted.

Point representation must state unit and precision at every boundary. A decimal without a unit can be interpreted as reais by a generic consumer. I would version the contract and reject unknown write fields while maintaining planned read compatibility. The quote includes an authorized destination and owner in the protected layer, not analytics.

The confirmation record should be sufficient to review which preview was presented without capturing an entire screen or travel data. Retention and access follow a policy agreed with Privacy. Reconstructing a decision does not require storing every gesture; it requires quote identity, copy revision and the financial result.

EXTERNAL PROPOSAL · CASE 17

The test separates UX from atomicity

PYTHON / ORIGINAL PROPOSAL

```python
    def test_same_balance_cannot_be_spent_twice(self):
        self.b.redeem('owner', 'q1', 'cash', 'a', 100)
        with self.assertRaises(ValueError):
            self.b.redeem('owner', 'q1', 'cash', 'b', 101)
        self.assertEqual(self.b.points, 0)
        self.assertEqual(self.b.cash, 57_791)

    def test_retry_preserves_original_receipt(self):
        first = self.b.redeem('owner', 'q1', 'cash', 'a', 100)
        retry = self.b.redeem('owner', 'q1', 'cash', 'a', 900)
        self.assertEqual(first, retry)
        self.assertEqual(self.b.cash, 57_791)

    def test_expired_preview_has_no_financial_effect(self):
        with self.assertRaises(ValueError):
            self.b.redeem('owner', 'q1', 'cash', 'a', 200)
        self.assertEqual(self.b.points, 1_907_136)
        self.assertEqual(self.b.cash, 0)
        self.assertEqual(self.b.receipts, {})
```

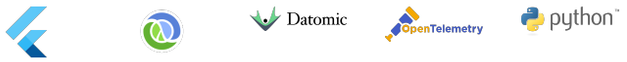
Literal test excerpt; all 13 lab tests passed locally.

All thirteen tests passed. The suite includes concurrent contention, an unauthorized quote attempt, a wrong owner, conflicting key and an invalid clock, including NaN. These tests demonstrate no effect for rejected inputs and single consumption in the model. They do not demonstrate that a person understood the copy or that the quote reproduces real loyalty rules.

I would combine the suite with assisted tasks: choose cash, explain the remaining balance and recognize that miles are no longer available for that same quantity. Integration tests would cover confirmation after a balance change and response loss after commit. Review must accept technical evidence and comprehension separately; neither replaces the other.

EXTERNAL PROPOSAL · CASE 17

Measure understanding, preserve choice



Baseline is unknown. The proposed primary metric is correct comprehension: participants who explain remaining balance and exclusivity without help divided by participants reaching preview, per research round and access mode. A second measures attempts to use an already consumed alternative per completed redemption over seven days, with minimal collection and no automatic inference of regret.

Reversal-related contacts per thousand redemptions, split by preview version and modality, measure observed effort. Expiry followed by abandonment per expired preview measures friction cost. Duplicate consumption per quote is the financial guardrail: any confirmed occurrence suspends the pilot. All windows and segments would be fixed before comparison, with sufficient accessibility samples.

I would initially target improved comprehension without reducing legitimate completion through unnecessary barriers. Research comes first, followed by a controlled experiment on presentation only. Rates, entitlements and reversibility are not randomized. Fewer contacts without better comprehension may mean people stopped seeking help and do not justify expansion.

EXTERNAL PROPOSAL · CASE 17

Cost of understanding, not point value

BRL 577.91 is the credit reported in the individual response; it is neither proven loss nor proposal benefit. I assume 40,000 monthly redemptions, 1% with a material misunderstanding, BRL 45 handling per case and BRL 100 approved discretionary remediation in 15% of them. This yields BRL 60 per potentially avoidable case. I also assume BRL 4,500 monthly operations and BRL 150,000 implementation; no parameter is observed at Nubank.

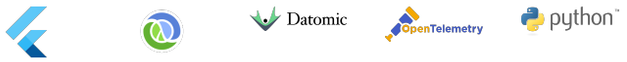
Scenario	Assumed prevention	Cases/month	Net potential
Conservative	25%	100	BRL 1,500
Base	50%	200	BRL 7,500
Ambitious	75%	300	BRL 13,500

Monthly break-even is 75 avoided cases, or 18.75% of the assumed 400. Hypothetical investment recovery is 100, 20 and 11.11 months respectively, if those costs are genuinely avoidable. The remediation component assumes neither an entitlement nor an existing practice. Finance must separate variable cost from redeployed capacity; the latter does not automatically become cash.

The model should be rejected if research shows the issue is rare or the new flow does not change understanding. I do not monetize remaining points or extrapolate the complaint to the Ultravioleta population.

EXTERNAL PROPOSAL · CASE 17

From language hypothesis to integration



Days 0-15: Design Research runs tasks with alternative presentations; Benefits Product validates rules and destinations. The Staff engineer records units, quotes, versions and terminal states. Exit requires reproducing the comprehension problem in research without assuming the complaint proves a general cause.

Days 16-30: Mobile prototypes consequences and accessibility, backend creates bound quotes in a sandbox and Security reviews scope. Days 31-60: QA exercises expiry, concurrency and response loss; Privacy approves the minimal record. The technical exit is single consumption with a recoverable receipt; the product exit is demonstrated task comprehension.

Days 61-90: the EM coordinates a presentation cohort with Support monitoring and Finance review. Any benefit-contract change requires its own governance before rollout. This proposed horizon does not promise a product change within 90 days; it makes dependencies that could prevent the decision explicit.

EXTERNAL PROPOSAL · CASE 17

Fail before consumption, explain afterward

Rollout begins with preview in a controlled environment and quote shadowing. The financial path changes only after the service demonstrates atomic revalidation. I would simulate expiry while someone reads, a balance change in another session and response loss after credit. In every case, the app must show an honest state and prevent confirmation on different terms.

Benefits owns unit reconciliation; SRE monitors quote expiry and failures; Support receives the authorized preview record. Rollback may restore the previous presentation but must preserve lookup of receipts issued under the new version. No flag should make a confirmed transaction appear cancellable.

Expansion to miles would be a separate integration project with partner-confirmation semantics and pending-state handling. I would keep that boundary explicit so a copy improvement does not carry a new reversal promise or an unapproved economic change.

EXTERNAL PROPOSAL · CASE 17

My contribution: precision and empathy

As Staff, I would make unit, quote and version explicit domain concepts. One important decision would be to keep the canonical balance independent of display forms. I would reject maintaining two balances adjusted by separate events to represent one benefit because that increases reconciliation and permits divergent interpretations. The specific storage choice would depend on the existing system and migration cost.

As Engineering Manager, I would bring Research, Mobile, Benefits and Support together around the same task instead of distributing screen requirements alone. I would protect time for accessibility participants and failure drills, which often disappear when deadlines tighten. Delivery review would require one comprehension demonstration and one integrity demonstration.

The proposal combines respect for published rules with attention to the disputed experience. I do not need to assume a systemic defect to propose a verifiable improvement. This is the judgment I would bring to the role: separating what evidence supports, what code guarantees and what still requires validation with people.

EXTERNAL PROPOSAL · CASE 17

Handoff: review the full consequence

Run python3 labs/case_17.py -v. Inspect Quote and Benefits.issue before redeem: quote authority is part of the guarantee. Reproduce the fixture and confirm point reduction and a single credit. Advance the clock to expiry, change the replay key and try another owner; each path must preserve the stated boundary.

Acceptance requires explicit units, an authority-issued quote, revalidated balance, understood exclusivity, one effect and a receipt accessible after response loss. Financial integration, an actual expiry policy, signed or authenticated quote transport, persistence and copy review remain missing. The local test certifies neither legal compliance nor airline-partner redemption.

I would proceed if the preview improves understanding without inventing contractual flexibility. I would stop if the solution depends on promising a reversal the domain does not offer. The final reviewer demonstration should make it possible to explain aloud, without jargon, the balance before, the choice and the balance afterward.

Executable case lab on GitHub

EXTERNAL PROPOSAL · CASE 18

Cancellation did not resolve portability

18

Actual crop of the public source, captured 23 October 2026. The text below includes the company reply and reported outcome.

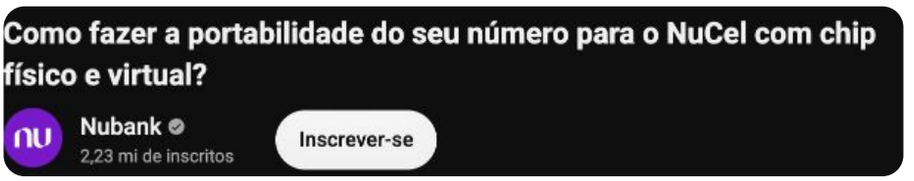
Complaint 254143071, dated July 17, 2026: the customer said she answered Vivo's SMS, could not complete portability to NuCel and was charged. On July 24, Nubank stated that the donor had no recorded confirmation, and reported cancellation of the provisional line and adjustment of possible charges. The customer disputed the explanation, marked Not resolved and gave a zero score. No amount or independent evidence attributes the failure to either carrier.

The proposal focuses on confirmation traceability and responsibility for the next step. It does not confuse administrative closure with completion of the original intent. Cancellation may be a valid customer choice; it is not equivalent to porting the number, and the displayed state should preserve that distinction.

Reclame AQUI · 254143071

EXTERNAL PROPOSAL · CASE 18

What each confirmation actually proves



Official Nubank tutorial, 10 October 2025. Product context; this is not a recording of the complaint.

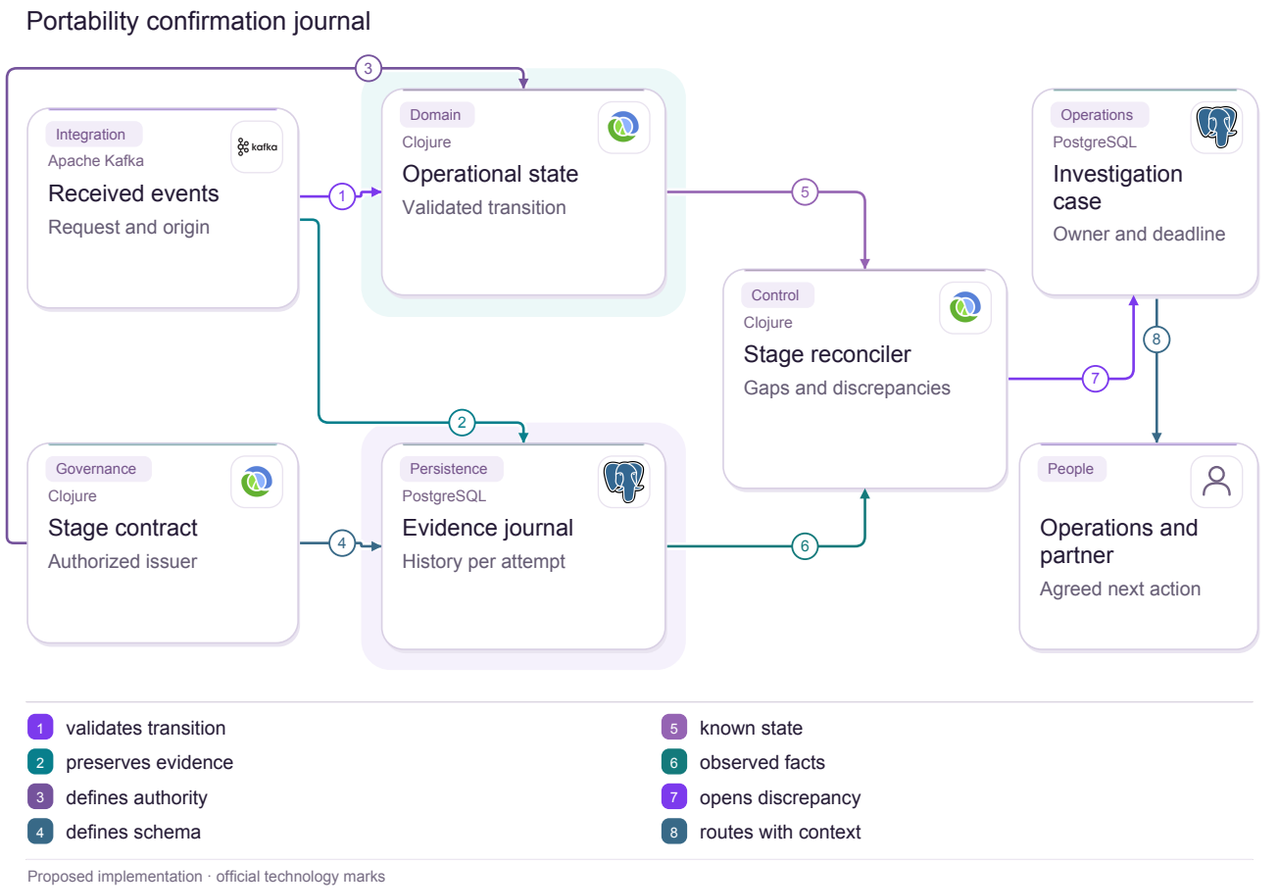
Answering a message, the carrier recording the response and the number becoming active are distinct events. My falsifiable hypothesis is that a correlated journal could locate the boundary lacking evidence and reduce circular handoffs. I would inspect shared identifiers, callback contracts, clocks and retry policy with the partner without assuming that a specific webhook caused this episode.

Nubank's official video about portability with physical and virtual SIMs, published October 10, 2025, provides product context. It is neither a record of this incident nor proof that confirmation reached the donor. I would use it to align self-service instructions with observable states and validate whether someone can identify who should act next.

Nubank · Portabilidade NuCel · vídeo oficial
Guia oficial de portabilidade

EXTERNAL PROPOSAL · CASE 18

Compare occurrence and confirmation



Proposed external architecture; technologies illustrate choices, not confirmed internal systems.

The contract defines who may declare each stage. Operational state and journal receive the same event, but reconciliation compares what was applied with what was observed. A gap creates an investigation with an owner, reference and next deadline; it does not invent confirmation. The people node has no technology logo because it represents coordination rather than software.

The proposed journal does not include SMS content. Donor authorization and recipient activation remain distinct. Provisional billing requires its own integration and policy; this design assumes no refund entitlement and changes no telecommunications contract.

EXTERNAL PROPOSAL · CASE 18

Origin is part of the event

PYTHON / ORIGINAL PROPOSAL

```python
    def apply(self, event, authenticated_source):
        with self.lock:
            for value in (event.event_id, event.request_id, event.kind, event.source):
                identifier(value)
            if event.request_id != self.request_id:
                raise ValueError('request scope')
            if event.source != authenticated_source:
                raise PermissionError('untrusted source')
            if self.SOURCES.get(event.kind) != event.source:
                raise PermissionError('event authority')
            if type(event.sequence) is not int or event.sequence < 1:
                raise ValueError('invalid sequence')
            if event.event_id in self.seen:
                if self.seen[event.event_id] != event:
                    raise ValueError('event identity conflict')
                return self.state
            if event.sequence <= self.sequences.get(event.source, 0):
                raise ValueError('out-of-order; reconcile')
            allowed = {'sms_sent': ('REQUESTED',),
                       'donor_authorized': ('WAIT_DONOR',),
                       'activated': ('AUTHORIZED',),
                       'cancelled': ('REQUESTED', 'WAIT_DONOR', 'AUTHORIZED')}
            if self.state not in allowed[event.kind]:
                raise ValueError('invalid transition; reconcile')
            self.state = {'sms_sent': 'WAIT_DONOR', 'donor_authorized':
                          'AUTHORIZED', 'activated': 'ACTIVE',
                          'cancelled': 'CANCELLED'}[event.kind]
            self.seen[event.event_id] = event
            self.sequences[event.source] = event.sequence
            self.journal.append(event)
            return self.state
```

Literal excerpt from the executed lab; imports and initialization are in the full file.

The model accepts donor authorization only from its corresponding issuer, and activation only after that authorization. A report of an answered SMS can open an investigation but cannot simulate intercompany confirmation. Repeated events are deduplicated; reusing identity with different content fails. Sequence is per issuer, avoiding comparison of different systems' numbers as if they formed a global clock.

Cancellation is terminal for that attempt. A late activation cannot resurrect the request: it requires reconciliation. The source parameter represents an already authenticated adapter; a string sent by the app would not be production authentication. Cryptographic validation, holder binding and durable storage still need to exist outside this lab.

Executable case lab on GitHub

EXTERNAL PROPOSAL · CASE 18

Dart: show dependency without blame

DART / ORIGINAL PROPOSAL

```dart
enum PortState { requested, waitingDonor, authorized, active, cancelled }

String nextStep(PortState state) => switch (state) {
  PortState.requested => "Follow the verified confirmation instructions",
  PortState.waitingDonor => "Confirmation pending; track investigation",
  PortState.authorized => "Authorization received; activation pending",
  PortState.active => "Portability completed",
  PortState.cancelled => "This attempt was cancelled; review billing status",
};

// Billing status is a separate authorized projection.
// A customer tap does not produce donor authorization.
```

Illustrative Dart, unexecuted; final copy requires research and localization.

I would avoid showing only pending. People need to distinguish an instruction they can perform, confirmation dependent on another organization and an ongoing investigation. The interface offers an update and help channel using the same identifier, without telling them to repeat a cancelled process or switch carriers blindly. Copy explains the known state without assigning unproven blame.

Billing and portability appear as separate dimensions. A financial adjustment under review cannot become an active line, and a cancelled line does not prove a completed credit. Mobile consumes authorized, accessible projections; the domain decides transitions. I would require joint NuCel Operations and Support review before publishing labels.

EXTERNAL PROPOSAL · CASE 18

Attempts do not share confirmation

Field	Meaning	Owner
request_id	One attempt	Recipient
issuer	Who asserts the fact	Authenticated adapter
sequence	Order per issuer	Issuing partner
billing_case_id	Separate review	Financial operations

JSON / ORIGINAL PROPOSAL

```json
{
  "schema": "portability-event.v1",
  "event_id": "synthetic-donor-confirmation",
  "request_id": "synthetic-attempt-2",
  "issuer": "donor",
  "kind": "donor_authorized",
  "sequence": 7,
  "billing_effect": "NONE"
}
```

Illustrative contract, unexecuted; no phone number or SMS content.

A second attempt receives a new identity. Evidence from the first may help Support but does not automatically authorize the second. I would retain occurrence and receipt timestamps to investigate delay without ordering all partners by one clock. The lab rejects an early event for review; a real system may park it in a durable queue and replay it after filling the gap.

The customer-facing tracking record uses an opaque reference and stage, without a full phone number. The identity-resolution table has stricter access and defined retention. The integration owner reviews schema breaks with the partner; Support should neither interpret raw payloads nor turn an absent event into an accusation against the customer.

EXTERNAL PROPOSAL · CASE 18

Reproduce the intercompany gap

PYTHON / ORIGINAL PROPOSAL

```python
    def test_user_sms_claim_cannot_authorize_donor(self):
        event = Event('d', 'request-a', 'donor_authorized', 'owner', 1)
        with self.assertRaises(PermissionError):
            self.p.apply(event, 'owner')
        self.assertEqual(self.p.state, 'WAIT_DONOR')

    def test_late_activation_cannot_revive_cancelled_request(self):
        cancel = Event('c', 'request-a', 'cancelled', 'owner', 1)
        self.p.apply(cancel, 'owner')
        late = Event('a', 'request-a', 'activated', 'recipient', 2)
        with self.assertRaises(ValueError):
            self.p.apply(late, 'recipient')
        self.assertEqual(self.p.state, 'CANCELLED')

    def test_early_activation_requires_reconciliation(self):
        event = Event('a', 'request-a', 'activated', 'recipient', 2)
        with self.assertRaises(ValueError):
            self.p.apply(event, 'recipient')
        self.assertNotIn('a', self.p.seen)
        self.assertEqual(self.p.state, 'WAIT_DONOR')
```

Literal test excerpt; all 12 lab tests passed locally.

All twelve tests passed. The suite covers the complete path, duplicates, a different request, spoofed issuer, stale sequence and replay after the missing authorization arrives. The central scenario is adverse: activation received before the necessary evidence cannot become success. Cancellation retains its meaning even when delayed events arrive.

In the integrated environment I would test signed callbacks, partner unavailability and diverging clocks. I would also simulate Support querying two similar attempts to verify isolation. These tests prove neither SMS delivery, network quality nor regulatory compliance. Those require partners, devices and telecommunications specialists with their own test plan and evidence.

EXTERNAL PROPOSAL · CASE 18

Measure completion and limbo separately

Baseline is unknown. Completion should count effectively active numbers divided by started eligible attempts, in weekly cohorts segmented by donor and SIM type. Cancellations appear as a separate outcome. The second metric is confirmation wait time: distribution from request to authorized event, including open cases as censored rather than treating them as instantaneous or excluding them.

The third measures investigations with a next step and owner divided by open discrepancies, daily. The fourth measures disputed billing without a decision per cancelled attempt over seven days; it does not presume improper billing. The fifth is transitions without authorized evidence per applied transition, with a proposed zero-tolerance expansion guardrail. Sensitive data stays out of labels and aggregate dashboards.

I would compare equivalent cohorts after observing seasonality and carrier mix. The improvement must increase actual completion or reduce limbo without pushing cancellations. If the average improves because difficult cases are closed earlier, the pilot has failed the original intent and requires review.

EXTERNAL PROPOSAL · CASE 18

Value of a genuinely activated line

The complaint publishes neither charge nor refund amounts. The proposed scenario uses a hypothetical cohort of 5,000 eligible requests, six active months per incremental line and BRL 12 contribution per active month after assumed variable costs. I include BRL 5,000 pilot operations and BRL 20,000 incremental implementation using existing infrastructure. This budget does not estimate a complete telecom rebuild; every input is assumed.

Scenario	Absolute uplift	Extra lines	Six-month net
Conservative	2 pp	100	-BRL 17,800
Base	5 pp	250	-BRL 7,000
Ambitious	8 pp	400	BRL 3,800

The formula is requests × absolute activation uplift × active months × monthly contribution, less BRL 25,000 costs. Break-even requires approximately 347.22 equivalent lines, or at least 348 whole lines, 6.96 percentage points for 348 lines. The conservative and base scenarios do not pay for the pilot; that is decision information, not a flaw to conceal.

Revenue is not margin, and activation without retention does not realize six months of contribution. Finance would need to validate churn, subsidy, Support and partner cost. I use neither banking ARPAC as telecom margin nor disputed charges as benefit.

EXTERNAL PROPOSAL · CASE 18

Dependencies that determine the schedule

Days 0-15: NuCel Operations and the donor-integration owner catalog events, authorizations and terminal states. The Staff engineer identifies correlation gaps; Product defines completion from the customer's perspective. Exit requires a stage-level responsibility contract and an investigation path both organizations can execute.

Days 16-30: backend builds a journal and reconciler in a sandbox, Mobile tests messages and Finance separates billing tracking. Days 31-60: QA injects missing, duplicate and delayed events with the partner; Support rehearses a second attempt without losing context. Exit is locating the pending boundary without exposing a full number or SMS content.

Days 61-90: the EM coordinates a pilot limited by operational capacity and reviews investigation cost. Expansion depends on partner acceptance, reconciliation and actual completion, not closed tickets. The schedule is conditional; if no verifiable donor response exists, no team should pretend a screen alone has solved the problem.

EXTERNAL PROPOSAL · CASE 18

Operate discrepancies without losing the customer

I would start the journal in shadow mode without changing transitions or billing. Reconciliation would surface cases to a small team with a queue limit and next-update deadline. Only after validation would the new state reach customers. A sudden increase in discrepancies may indicate a broken contract rather than a real increase in portability failures.

The integration owner receives missing-confirmation and unexpected-sequence alerts; Operations decides the donor handoff; Finance tracks authorized adjustments separately. The main drill is activation after cancellation: a line cannot be considered desired merely because a callback arrived. Resolution must verify the actual state on both sides.

Rollback disables new automated decisions while keeping the journal for investigation. It neither deletes evidence nor reopens cancelled requests. The customer receives an explanation of what is known, what is missing and the next update, without a deadline promise the partner has not accepted.

EXTERNAL PROPOSAL · CASE 18

My contribution: coordinate boundaries

As Staff, I would separate authorization evidence, operational state and support interpretation. I would reject advancing portability solely on a claim that the SMS was answered because that crosses donor authority. I would also reject treating every timeout as final failure: delay requires investigation and may be reconciled. Explicit states prevent implementation convenience becoming a product rule.

As Engineering Manager, I would organize work with an owner per contract and next action, including the partner in the dependency plan. Review would locate missing evidence and who can obtain it rather than assign blame. Protecting that discipline lets the team respond to customers without creating pressure to manufacture confirmation.

I would bring a definition of done that includes support continuity and billing reconciliation. I claim no internal NuCel experience; I demonstrate how I would investigate a distributed journey where one party’s technical success can still leave the customer’s intent unfinished.

EXTERNAL PROPOSAL · CASE 18

Handoff: show who may assert what

Run python3 labs/case_18.py -v. Follow Portability.apply and the SOURCES table. Send activation before authorization, confirm no progress, add the missing evidence and replay the event. Then cancel another attempt and send a late activation. The contrast demonstrates why arrival order, authority and current intent must be assessed separately.

The checklist requires isolation between attempts, authenticated issuers, deduplication, unexpected-order reconciliation, respected cancellation and independent financial state. Real callbacks, signatures, persistence, partner investigation, consents and device testing remain missing. The lab includes neither billing nor refunds and cannot certify real number portability.

I would recommend proceeding when the organizations can resolve a synthetic discrepancy using a shared reference and response deadline. I would stop if the solution depends on hiding absent confirmation or redefining cancellation as success. The reviewer should be able to point to each diagram edge and identify which contract authorizes the next state.

Executable case lab on GitHub

EXTERNAL PROPOSAL · CASE 19

A limitation, an acknowledged resolution

19

Actual crop of the public source, captured 23 October 2026. The text below includes the company reply and reported outcome.

Complaint 243002037, dated March 11, 2026: the customer said he converted approximately BRL 50,000 to euros for an external transfer and alleged BRL 1,600 in taxes and fees upon discovering the limitation. On March 18, Nubank explained purchases and withdrawals, without external transfers in that product, and reported a private solution. On March 21, the customer marked Resolved, scored service 10 and continued the relationship. Final resolution terms and amounts were not published.

The proposal identifies purpose before creating a costly conversion. It does not suggest the complaint remains open or turn an expectation into proof of a promised feature. The technological integration mentioned in the complaint does not mean every capability of another product is present.

Reclame AQUI · 243002037

EXTERNAL PROPOSAL · CASE 19

Capability is product-specific

A euro balance answers which currency someone holds, not which actions they can perform. The design hypothesis is that asking purpose before quotation exposes mismatches early: card spending, withdrawal and sending to others are different tasks. I would test whether the question is understandable without requiring knowledge of infrastructure or the underlying partner.

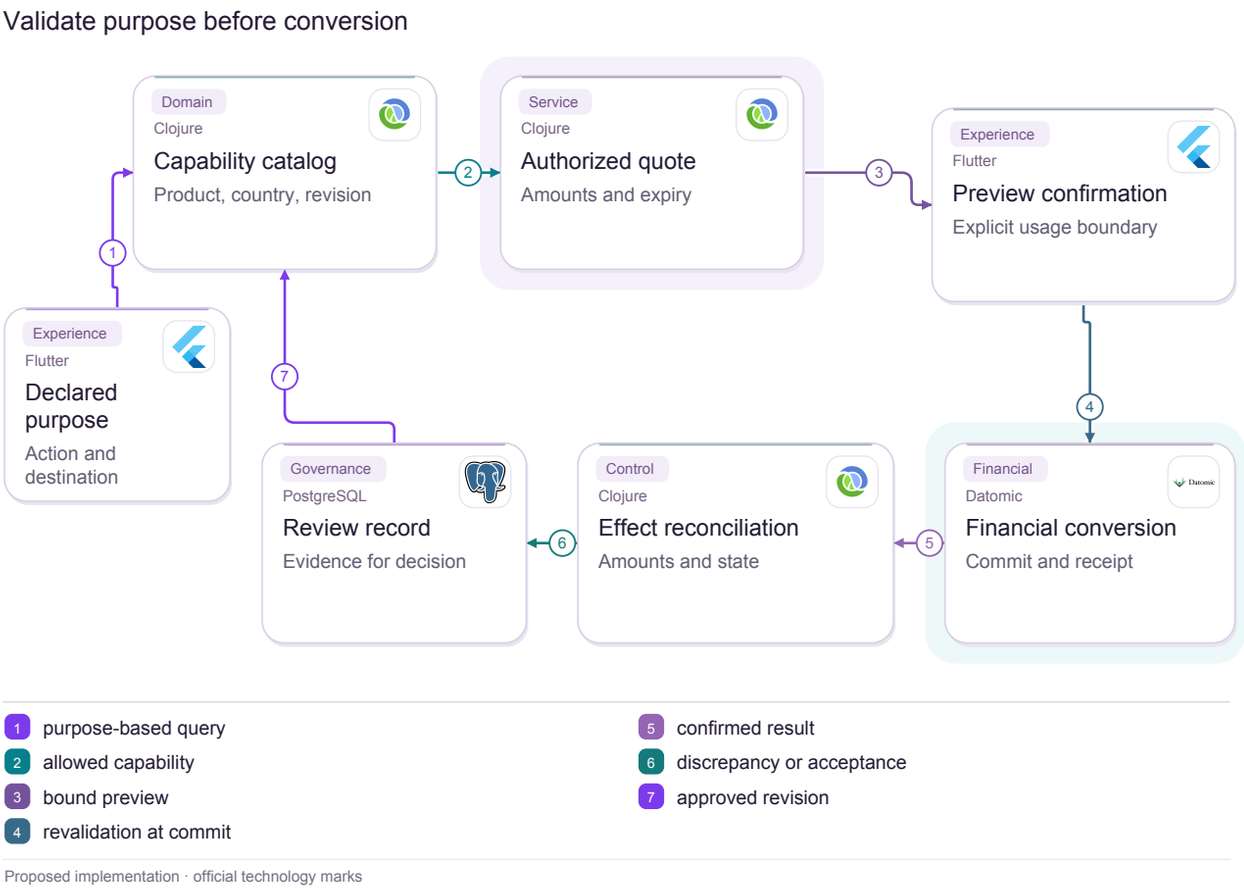
Investigation would map capabilities by product, currency, destination and applicable version. The reported facts are the unmet expectation and acknowledged resolution; we do not know the historical screen or exact contracting sequence. I would therefore not attribute the cause to missing copy. A prototype must demonstrate that the new sequence prevents incompatible conversion without blocking legitimate uses.

The official international-transfer article provides scope context but does not replace historical review of an individual contract. I would keep a versioned catalog and decision provenance rather than present current capability as proof of what was available on another date.

Nubank · Transferência internacional

EXTERNAL PROPOSAL · CASE 19

Capability travels with the quote



Proposed external architecture; technologies illustrate choices, not confirmed internal systems.

The catalog permits or rejects purpose before issuing a quote. The preview carries the considered revision, but commit checks current policy again: a capability removed during reading invalidates execution. Reconciliation validates the financial effect; the record may support an approved catalog revision without a complaint automatically changing capabilities.

The design does not create international remittances. It prevents a remittance intent from proceeding as conversion in another product. The lab models only one synthetic card-spending, currency and country combination; supporting new combinations requires separate decisions and integrations.

EXTERNAL PROPOSAL · CASE 19

A quote does not outlive policy

PYTHON / ORIGINAL PROPOSAL

```python
    def commit(self, actor, quote_id, key, now):
        with self.lock:
            for value in (actor, quote_id, key):
                identifier(value)
            if type(now) is not int or now < 0:
                raise ValueError('invalid clock')
            if actor != self.owner:
                raise PermissionError('account scope')
            if not key:
                raise ValueError('idempotency key required')
            if key in self.receipts:
                if self.receipts[key][0] != quote_id:
                    raise ValueError('idempotency conflict')
                return self.receipts[key]
            quote = self.quotes[quote_id]
            capability = (quote.purpose, quote.currency, quote.country)
            if quote.capability_version != self.version:
                raise ValueError('capability changed; start again')
            if capability not in self.allowed or now >= quote.expires_at:
                raise ValueError('not executable')
            if quote.debit_cents > self.balance:
                raise ValueError('insufficient funds')
            if any(r[0] == quote_id for r in self.receipts.values()):
                raise ValueError('quote already consumed')
            self.balance -= quote.debit_cents
            self.foreign[quote.currency] = self.foreign.get(quote.currency, 0) + \
                quote.foreign_minor
            receipt = (quote_id, quote.debit_cents, quote.foreign_minor)
            self.receipts[key] = receipt
            return receipt
```

Literal excerpt from the executed lab; imports and initialization are in the full file.

The quote binds purpose, currency, country, amount and capability version. The write checks owner, validity and balance, consuming the quote once. Replaying the same key retrieves the committed receipt even after policy changes; a new execution encounters an incompatible revision and stops. Expiry does not trigger automatic reverse conversion: nothing was debited before commit.

Test values are synthetic and represent neither a historical exchange rate nor taxation. The catalog is local and issuer authorization is simulated. In production, a durable financial authority must preserve atomicity across debit, foreign balance and receipt, alongside approved pricing and product rules.

Executable case lab on GitHub

EXTERNAL PROPOSAL · CASE 19

An explicit capability contract

TYPESCRIPT / ORIGINAL PROPOSAL

```typescript
type Purpose = "card_spend" | "cash_withdrawal" | "external_transfer";

type Capability = {
  product: "global-demo";
  purpose: Purpose;
  currency: "EUR" | "USD";
  country: string;
  revision: number;
  allowed: boolean;
  explanationCode: string;
};

function canRequestQuote(c: Capability): boolean {
  return c.allowed && c.revision > 0;
}
// The backend revalidates capability at financial commit.
```

Illustrative TypeScript, unexecuted; types do not replace runtime validation.

The contract makes rejection specific: this product does not support this purpose under the queried conditions. The interface can explain the limitation before showing a price, without suggesting that someone convert first and find out later. I would avoid a huge on-device catalog; the service responds only to the current intent with a revision and publishable reason.

Product owns capability; Pricing owns quotation; financial authority owns the effect. These boundaries allow a feature to change without rewriting FX calculations. The service must treat missing responses as unknown, not permission. Compile-time typing helps development, but external data requires its own validation and authentication.

EXTERNAL PROPOSAL · CASE 19

Retain intent, minimize exposure

Record	Minimum fields	Use
Capability	purpose + revision	Explain decision
Quote	amounts + expiry	Revalidate preview
Commitment	quote_id + receipt	Prevent duplication
Review	reason + reference	Investigate exception

JSON / ORIGINAL PROPOSAL

```json
{
  "schema": "capability-check.v1",
  "product": "global-demo",
  "purpose": "external_transfer",
  "currency": "EUR",
  "country": "IE",
  "revision": 1,
  "allowed": false,
  "quote_id": null
}
```

Proposed synthetic contract; no conversion should originate from this rejection.

Purpose should support the decision without collecting unnecessary family history, travel details or documents. Country in the model is a technical capability dimension, not an inference of nationality. Privacy review defines retention and permitted use. A support reference allows rejection investigation without copying financial data into the ticket.

The contract evolves through explicit revisions. An incompatible change can invalidate an open preview and require new confirmation; it does not change an issued receipt. I would retain the version used to explain historical decisions without indefinitely freezing a permission the product no longer offers.

EXTERNAL PROPOSAL · CASE 19

Reject before charging

PYTHON / ORIGINAL PROPOSAL

```python
    def test_unsupported_transfer_never_debits(self):
        q = FXQuote('no', 'owner', 'external_transfer', 'EUR', 'IE',
                    10_000, 1500, 1, 200)
        with self.assertRaises(ValueError):
            self.w.issue(q, 'pricing')
        self.assertEqual(self.w.balance, 100_000)
        self.assertNotIn('no', self.w.quotes)

    def test_capability_change_invalidates_old_preview(self):
        self.w.update_capabilities(set(), 'product-policy')
        with self.assertRaises(ValueError):
            self.w.commit('owner', 'q', 'a', 100)
        self.assertEqual(self.w.foreign, {})

    def test_expiry_does_not_trigger_reverse_conversion(self):
        with self.assertRaises(ValueError):
            self.w.commit('owner', 'q', 'a', 200)
        self.assertEqual(self.w.balance, 100_000)
        self.assertEqual(self.w.receipts, {})
```

Literal test excerpt; all 14 lab tests passed locally.

All fourteen tests passed, including invalid clocks, concurrent replay, a second key for the same quote, wrong owner and unauthorized capability changes. The first test deliberately targets incompatible intent: rejection happens before quote issuance and does not modify balance. The expiry test proves absence of debit, not a later refund.

Integration would need to test policy changes between quote and commit across separate processes, partial ledger failure and receipt lookup after timeout. I would also assess whether screen readers identify the limitation before price. No local test calculates taxes, validates rates or asserts that the historical product should have offered external transfers.

EXTERNAL PROPOSAL · CASE 19

Success means a compatible purpose

Baseline is unknown. I would measure incompatibility discovered before FX: unsupported intents identified before quotation divided by unsupported intents confirmed through research or review, weekly. This denominator requires a reviewed sample; it cannot be inferred only from people clicking help. Another metric is limitation comprehension among exposed participants, segmented by access mode.

Conversion followed by a capability-related contact within seven days per completed conversion measures a friction signal without proving causality. Incorrect blocking of a supported purpose per reviewed intent measures catalog harm. Commit with an invalid capability version per commit is the technical guardrail. Collection uses reason codes, version and aggregate counts without recipient data.

The proposed experiment changes information order, not prices or entitlements. Expansion considers comprehension and false negatives alongside conversion, avoiding rewards for a flow that reduces problems simply by blocking everything. Any debit following capability rejection or data exposure stops the pilot.

EXTERNAL PROPOSAL · CASE 19

An avoided conversion also has a cost

The approximately BRL 50,000 converted and BRL 1,600 costs are individual allegations; the case was resolved without disclosing compensation. They are not recovered revenue. For planning I assume 10,000 monthly intents, 2% mismatched, BRL 30 handling and BRL 100 remediation in 20% of cases. I subtract BRL 3 forgone contribution per avoided conversion, BRL 3,000 monthly operations and BRL 80,000 implementation.

Scenario	Assumed prevention	Avoided cases	Net potential/month
Conservative	25%	50	-BRL 650
Base	50%	100	BRL 1,700
Ambitious	75%	150	BRL 4,050

Each avoided case is hypothetically worth BRL 30 + 20% × BRL 100 - BRL 3 = BRL 47 before operations. Break-even is 63.83 cases per month, or 31.915% of the assumed 200. Implementation recovery would take 47.06 months in the base scenario and 19.75 in the ambitious scenario if costs materialize as savings; it does not occur in the conservative scenario.

All business inputs are assumed. Tax is not bank margin; costs spared to customers are another dimension and are not added to company results. I would revisit the project if required prevention exceeds research evidence or integration requires materially greater investment.

EXTERNAL PROPOSAL · CASE 19

Owners of capability and price

Days 0-15: Global Product identifies capabilities and revisions with the partner; Research validates the purpose question. The Staff engineer documents the capability-versus-price boundary. Exit requires a small authorized catalog with negative examples, not a list inferred from marketing or another company's features.

Days 16-30: backend implements queries and preview in a sandbox; Mobile evaluates accessibility and localization. Days 31-60: QA exercises policy-change races, expiry and receipt lookup after timeout; Pricing and Finance accept revalidation. The gate requires rejection without debit while supported uses remain possible.

Days 61-90: the EM coordinates a presentation pilot and reviews false-blocking cost. Support receives consistent reasons and Product accepts comprehension metrics. The schedule is proposed and integration-dependent; actually adding international transfers is a different initiative with its own risks and contracts, not a shortcut in this plan.

EXTERNAL PROPOSAL · CASE 19

Change the catalog safely

The first stage runs the catalog in shadow mode and compares decisions with current policy. Differences need Product review rather than automatic adjustment to increase conversion. A new revision is published with identity, regression examples and an owner. If the catalog does not respond, the system reports an unavailable query and does not issue an authorized quote by assumption.

SRE monitors availability and revision age; Product owns false negatives and incorrect capabilities; Finance reconciles effects. I would exercise capability removal during confirmation and restoration of the previous version. Rollback must consider open quotes: compatibility policy determines which remain valid without reinterpreting committed receipts.

Incident communication distinguishes query failure from product unavailability. A potentially costly reverse conversion should not be offered as an automatic alternative. An assisted case queries authorized history and applies approved policy, keeping an individual resolution separate from a general refund promise.

EXTERNAL PROPOSAL · CASE 19

My contribution: model the intent

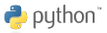
As Staff, I would bring purpose into the contract before pricing, with a revision bound through commit. I would reject a generic footer warning alone: it does not prevent an incompatible journey from advancing and makes comprehension difficult to verify. I would also avoid encoding capability rules in multiple clients because a product change would leave older versions making different promises.

As Engineering Manager, I would organize agreement across Product, the partner, Pricing, Mobile and Support, exposing early the decisions engineering cannot make alone. Priority would be a small journey slice with comprehension evidence before catalog expansion. I would track contract dependencies and research quality, not merely the number of screens delivered.

The resolved case also shows value in support acknowledged by the customer. My proposal does not erase that outcome: it tries to move explanation earlier so fewer people need a later resolution. That balance between learning and precision is the contribution I propose for the role.

EXTERNAL PROPOSAL · CASE 19

Handoff: follow a rejection without effect



Run python3 labs/case_19.py -v. Start at GlobalWallet.issue with external_transfer and observe that no quote is recorded. Then use the card_spend fixture, change capabilities before commit and confirm no movement. Finally complete a valid operation and replay the key after changing policy: the original receipt remains queryable without another effect.

Acceptance: explicit purpose, rejection before cost, revalidated version, a quote consumed once, valid clock and protected balance. Production still requires real pricing, applicable tax rules, transactional authority, authentication, reconciliation and a product-authorized catalog. The model is not an FX implementation and determines neither the rate nor the remedy appropriate to the public case.

I would proceed if someone understands the limitation before committing funds and the backend preserves it during an update race. I would stop if the catalog lacks an owner or depends on inferences about another product. Final review must show both an allowed path and a useful rejection without a debit.

Executable case lab on GitHub

EXTERNAL PROPOSAL · CASE 20

Refund completed, waiting time disputed

20

Actual crop of the public source, captured 23 October 2026. The text below includes the company reply and reported outcome.

Complaint 255046021, dated July 28, 2026: the customer said a May purchase from Cobasi paid through NuPay was charged without a processed order. On August 3, Nubank reported contacting the store, which found the payment and refunded it fully on July 30. On August 4, the customer marked Resolved but scored service zero for support and delay. The amount and technical cause are not public; no final loss is established.

The response also separates payment intermediation from the purchase-cancellation decision. My proposal preserves that authority while improving gap detection, tracking and reconciliation. It assumes neither a lost webhook nor unilateral refunds as the solution to every timeout.

Reclame AQUI · 255046021

EXTERNAL PROPOSAL · CASE 20

Two systems can disagree without lying

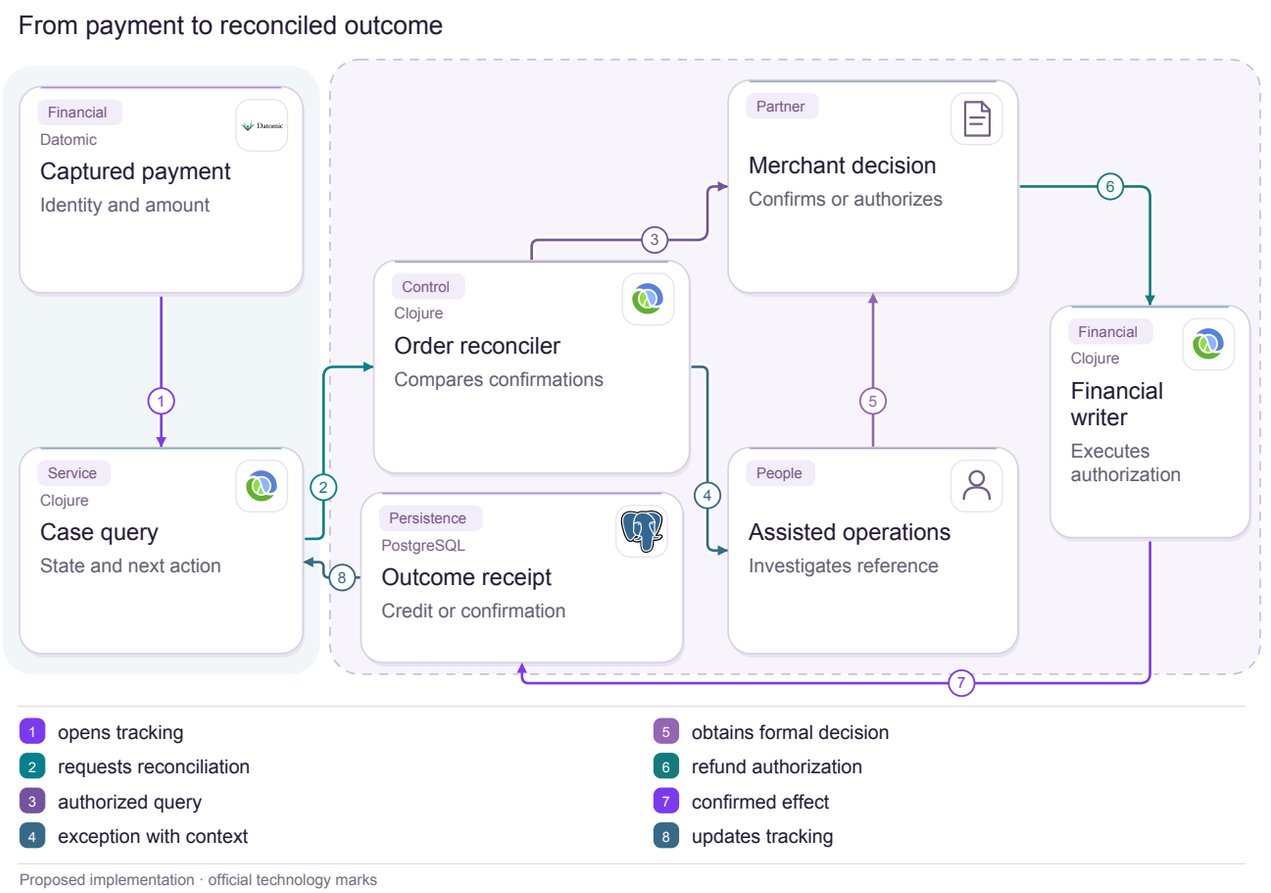
Captured payment and acknowledged order are different facts. A customer sees one purchase, but payment and fulfillment authorities may confirm stages at different times. The design hypothesis is that early identification of payments without order acknowledgment, using a shared reference, reduces time to useful investigation. The complaint does not reveal where the inconsistency originated.

I would start with the merchant contract: transaction identity, order reference, terminal states and an authorized query method. An absent order in customer history does not prove absence in the commercial system; reconciliation must seek evidence from authorized sources. The app should show what is confirmed and what is under review without making the customer arbitrate competing accounts between companies.

I would also investigate support continuity: who owned the case and when was the next update? Resolution with a zero score shows that financial recovery and the waiting experience deserve separate metrics. We cannot infer exact operating duration from the score, but we can define a better contract that prevents ownerless investigations.

EXTERNAL PROPOSAL · CASE 20

Investigation does not create refund authority



Proposed external architecture; technologies illustrate choices, not confirmed internal systems.

A pending case opens reconciliation and, if necessary, assisted investigation. Both seek the merchant's authorized decision. Only the financial writer applies the credit with a receipt and idempotent identity. Case queries receive the confirmed result to explain the outcome to the customer. People and the partner company have no software-component logos.

The diagram shows the refund branch; order acknowledgment may close the gap without a credit. The lab sends no real cancellation, implements no chargeback and determines no customer entitlement. Those paths require separate contracts and policies with appropriate auditability.

EXTERNAL PROPOSAL · CASE 20

Timeout opens a case, not a money movement

PYTHON / ORIGINAL PROPOSAL

```python
    def authorize_refund(self, merchant, payment_id, authorization_id):
        with self.lock:
            for value in (merchant, payment_id, authorization_id):
                identifier(value)
            if merchant != self.p.merchant or payment_id != self.p.payment_id:
                raise PermissionError('merchant/payment scope')
            if not authorization_id:
                raise ValueError('authorization required')
            if self.refund_authority not in (None, authorization_id):
                raise ValueError('conflicting authority')
            self.refund_authority = authorization_id

    def refund(self, actor, authorization_id, key):
        with self.lock:
            identifier(authorization_id)
            identifier(key)
            if actor != 'financial-writer':
                raise PermissionError('financial writer required')
            if not key or not authorization_id:
                raise ValueError('identity required')
            if self.refund_authority != authorization_id:
                raise PermissionError('merchant authorization missing')
            if self.credits:
                old_key, receipt = next(iter(self.credits.items()))
                if old_key != key:
                    raise ValueError('payment already refunded')
                return receipt
            receipt = (self.p.payment_id, self.p.cents, authorization_id)
            self.credits[key] = receipt
            self.state = 'REFUNDED'
            return receipt
```

Literal excerpt from the executed lab; imports and initialization are in the full file.

Refund authorization is bound to merchant and payment. The financial writer validates its own identity and the authorization before producing an effect. Replaying the same key returns the receipt; using another key after credit fails. A synthetic fifteen-minute window only opens an investigation and never creates refund permission.

The model implements one full refund, not installments or partial refunds. A late order acknowledgment after refund authorization requires reconciliation to avoid fulfillment incompatible with the decision. Real authority may include additional rules and other dispute paths; the lab is limited to the described flow.

Executable case lab on GitHub

EXTERNAL PROPOSAL · CASE 20

SQL for uniqueness and traceability

SQL / ORIGINAL PROPOSAL

```sql
CREATE TABLE refund_receipt (
  merchant_id TEXT NOT NULL,
  payment_id TEXT NOT NULL,
  authorization_id TEXT NOT NULL,
  idempotency_key TEXT NOT NULL,
  amount_cents BIGINT NOT NULL CHECK (amount_cents > 0),
  ledger_receipt_id TEXT NOT NULL,
  PRIMARY KEY (merchant_id, payment_id),
  UNIQUE (merchant_id, idempotency_key)
);

-- Insert must share the financial transaction or a verified ledger command.
-- A uniqueness constraint alone does not make an external credit atomic.
```

Illustrative SQL, unexecuted; this model allows one full refund only.

The constraint protects refund identity but does not make an external effect atomic. I would choose a transaction within financial authority or an idempotent command whose receipt can be queried after failure. Inserting a row and then calling a service without a retry contract would create another gap, precisely the kind this project aims to reduce.

The commercial key includes the merchant, preventing collisions between identical order numbers at different stores. Support queries authorized references rather than direct SQL. The financial receipt should explain how the credit appears to the customer without promising that commercial history will be deleted or rewritten.

EXTERNAL PROPOSAL · CASE 20

Order and payment retain their identities

Identity	Owner	Rule
payment_id	Payments	Immutable after capture
order_ref	Merchant	Bound to payment
authorization_id	Commercial authority	Scope and evidence
case_id	Operations	Next update

JSON / ORIGINAL PROPOSAL

```json
{
  "schema": "order-reconciliation.v1",
  "payment_id": "synthetic-payment",
  "merchant_ref": "synthetic-merchant",
  "order_ref": "synthetic-order",
  "payment_state": "CAPTURED",
  "order_state": "UNCONFIRMED",
  "case_state": "INVESTIGATING",
  "refund_authorization": null
}
```

Proposed synthetic contract, unexecuted; absent authorization is not permission.

I would keep financial and commercial states separate with occurrence and receipt timestamps. A generic failed status can hide a confirmed payment and encourage a repeat purchase. The API responds with state, available evidence and next action; it does not assign merchant blame from network silence.

Reconciliation uses amounts and references within a protected zone. Telemetry receives discrepancy category, age and version, without cart contents or unnecessary purchase details. Retention and access must support investigation and existing policies. An operator who edits the case does not automatically gain permission to issue credit.

EXTERNAL PROPOSAL · CASE 20

Prove the absence of an unauthorized credit

PYTHON / ORIGINAL PROPOSAL

```python
    def test_timeout_is_not_refund_authority(self):
        self.assertEqual(self.r.investigate(1000), 'INVESTIGATING')
        with self.assertRaises(PermissionError):
            self.r.refund('financial-writer', 'invented', 'a')
        self.assertEqual(self.r.credits, {})

    def test_cross_merchant_authority_is_rejected(self):
        with self.assertRaises(PermissionError):
            self.r.authorize_refund('shop-b', 'payment-a', 'grant')
        self.assertIsNone(self.r.refund_authority)

    def test_concurrent_refunds_credit_once(self):
        self.r.authorize_refund('shop-a', 'payment-a', 'grant')
        def refund(_):
            return self.r.refund('financial-writer', 'grant', 'a')
        with ThreadPoolExecutor(max_workers=4) as pool:
            results = list(pool.map(refund, range(20)))
        self.assertEqual(len(set(results)), 1)
        self.assertEqual(sum(v[1] for v in self.r.credits.values()), 5000)
```

Literal test excerpt; all 13 lab tests passed locally.

All thirteen tests passed. The suite includes timeout without authority, wrong merchant, late acknowledgment, an alternative key, an incorrect technical actor and an invalid clock. The concurrency test repeats one refund twenty times and verifies one credit. An acknowledged-order test followed by authorized cancellation distinguishes a legitimate refund from creating permission out of an absent order.

In the integrated environment I would interrupt the flow after the financial effect but before the local receipt to test recovery. I would also exercise duplicate authorization, a changed merchant reference and statement reconciliation. The lab simulates no commercial network and proves no actual refund deadline; each integration must confirm its guarantees and limits.

EXTERNAL PROPOSAL · CASE 20

Resolve both the money and the wait

Baseline is unknown. The first proposed metric is payments without commercial acknowledgment after an agreed window divided by valid captures, per merchant and day. The lab’s fifteen-minute window is educational; production must combine contract and observed behavior. Unanswered cases remain in the denominator until they receive a trustworthy state, preventing improvement merely through record expiry.

Time to first useful action runs from detection to an authorized query or owned handoff, in weekly cohorts. Kept updates measure cases notified by the promised deadline divided by cases due an update. Reconciled refunds measure confirmed credits divided by received authorizations, separate from merchant-authorization time. Duplicate credit per payment is the financial expansion-stopping guardrail.

Evaluation combines customer, operational and financial views. A smaller backlog achieved by closing pending cases without evidence is not an improvement. I would compare merchants with similar contracts and track old cases separately because aging may explain much of the effort an average hides.

EXTERNAL PROPOSAL · CASE 20

Capacity and exposure are different dimensions

The complaint discloses no purchase amount. I assume 500 monthly investigations, 25 manual minutes per case before the change, BRL 60 per loaded hour, BRL 2,000 monthly operations and BRL 60,000 implementation. These are planning assumptions, not a measured baseline. Capacity benefit is cases × minutes saved ÷ 60 × hourly cost.

Scenario	Minutes saved	Hours/month	Net value/month
Conservative	10	83.33	BRL 3,000
Base	15	125	BRL 5,500
Ambitious	20	166.67	BRL 8,000

Operating break-even requires four saved minutes per case. Recovering implementation would take 20, 10.91 or 7.5 months if all capacity became cash. Separately, assuming BRL 120 per case and two, five or eight fewer pending days, exposure decreases by 120,000, 300,000 or 480,000 BRL-days. That unit measures amount multiplied by uncertainty time; it is neither profit, recovered balance nor a benefit to add to the first.

Finance would validate marginal cost, and Operations would demonstrate that minutes were actually removed without reducing quality. I count neither refunded principal as revenue nor the zero score as monetized dissatisfaction. The project must justify its economics through a sample of actual work.

EXTERNAL PROPOSAL · CASE 20

A joint plan with the merchant

Days 0-15: Payments and Merchant Operations identify the contract, references and partner owners. The Staff engineer describes states and refund authority; Support defines the next update. Exit requires locating a synthetic payment and order without asking the customer to carry evidence between teams.

Days 16-30: backend implements a shadow detector and authorized queries; the partner validates fixtures and exceptions. Days 31-60: QA tests late authorization, duplicate capture, a lost receipt and reconciled billing. Exit is an owned investigation and one financial effect, without depending on a manual flag any operator can toggle.

Days 61-90: the EM coordinates a pilot with a few merchants, reserved support capacity and cost review. Payments accepts integrity, the partner accepts commercial semantics and Support accepts continuity. The horizon is proposed; absent query or authorization contracts prevent financial automation and keep the case on an assisted route.

EXTERNAL PROPOSAL · CASE 20

Operate aging cases

The shadow detector first measures false positives per merchant. A partner acknowledging orders in batches needs a different window from immediate acknowledgment; the rule must be explicit and approved. The initial cohort receives better tracking, followed only later by automation of steps with verifiable authority. Opened case volume must never exceed agreed investigation capacity.

Payments on-call receives credit inconsistencies; Merchant Operations receives absent partner responses; Support receives missed update promises. I would exercise a crash after refund and later arrival of commercial acknowledgment. Recovery queries receipts and prevents duplicate refunds while an authorized person resolves any fulfillment conflict.

Rollback suspends the detector or new execution while preserving cases and receipts. Old pending work does not disappear when a version leaves production. Daily review checks age, next action and owner; a case with no news may still need an honest customer update without inventing a deadline or assigning blame.

EXTERNAL PROPOSAL · CASE 20

My contribution: close the whole journey

As Staff, I would choose contracts that preserve identity across payment, order, authorization and receipt. I would reject automatically refunding every payment without an order after timeout: silence creates no authority and may cause duplicate credit or unintended free fulfillment. I would also reject calling integration complete merely because capture works; recovery belongs in the contract from the beginning.

As Engineering Manager, I would include the partner and Support in the definition of done, with a next-action owner and reserved exception capacity. Weekly review would show aging cases and blocked decisions, not only technical deliveries. I would negotiate scope to launch fewer merchants with verifiable recovery before expanding an integration that shifts its cost onto the customer.

This case demonstrates the responsibility I propose taking: connecting financial integrity, operations and experience beyond the happy path. I claim neither knowledge of the complaint's cause nor involvement in its resolution. The proposal's value is making explicit how a team would know the journey truly ended.

EXTERNAL PROPOSAL · CASE 20

Handoff: track through the final receipt

Run python3 labs/case_20.py -v. Start at Reconciler.investigate: crossing the window changes only investigation state. Attempt refund without authorization and confirm no credit. Then authorize through the correct merchant, execute concurrently and verify one receipt. Send a late commercial acknowledgment and observe the need for reconciliation without automatically reopening the commitment.

Acceptance includes merchant and payment isolation, verifiable authorization, idempotent writes, separate states, recovery after response loss and an owned next update. Persistence, integration signatures, a real ledger, commercial queries, statements, dispute governance and assisted operations remain missing. SQL and JSON are unexecuted illustrative contracts, and the model is limited to one full refund.

I would recommend proceeding if the partner can provide and reconcile evidence and the customer can track the case without retelling the story. I would stop if the plan depends on credit without authority or closing pending work to improve metrics. A resolved purchase needs a trustworthy financial outcome and communication the person can recognize.

Executable case lab on GitHub

LEADERSHIP AND EXECUTION

The first 30 days: learn the system.

I would begin by following a complete journey, including customer support and operations. The objective would not be to produce a perfect map of the organization, but to identify where the team encounters recurring decisions, which risks concern people most, and which commitments already exist with customers and partners. The first deliverable would be a bounded problem described in the language of the people experiencing it.

In individual conversations, I would explore expectations, friction points, perceived autonomy, and development opportunities. With Product and Design, I would understand the metrics in use and the reasoning behind the current roadmap. With Security, Risk, and Customer Support, I would investigate how exceptions reach engineering and how much context is lost along the way. The diagnosis should be shared and open to correction.

Deliverable	How to produce it	Sign of usefulness
Journey map	Follow the app, service, and support	The team recognizes boundaries and gaps.
Baseline	Define events and denominators	The number can be reproduced.
First problem	Choose it with the team	Clear owner and completion criterion.

Progression criterion

Propose a structural change only when its cause is sufficiently understood and the affected people have participated in defining the problem.

LEADERSHIP AND EXECUTION

Days 31-60: make the hypothesis testable.

With initial context established, I would choose a pilot that produces evidence within a few weeks and can be rolled back. For example, improving a status inquiry for an existing operation may be more controllable than changing an entire payment flow. The scope needs to include observability, support, and communication, because a change that is invisible to operations transfers work elsewhere.

The team would define one primary hypothesis, two or three guardrail metrics, and an exposure limit. The plan must specify which events end the pilot, who can pause it, and what happens to customers who have already entered the journey. Before launch, I would ask for a failure rehearsal with the rollback path, including compatibility between versions.

Decision	Artifact	Proposed owner
Can we start?	Reviewed contract and baseline	Tech lead + Product + Risk.
Can we expand?	Sample and guardrails	Journey owner.
Do we need to stop?	Observable trigger and runbook	Operational owner.

How I would contribute

As an EM, I would remove dependencies and protect focus. As a Staff engineer, I would examine contracts and failure modes in depth. In both roles, I would aim for the team to be able to explain the decision without depending on me.

LEADERSHIP AND EXECUTION

Days 61-90: decide what deserves to scale.

At the end of the pilot, the work does not finish with a results presentation. We need to decide whether the hypothesis was supported, whether operations can sustain the solution, and whether the cost of expansion still makes sense. A localized improvement can reveal data problems, unserved segments, or dependencies that prevent replication without additional investment.

I would present the decision as three alternatives: expand, revise, or end. Expansion requires a plan for capacity, ownership, support, and compatibility. Revision requires a specific uncertainty to resolve. Ending preserves the learning and removes the ongoing cost of a hypothesis that did not hold. None of these outcomes should be treated as a personal failure.

Alternative	Required evidence	Next action
Expand	Consistent benefit and controlled risks	Expand by cohort with observation.
Revise	Promising signal, bounded uncertainty	A new time-bound experiment.
End	Insufficient benefit or excessive cost	Remove the experiment and record the learning.

The 90-day outcome

A team with more shared context, a verifiable improvement or an informed decision not to expand, and practices that make the next cycle more efficient.

LEADERSHIP AND EXECUTION

Capacity is a design constraint.

Planning as if all team time were available for features makes every incident an unexpected delay. I would separate capacity for evolution, operations, consequential debt, and learning, using data from the actual flow of work to adjust the allocation. The initial proportion is a management hypothesis, not a fixed rule imposed on every team.

The capacity discussion needs to account for knowledge concentration. If only one person can operate a critical integration, the team has a risk that does not appear in story points. Pairing, documentation, and progressive rotation can reduce that dependency, but they need space in the plan and quality monitoring, not just a ceremony.

Signal	Interpretation to investigate	Possible action
Many interruptions	Support requests arriving without triage	A channel with context and severity.
Aging work	Unresolved dependency or scope	Reduce batch size and decide how to address the blocker.
One person involved in everything	Concentrated knowledge	Pairing on an actual delivery.

As an EM, I would make these constraints visible when negotiating timelines. As a Staff engineer, I would help reduce the complexity that generates interruptions. Promising speed without addressing the system of work tends to exhaust the most experienced people and leave the team less capable in the next cycle.

LEADERSHIP AND EXECUTION

Delegation with explicit boundaries.

Good delegation does not mean handing over a problem and disappearing. It means agreeing on the expected outcome, decision space, risks that require discussion, and check-in points. Someone who is developing can lead an important change when scope and support are appropriate. The leader remains responsible for creating the conditions for learning without invisible risks.

I would use real decisions as growth opportunities: one person leads the contract, another conducts the failure test, and another presents alternatives to stakeholders. Feedback should identify observable behavior and its effect. “Be more strategic” is vague; “explain the consequence of each alternative before choosing” allows practice and evaluation.

Delegation	Autonomy	When to escalate
Local component	Implementation within the contract	State change or sensitive data.
Cross-team contract	Proposal and negotiation	Incompatibility or disputed authority.
Rollout	Execution within the limits	Guardrail or insufficient sample.

Evidence of leadership

Success appears when more people can formulate, explain, and execute important decisions well, not when every decision goes through the manager.

LEADERSHIP AND EXECUTION

An ADR resolves a concrete uncertainty.

An architecture decision record should preserve the reasoning that will be needed later. I would avoid documents that merely describe the selected solution and omit the problem. The first page needs to explain context, constraints, alternatives, and the consequence of being wrong. Readers should be able to disagree with a premise before discussing the tool.

Part	Question	Example in the book
Context	Which decision is pending?	How to represent an unknown outcome.
Alternatives	What could we do?	Authoritative query, projection, or waiting.
Choice	Why this option now?	Lower risk with current capacity.
Review	When should we reopen the decision?	Delay, cost, or load beyond the limit.

The ADR should have an owner and an event-triggered review date, rather than relying only on the calendar. If the decision was made under deadline pressure, that should be visible. If an alternative was rejected because operational capacity was lacking, the rationale must survive the departure of the people who participated in the conversation.

As a Staff engineer, I would use the document to create alignment and expose trade-offs. As an EM, I would protect the time needed to decide and prevent the ADR from becoming a mandatory bureaucratic step for trivial changes. The degree of formality should match reach, reversibility, and risk.

LEADERSHIP AND EXECUTION

Incidents: restore, understand, improve.

During an incident, the priority is to reduce impact through clear coordination. I would separate incident command, technical investigation, and communication so that the same person does not have to do everything. Updates should state what is known, what remains uncertain, and which action is underway, without promising a timeline unsupported by evidence.

After stabilization, the review needs to examine system conditions: missing signals, reasonable decisions made with incomplete information, dependencies, and recovery mechanisms. Searching for someone to blame produces silence; ignoring accountability prevents improvement. The useful path is to turn what happened into verifiable changes, with owners, priority, and an effectiveness test.

Phase	Main question	Output
Response	How can we reduce harm now?	Coordinated mitigation and communication.
Reconciliation	Are there pending or divergent effects?	Verified operations and handled cases.
Learning	What makes recurrence less likely?	A testable action with an owner.

Effectiveness test

A postmortem action is complete only when we can demonstrate that it detects, prevents, or reduces the impact of the scenario that motivated it.

LEADERSHIP AND EXECUTION

Influence across teams without centralizing everything.

Journey problems rarely fit within a single repository. The interface may depend on another domain's data, while support absorbs the impact of an unclear contract. I would begin by making each team's incentives and boundaries visible. A solution that improves one team's metric while transferring cost to another needs renegotiation.

Technical influence comes from reducing the cost of deciding: a small prototype, reproducible data, comparable alternatives, and a migration proposal that respects the capacity of those involved. In disagreement, I would distinguish facts, preferences, and constraints. Not every conflict requires consensus; it needs a legitimate decision-maker and an understandable explanation.

Boundary	Required agreement	How to verify
Producer / consumer	Version and compatibility	Fixtures and a contract test.
Engineering / customer support	State and next action	Simulation of a realistic case.
Product / risk	Benefit and exposure limit	Guardrails and cohort review.

The outcome I seek is an organization that can change with fewer surprises. That depends on useful standards, clear ownership, and channels for challenging decisions when new evidence emerges. The Staff engineer facilitates this system; the EM ensures that people and capacity keep pace with the ambition.

ENGINEERING ECONOMICS

Value: start with the denominator.

Public financial context helps calibrate a discussion; it cannot supply the missing denominator of a complaint. Nu reported approximately 139million global customers, 83.5% monthly activity and approximately US$17 monthly ARPAC for Q2 2026. These are company-level figures for the quarter ended 30 June, not measurements of the twenty affected cohorts.

Value category	Calculation	Evidence required
Customer liquidity	Principal released or time without access.	Confirmed amount and settlement; never bank profit.
Operational capacity	Hours avoided × measured loaded cost.	Causal reduction, usable capacity and cost actually avoided.
Incremental contribution	Eligible volume × causal uplift × contribution margin.	Valid counterfactual, marginal economics and recurring cost.
Payback	Initial investment / positive monthly net benefit.	No payback when recurring benefit is nonpositive.

For perspective, retaining 100 additional active customers for 12 months at the approximate group ARPAC would correspond to US$20,400 of gross revenue. The 100 customers, causal retention and 12-month survival are assumptions; ARPAC is an average proxy, not incremental margin. Credit losses, funding, servicing, build and operating costs still have to be deducted. This illustration is not a forecast and is not applied to NuCel.

Investment decision

Each chapter exposes its own assumptions and sensitivity. Do not add the twenty scenarios: populations and benefits can overlap. A negative scenario is a reason to narrow or stop the initiative.

Nu Holdings · Q2 2026 results · 13 August 2026

ENGINEERING ECONOMICS

Opportunity cost is also a cost.

The choice between two proposals should not rely solely on projected gross benefit. One initiative may require a broad migration, partner coordination, and ongoing maintenance. Another may produce less benefit in isolation and still be the better first delivery because it reduces risk, validates an assumption, or frees capacity for the next step.

Synthetic alternative	Expected benefit	Operational commitment
New global projection	Faster reads	Ongoing pipeline, reconciliation, and support.
Contract correction	Less ambiguity	Coordinated change and tests.
Content improvement	Clearer guidance	Editorial governance and measurement.

I would compare alternatives using the same time horizon and the same journey outcome. I would also separate reversible costs from commitments that are difficult to undo. Introducing another service creates responsibility for availability, updates, security, observability, and knowledge transfer. The architecture needs to bring that cost into the initial discussion.

As an EM, I would make the trade-off with Product and stakeholders using actual capacity. As a Staff engineer, I would seek to reduce the technical cost of the selected alternative. The best solution may be a new component, a small change to an existing contract, or a decision not to build until there is sufficient evidence.

ENGINEERING ECONOMICS

Performance: measure the tail and the task.

Average latency can conceal the experience of the most affected people. A fast endpoint may also fail to improve the journey if the app repeats calls, waits on a slow dependency, or presents a confusing state. I would start with a complete timeline and measure the step at which users perceive waiting, failure, or a need to repeat work.

Measure	Question it answers	Limit
P50 / P95 / P99	How is waiting time distributed?	Does not measure completion by itself.
Confirmation rate	Did the task finish?	Depends on complete events.
Subsequent contacts	Did the person remain uncertain?	There is a delay and there are other causes.
Cost per journey	Which resource was consumed?	Requires consistent attribution.

The comparison should use similar cohorts, sufficient volume, and a window that includes relevant peaks. Cache optimization can reduce latency while worsening freshness; parallelism can improve time while increasing saturation; retries can increase completion while amplifying load. Every gain must be considered alongside its guardrail metrics.

Decision criterion

I would expand an optimization when the task benefit is supported and correctness, data freshness, and operating cost remain within the agreed limits.

ENGINEERING ECONOMICS

Risk: measure both types of error.

A fraud-prevention decision has asymmetric effects. Allowing an improper action can generate a loss; blocking a legitimate action can prevent a real need from being met and create recovery costs. The discussion cannot treat every reduction in approval as a security improvement, or every reduction in friction as a product improvement.

Dimension	Proposed measure	Considerations
Protection	Loss in the eligible population	Delayed labels and underreporting.
Friction	Legitimate actions blocked / legitimate actions evaluated	Ground truth and review.
Recovery	Time to a safe resolution	Do not exclude silent abandonment.
Operational fairness	Performance by relevant cohort	Sample and data access.

The model needs a challenge pathway and human review proportionate to the risk. Review outcomes feed learning, but may also contain selection bias: people who give up do not appear as resolved cases. I would seek to measure that absence and discuss decision criteria with risk and product specialists.

My fraud-prevention experience makes this balance central to my candidacy. In the ML and access-recovery cases, the book separates signal, policy, effect, and review. The objective is to protect the person and the institution through decisions that can be explained and corrected.

LABORATORY GUIDE

How to run the laboratories.

The companion package contains one independent laboratory per case. They use Python's standard library and synthetic data, with no credentials, calls to Nubank, or account access. The purpose is to make rules and failures examinable on a local machine. Every file includes its own tests and can be run independently.

BASH / ORIGINAL PROPOSAL

```bash
# Dentro da pasta labs do pacote extraído:
python3 case_01.py
python3 case_17.py

# Todos os casos em uma única execução:
python3 -m unittest discover -p "case_*.py" -v

# Núcleo Clojure, com Clojure 1.12.0 instalado:
clojure -M decision_core.clj
```

Local commands; they do not access banking services. Consult LEIA-ME.html for versions and scope.

The Clojure laboratory was executed with Java 17 and Clojure 1.12.0. The package provides the original code, the reproduction command, and the execution record. Flutter/Dart, SQL, YAML, and cloud integration examples are identified as references where they were not executed. The book does not present absent compilation as validation.

What to test next

Integration with real adapters, concurrency across processes, durable persistence, load, network failures, authorization, and operations belong to a subsequent stage in an appropriate environment.

LABORATORY GUIDE

From local proof to an integrated pilot.

A passing unit test is evidence of a property in the context where it was executed. Moving to a pilot requires new questions. Does the adapter preserve the core's semantics? Does persistence perform the version comparison and write atomically? Does a retry arrive with the same identity? Does the consumer remain correct when the process restarts?

Level	Validation scenario	Evidence to retain
Contract	An old consumer receives a new event	Fixture, version, and result.
Integration	Failure after commit and before ACK	Final state and recovery trail.
Load	A peak with a slow dependency	Latency, saturation, and losses.
Operations	Rollback with customers in progress	Procedure and reconciliation.

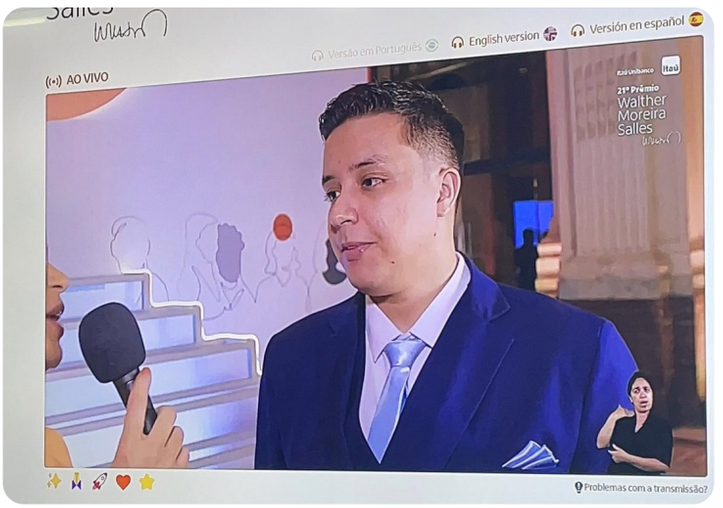
I would maintain a matrix connecting risk, testing, and the release decision. We do not need to test every imaginable combination before learning something; we need to cover the properties that prevent harm within the chosen exposure scope. The pilot should be smaller when the evidence is weaker or rollback is more difficult.

Integrity criterion

No local result in this book is extrapolated to Nubank's actual availability, security, or performance. The proposals become stronger when their limits are known and can guide the next validation.

CLOSING

A proposal for contribution, in my own words.



Public record of the candidate being interviewed during a recognition event, published in his portfolio.

I want to work on problems that require depth and collaboration. I enjoy getting down to the contract and the code, while also building context so that other people can make decisions. My experience in fraud, credit, and technical product work has taught me that quality appears in exceptions as much as in the happy path.

At Nubank, I want to contribute to teams that turn financial complexity into understandable experiences. As an EM, my focus will be developing people and delivery capacity. As a Staff engineer, it will be improving technical decisions that cross domains. In both roles, I want to take responsibility, learn from the existing context, and make the outcome verifiable.

Let’s discuss the real problem.

Wesley Vieira · LinkedIn: /in/techvieira · Portfolio: wesleyvieira99.github.io/WVPortfolio · GitHub: wesleyvieira99

TRACEABLE BIBLIOGRAPHY

Sources for cases 01-05

01 · BRL 300.00 Pix: clarify the refund

Individual complaint with the company's public response · 2026-06-03 Reclame AQUI - public complaint and response

Historical report; the chapter includes the outcome and evidential limits. It does not establish incidence, technical cause or a current defect.

02 · Incoming Pix, stale balance

News report with the company's response · 2026-07-08 Canaltech - July 8, 2026 incident

Historical report; the chapter includes the outcome and evidential limits. It does not establish incidence, technical cause or a current defect.

03 · Account restriction: review and exit

News report with the company's response · 2026-03-10 Metrópoles - restriction and reported outcome

Historical report; the chapter includes the outcome and evidential limits. It does not establish incidence, technical cause or a current defect.

04 · Broken camera, secure recovery

Individual complaint with the company's public response · 2025-10-31 Reclame AQUI - public complaint and response

Historical report; the chapter includes the outcome and evidential limits. It does not establish incidence, technical cause or a current defect.

05 · New phone, missing verification code

Individual complaint with the company's public response · 2026-03-06 Reclame AQUI - public complaint and response

Historical report; the chapter includes the outcome and evidential limits. It does not establish incidence, technical cause or a current defect.

01 · Reclame AQUI - public complaint and response
02 · Canaltech - July 8, 2026 incident
03 · Metrópoles - restriction and reported outcome
04 · Reclame AQUI - public complaint and response
05 · Reclame AQUI - public complaint and response

TRACEABLE BIBLIOGRAPHY

Sources for cases 06-10

06 · New evidence during a dispute

Individual complaint with the company's public response · 2026-02-10 Reclame AQUI - public complaint and response

Historical report; the chapter includes the outcome and evidential limits. It does not establish incidence, technical cause or a current defect.

07 · MED: track progress without promising a refund

Individual complaint with the company's public response · 2025-10-02 Reclame AQUI - public complaint and response

Historical report; the chapter includes the outcome and evidential limits. It does not establish incidence, technical cause or a current defect.

08 · Disputed loan: a decision must remain reviewable

Individual complaint with the company's public response · 2026-01-14 Reclame AQUI - case 08 · 2026-01-14

Historical report; the chapter includes the outcome and evidential limits. It does not establish incidence, technical cause or a current defect.

09 · Paid statement, uncertain credit: follow every cent

Individual complaint with the company's public response · 2026-03-10 Reclame AQUI - case 09 · 2026-03-10

Historical report; the chapter includes the outcome and evidential limits. It does not establish incidence, technical cause or a current defect.

10 · NuPay after an error message: recover intent before retrying

Individual complaint with the company's public response · 2026-02-27 Reclame AQUI - case 10 · 2026-02-27

Historical report; the chapter includes the outcome and evidential limits. It does not establish incidence, technical cause or a current defect.

06 · Reclame AQUI - public complaint and response
07 · Reclame AQUI - public complaint and response
08 · Reclame AQUI - case 08 · 2026-01-14
09 · Reclame AQUI - case 09 · 2026-03-10
10 · Reclame AQUI - case 10 · 2026-02-27

TRACEABLE BIBLIOGRAPHY

Sources for cases 11-15

11 · A refund in three entries: the statement must explain its own sum

Individual complaint with the company's public response · 2025-03-31 Reclame AQUI - case 11 · 2025-03-31

Historical report; the chapter includes the outcome and evidential limits. It does not establish incidence, technical cause or a current defect.

12 · Delivered in tracking, missing in hand: separate delivery from possession

Individual complaint with the company's public response · 2025-10-23 Reclame AQUI - case 12 · 2025-10-23

Historical report; the chapter includes the outcome and evidential limits. It does not establish incidence, technical cause or a current defect.

13 · A virtual card cap is not available credit

Individual complaint with the company's public response · 2026-03-30 Reclame AQUI - case 13 · 2026-03-30

Historical report; the chapter includes the outcome and evidential limits. It does not establish incidence, technical cause or a current defect.

14 · Early repayment received, execution delayed: make the plan verifiable

Individual complaint with the company's public response · 2026-01-23 Reclame AQUI - case 14 · 2026-01-23

Historical report; the chapter includes the outcome and evidential limits. It does not establish incidence, technical cause or a current defect.

15 · Money Boxes: visible money, reliable withdrawal

Individual complaint with the company's public response · 2026-04-28 Reclame AQUI · 247218209

Historical report; the chapter includes the outcome and evidential limits. It does not establish incidence, technical cause or a current defect.

11 · Reclame AQUI - case 11 · 2025-03-31
12 · Reclame AQUI - case 12 · 2025-10-23
13 · Reclame AQUI - case 13 · 2026-03-30
14 · Reclame AQUI - case 14 · 2026-01-23
15 · Reclame AQUI · 247218209

TRACEABLE BIBLIOGRAPHY

Sources for cases 16-20

16 · Portability: eligible does not mean overdue

Individual complaint with the company's public response · 2026-06-01 Reclame AQUI · 250210183

Historical report; the chapter includes the outcome and evidential limits. It does not establish incidence, technical cause or a current defect.

17 · Ultravioleta: one choice, one balance

Individual complaint with the company's public response · 2026-04-03 Reclame AQUI · 245086475

Historical report; the chapter includes the outcome and evidential limits. It does not establish incidence, technical cause or a current defect.

18 · NuCel: confirmation across two carriers

Individual complaint with the company's public response · 2026-07-17 Reclame AQUI · 254143071

Historical report; the chapter includes the outcome and evidential limits. It does not establish incidence, technical cause or a current defect.

19 · Global Account: purpose before FX

Individual complaint with the company's public response · 2026-03-11 Reclame AQUI · 243002037

Historical report; the chapter includes the outcome and evidential limits. It does not establish incidence, technical cause or a current defect.

20 · NuPay: confirmed payment, missing order

Individual complaint with the company's public response · 2026-07-28 Reclame AQUI · 255046021

Historical report; the chapter includes the outcome and evidential limits. It does not establish incidence, technical cause or a current defect.

16 · Reclame AQUI · 250210183
17 · Reclame AQUI · 245086475
18 · Reclame AQUI · 254143071
19 · Reclame AQUI · 243002037
20 · Reclame AQUI · 255046021

VISUAL PROVENANCE

Image and brand credits

Logos and photographs were obtained from the official project pages, Nubank publications or the candidate’s public portfolio. Trademarks identify the technologies discussed. Editorial use does not imply an affiliation or endorsement. The package includes a provenance manifest with file URLs, source pages and asset hashes. Source screenshots are kept in their original language to preserve the evidence.

Asset	Source	Use
nubank	www.datomic.com	Technology identification
clojure	clojure.org	Technology identification
datomic	www.datomic.com	Technology identification
kafka	kafka.apache.org	Technology identification
prometheus	prometheus.io	Technology identification
flutter	flutter.dev	Technology identification
dart	dart.dev	Technology identification
python	www.python.org	Technology identification
kubernetes	github.com	Technology identification

nubank · source
clojure · source
kafka · source
prometheus · source
flutter · source
dart · source
python · source
kubernetes · source

VISUAL PROVENANCE

Image and brand credits · continued

Logos and photographs were obtained from the official project pages, Nubank publications or the candidate's public portfolio. Trademarks identify the technologies discussed. Editorial use does not imply an affiliation or endorsement. The package includes a provenance manifest with file URLs, source pages and asset hashes. Source screenshots are kept in their original language to preserve the evidence.

Asset	Source	Use
opentelemetry	opentelemetry.io	Technology identification
postgresql	www.postgresql.org	Technology identification
grafana	grafana.com	Technology identification
java	dev.java	Technology identification
portrait	wesleyvieira99.github.io	Editorial / career photograph
award-0	wesleyvieira99.github.io	Editorial / career photograph
award-1	wesleyvieira99.github.io	Editorial / career photograph
award-2	wesleyvieira99.github.io	Editorial / career photograph
aws	aws.amazon.com	Technology identification

opentelemetry · source
postgresql · source
grafana · source
java · source
portrait · source
aws · source

VISUAL PROVENANCE

Image and brand credits · continued

Logos and photographs were obtained from the official project pages, Nubank publications or the candidate's public portfolio. Trademarks identify the technologies discussed. Editorial use does not imply an affiliation or endorsement. The package includes a provenance manifest with file URLs, source pages and asset hashes. Source screenshots are kept in their original language to preserve the evidence.

Asset	Source	Use
redis	redis.io	Technology identification
typescript	www.typescriptlang.org	Technology identification
nu-app-life	international.nubank.com.br	Editorial / career photograph
nu-engineer	international.nubank.com.br	Editorial / career photograph
nu-meeting	international.nubank.com.br	Editorial / career photograph
nu-office	international.nubank.com.br	Editorial / career photograph
nu-collaboration	international.nubank.com.br	Editorial / career photograph
terraform	github.com	Technology identification

redis · source
typescript · source
nu-app-life · source
terraform · source